Table of Contents

EHR Information Model

Amendment Record

Acknowledgements

- Editor
- Contributors
- Support
- Trademarks

1. Preface
   - 1.1. Purpose
   - 1.2. Related Documents
   - 1.3. Status
   - 1.4. Feedback
   - 1.5. Conformance
2. Background
   - 2.1. Requirements
     - 2.1.1. Original GEHR Requirements
     - 2.1.2. GEHR Australia Requirements
     - 2.1.3. European Synapses and SynEx Project Requirements
     - 2.1.4. European EHCR Support Action Requirements
     - 2.1.5. ISO EHR Requirements
     - 2.1.6. openEHR Requirements
   - 2.2. Relationship to other Health Information Models
     - 2.2.1. ISO 13606
     - 2.2.2. HL7 Version 3
     - 2.2.3. OMG HDTF
3. The EHR Information Model
   - 3.1. Overview
4. EHR Package
   - 4.1. Overview
   - 4.2. The Parts of the EHR
     - 4.2.1. Root EHR Object
     - 4.2.2. EHR Access
     - 4.2.3. EHR Status
     - 4.2.4. Compositions
       - 4.2.4.1. Persistent Compositions
       - 4.2.4.2. Event Compositions
       - 4.2.4.3. Episodic Compositions

- 4.2.5. Folders
- 4.3. Change Control in the EHR
  - 4.3.1. Versioning of Compositions
  - 4.3.2. Versioning Scenarios
- 4.4. EHR Creation Semantics
  - 4.4.1. EHR Identifier Allocation
  - 4.4.2. EHR Creation
- 4.5. EHR Active Status
- 4.6. Time in the EHR
- 4.7. Historical Views of the Record
- 4.8. Class Descriptions
  - 4.8.1. EHR Class
  - 4.8.2. VERSIONED_EHR_ACCESS Class
  - 4.8.3. EHR_ACCESS Class
  - 4.8.4. VERSIONED_EHR_STATUS Class
  - 4.8.5. EHR_STATUS Class
  - 4.8.6. VERSIONED_COMPOSITION Class
- 5. Composition Package
  - 5.1. Overview
  - 5.2. Context Model of Recording
    - 5.2.1. Overview
    - 5.2.2. Composer
    - 5.2.3. Event Context
      - 5.2.3.1. Overview
      - 5.2.3.2. Occurrence in Data
      - 5.2.3.3. Time
      - 5.2.3.4. Participations
      - 5.2.3.5. Healthcare Facility, Location and Setting
  - 5.3. Composition Content
  - 5.4. Class Descriptions
    - 5.4.1. COMPOSITION Class
    - 5.4.2. EVENT_CONTEXT Class
- 6. Content Package
  - 6.1. Overview
  - 6.2. Class Descriptions
    - 6.2.1. CONTENT_ITEM Class
- 7. Navigation Package
  - 7.1. Overview

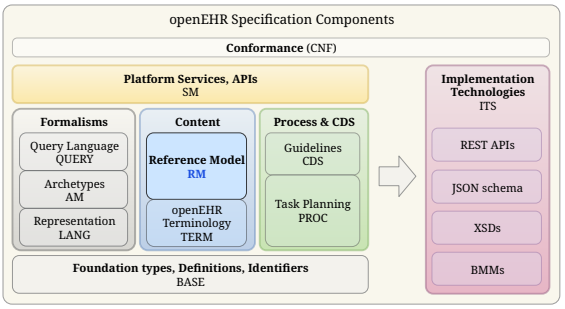
- 7.2. Class Descriptions
  - 7.2.1. SECTION Class
- 7.3. Section Instance Structures
  - 7.3.1. Problem/SOAP Headings
- 8. Entry Package
  - 8.1. Design Principles
    - 8.1.1. Information Ontology
    - 8.1.2. Clinical Statement Status and Negation
    - 8.1.3. Standard Clinical Types of Data
    - 8.1.4. Demographic Data in the EHR
  - 8.2. Entry and its Subtypes
    - 8.2.1. The Entry class
    - 8.2.2. Care_entry and Admin_entry
    - 8.2.3. Observation
      - 8.2.3.1. Timing in Observations
      - 8.2.3.2. Valid Contents of a History
      - 8.2.3.3. Clinical Semantics of Event Time
      - 8.2.3.4. Two ways of Recording State
    - 8.2.4. Evaluation
    - 8.2.5. Instruction and Action
      - 8.2.5.1. Requirements
      - 8.2.5.2. Design Principles
      - 8.2.5.3. Model Overview
      - 8.2.5.4. Instruction Activity Timing
      - 8.2.5.5. The Standard Instruction State Machine (ISM)
      - 8.2.5.6. Instruction Aggregate State
      - 8.2.5.7. Careflow Process to State Machine Mapping
      - 8.2.5.8. Relationship to Archetypes
    - 8.2.6. Clinical Workflow Definition
  - 8.3. Class Descriptions
    - 8.3.1. ENTRY Class
    - 8.3.2. ADMIN_ENTRY Class
    - 8.3.3. CARE_ENTRY Class
    - 8.3.4. OBSERVATION Class
    - 8.3.5. EVALUATION Class
    - 8.3.6. INSTRUCTION Class
    - 8.3.7. ACTIVITY Class
    - 8.3.8. ACTION Class

8.3.9. INSTRUCTION_DETAILS Class
8.3.10. ISM_TRANSITION Class
8.4. Instance Structures
8.4.1. OBSERVATION
8.4.1.1. Heartrate Measurement Series
8.4.1.2. Blood Pressure with Protocol
8.4.1.3. Glucose Tolerance Test
8.4.2. EVALUATION
8.4.2.1. Partial Asthma Management Plan
8.4.3. INSTRUCTION
8.4.3.1. Chained Medication Order
8.4.3.2. Multi-drug Therapy
Appendix A: Glossary
References

openEHR

EHR Information Model

Issuer: openEHR Specification Program (https://www.openehr.org/programs/specification)	
Release: RM Release-1.1.0	**Status**: STABLE
Revision: [latest_issue]	**Date**: [latest_issue_date]
Keywords: EHR, EMR, reference model, openehr	



© 2003 - 2021 The openEHR Foundation

The openEHR Foundation (https://www.openehr.org) is an independent, non-profit foundation, facilitating the sharing of health records by consumers and clinicians via open specifications, clinical models and open platform implementations.

Licence	Creative Commons Attribution-NoDerivs 3.0 Unported. https://creativecommons.org/licenses/by-nd/3.0/
Support	Issues: Problem Reports (https://specifications.openehr.org/components/RM/open_issues) Web: specifications.openEHR.org (https://specifications.openehr.org)

Amendment Record

Issue	Details	Raiser	Completed
	RM Release 1.1.0		
5.2.0	SPECRM-55 (https://specifications.openehr.org/tickets/SPECRM-55). Add `folders:LIST<OBJECT_REF>` on `EHR`; make `directory` refer to first member of `folders`. See Section 4.2.5.	B Naess, S Iancu	23 Sep 2020
	SPECPUB-7 (https://specifications.openehr.org/tickets/SPECPUB-7): Convert citations to bibtex form.	T Beale	15 Dec 2019
	SPECRM-92 (https://specifications.openehr.org/tickets/SPECRM-92). Correct numerous minor typographical and formatting errors.	J Mewes	09 Nov 2019
	SPECRM-89 (https://specifications.openehr.org/tickets/SPECRM-89). Add 'episodic' type to `COMPOSITION.category`. Update text and diagrams in sections 4.2.4, 4.3.	P Pazos, I McNicoll	01 Nov 2019
	SPECRM-56 (https://specifications.openehr.org/tickets/SPECRM-56). Add archetypable details meta-data to `FOLDER`. (addresses SPECPR-85 (https://specifications.openehr.org/tickets/SPECPR-85)).	B Naess, S Iancu, P Pazos, I McNicoll, T Beale	08 Oct 2019
	SPECRM-88 (https://specifications.openehr.org/tickets/SPECRM-88). Improve documentation relating to use of `uid` in versioning and `LOCATABLE` descendants in `change_control` package; improve advice on setting `uid` in `COMPOSITION`, `EHR_STATUS` and `EHR_ACCESS`. (Addresses SPECPR-322 (https://specifications.openehr.org/tickets/SPECPR-322)).	P Pazos, M Polajnar, S Arikan, T Beale	29 Sep 2019

Issue	Details	Raiser	Completed
	SPECRM-85 (https://specifications.openehr.org/tickets/SPECRM-85). Correct typographical errors: 'planned_step' should be 'plan_step' in section 8.2.5.7. Careflow Process to State Machine Mapping (see SPECPR-204 (https://specifications.openehr.org/tickets/SPECPR-204)).	P Pazos	22 May 2019
	RM Release 1.0.4		
5.1.3	SPECRM-84 (https://specifications.openehr.org/tickets/SPECRM-84). Improve documentation of `EHR_STATUS.`*`is_modifiable`* and EHR active status (see SPECPR-284 (https://specifications.openehr.org/tickets/SPECPR-284)). Added new sub-section "EHR Active Status" in section 4.	M Asgari, I McNicoll, T Beale	28 Dec 2018
	SPECRM-71 (https://specifications.openehr.org/tickets/SPECRM-71). Make `ACTIVITY.`*`timing`* optional. SPECRM-68 (https://specifications.openehr.org/tickets/SPECRM-68). Improve guidance on use of `ACTIVITY.`*`timing`* (Fixes SPECPR-242 (https://specifications.openehr.org/tickets/SPECPR-242). SPECRM-69 (https://specifications.openehr.org/tickets/SPECRM-69). Add missing ISM transitions and improve documentation of states (Fixes SPECPR-203 (https://specifications.openehr.org/tickets/SPECPR-203).	P Pazos	26 Nov 2018
	SPECRM-80 (https://specifications.openehr.org/tickets/SPECRM-80). Improve description of *`system_id`* field in `EHR`, `AUDIT_DETAILS` and `FEEDER_AUDIT_DETAILS`.	H Frankel, T Beale	

Issue	Details	Raiser	Completed
	SPECRM-59 (https://specifications.openehr.org/tickets/SPECRM-59). Add identifier recommendations to EHR spec(s).	B Lah	23 Apr 2018
	SPECRM-52 (https://specifications.openehr.org/tickets/SPECRM-52). Remove `COMPOSITION` invariant preventing context for persistent Compositions. (Fixes SPECPR-78 (https://specifications.openehr.org/tickets/SPECPR-78)) Update Clinical Investigator Recording Process diagram.	H Frankel, S Heard	07 Feb 2017
	SPECPUB-3 (https://specifications.openehr.org/tickets/SPECPUB-3). Re-instate inheritance from descendants of `VERSIONED_OBJECT<T>` to `VERSIONED_OBJECT` from Release 1.0.2 (lost in original UML conversion). Correct `EVENT_CONTEXT.`*`health_care_facility`* and *`participations`* links that were mangled in initial UML conversion.	T Beale	07 Jun 2016
	SPECRM-46 (https://specifications.openehr.org/tickets/SPECRM-46). Fix RM Release 1.0.3 typos and minor documentary errors. Correct `ACTION.`*`description`* text to past tense. (SPECPR-77 (https://specifications.openehr.org/tickets/SPECPR-77)).	P Pazos	04 Apr 2016
RM Release 1.0.3			

Issue	Details	Raiser	Completed
5.1.2	SPECRM-28 (https://specifications.openehr.org/tickets/SPECRM-28). Corrections to EHR and Common IM documentation. Improve explanation in section 4.4.2 (EHR Creation) of `EHR_STATUS.is_modifiable` (SPECPR-28 (https://specifications.openehr.org/tickets/SPECPR-28)). Fix description of `CARE_ENTRY.guideline_id` (SPECPR-75 (https://specifications.openehr.org/tickets/SPECPR-75)). Fix typo in EHR IM in section 'Standard Clinical Types of Data' (SPECPR-87 (https://specifications.openehr.org/tickets/SPECPR-87)). EHR IM missing example in section 'Care_entry and Admin_entry'. (SPECPR-89 (https://specifications.openehr.org/tickets/SPECPR-89)).	C Ma, P Pazos	10 Oct 2015
	SPECRM-37 (https://specifications.openehr.org/tickets/SPECRM-37): Add optional `reason: List<DV_TEXT>` attribute to `ISM_TRANSITION` class (SPECPR-58 (https://specifications.openehr.org/tickets/SPECPR-58)).	S Heard	
	SPECRM-43 (https://specifications.openehr.org/tickets/SPECRM-43): Improve documentation of `EHR.system_id` in clone/copy scenario (SPECPR-128 (https://specifications.openehr.org/tickets/SPECPR-5128)).	S Heard	
Release 1.0.2			
5.1.1	SPEC-274 (https://specifications.openehr.org/tickets/SPEC-274). Observation should be Evaluation in problem/SOAP structure figure.	R Chen	16 Aug 2008
	SPEC-275 (https://specifications.openehr.org/tickets/SPEC-275): Update Entry package design principles in EHR IM.	T Beale	

Issue	Details	Raiser	Completed
	SPEC-253 (https://specifications.openehr.org/tickets/SPEC-253): Clarify explanation of Instruction/Action model in EHR IM to indicate state machine per Activity.	T Beale	
	Release 1.0.1		
5.1.0	SPEC-200 (https://specifications.openehr.org/tickets/SPEC-200): Correct Release 1.0 typographical errors. Correct `INSTRUCTION_DETAILS`.*`instruction_id`* type to `LOCATABLE_REF`. Correct inheritance of `CONTENT_ITEM` to `LOCATABLE`.	S Heard M Forss C Ma	08 Apr 2007
	SPEC-201 (https://specifications.openehr.org/tickets/SPEC-201): Add archetype ids to Instruction `ACTIVITY` class.	S Heard	
	SPEC-203 (https://specifications.openehr.org/tickets/SPEC-203): Release 1.0 explanatory text improvements. Minor changes to Entry section. Improved section on "time in the EHR".	T Beale	
	SPEC-210 (https://specifications.openehr.org/tickets/SPEC-210): Remove `LOCATABLE` inheritance from `ISM_TRANSITION` and `INSTRUCTION_DETAILS` classes	S Heard	
	SPEC-130 (https://specifications.openehr.org/tickets/SPEC-130): Correct security details in `LOCATABLE` and `ARCHETYPED` classes. Add `EHR_ACCESS` class.	T Beale	
	SPEC-218 (https://specifications.openehr.org/tickets/SPEC-218): Add *`language`* attribute to `COMPOSITION`.	G Grieve	

Issue	Details	Raiser	Completed
	SPEC-219 (https://specifications.openehr.org/tickets/SPEC-219): Use constants instead of literals to refer to terminology in RM.	R Chen	
	SPEC-244 (https://specifications.openehr.org/tickets/SPEC-244): Separate `LOCATABLE` path functions into `PATHABLE` class.	T Beale H Frankel	
	SPEC-246 (https://specifications.openehr.org/tickets/SPEC-246): Correct openEHR terminology rubrics.	B Verhees M Forss	
Release 1.0			
5.0	SPEC-14 (https://specifications.openehr.org/tickets/SPEC-14): Adjust `HISTORY`.	S Heard	25 Jan 2006
	SPEC-140 (https://specifications.openehr.org/tickets/SPEC-140). Redevelop Instruction, based on workflow principles.	S Heard T Beale	
	SPEC-147 (https://specifications.openehr.org/tickets/SPEC-147). Make `DIRECTORY` Re-usable.	R Chen	
	SPEC-162 (https://specifications.openehr.org/tickets/SPEC-162). Allow party identifiers when no demographic data. Changes to `EHR`, `EVENT_CONTEXT`, and `ENTRY`.	S Heard T Beale	
	SPEC-164 (https://specifications.openehr.org/tickets/SPEC-164). Improve description of use of times in `OBSERVATION`.	S Heard H Frankel	

Issue	Details	Raiser	Completed
	SPEC-174 (https://specifications.openehr.org/tickets/SPEC-174). Add `ADMIN_ENTRY` subtype.	S Heard T Beale	
	SPEC-175 (https://specifications.openehr.org/tickets/SPEC-175). Make `ENTRY.provider` optional.	S Heard	
	SPEC-177 (https://specifications.openehr.org/tickets/SPEC-177). Make `COMPOSITION.content` a `CONTENT_ITEM`.	S Heard, D Kalra	
	SPEC-180 (https://specifications.openehr.org/tickets/SPEC-180). Move `EVENT_CONTEXT.composer` to `COMPOSITION`	T Beale S Heard	
	SPEC-181 (https://specifications.openehr.org/tickets/SPEC-181): Change `ENTRY.provider` to `PARTY_PROXY`.	T Beale	
	SPEC-182 (https://specifications.openehr.org/tickets/SPEC-182): Rationalise `VERSION.lifecycle_state` and `ATTESTATION.status`.	C Ma D Kalra	
	SPEC-187 (https://specifications.openehr.org/tickets/SPEC-187): Correct modelling errors in `DIRECTORY` class and rename.	T Beale	
	SPEC-188 (https://specifications.openehr.org/tickets/SPEC-188): Add `generating_type` function to `ANY` for use in invariants.	T Beale	
	SPEC-189 (https://specifications.openehr.org/tickets/SPEC-189). Add `LOCATABLE.parent`. New invariants in EHR and `COMPOSITION`.	S Heard	

Issue	Details	Raiser	Completed
	SPEC-190 (https://specifications.openehr.org/tickets/SPEC-190). Rename `VERSION_REPOSITORY` to `VERSIONED_OBJECT`.	T Beale	
	SPEC-191 (https://specifications.openehr.org/tickets/SPEC-191): Add `EHR_STATUS` class to `ehr` package.	H Frankel	
	SPEC-194 (https://specifications.openehr.org/tickets/SPEC-194): Correct anomalies with `LOCATABLE.`*`uid`*	H Frankel T Beale	
	SPEC-195 (https://specifications.openehr.org/tickets/SPEC-195): Rename `EHR.`*`all_compositions`* to *`compositions`*.	S Heard	
	SPEC-161 (https://specifications.openehr.org/tickets/SPEC-161). Support distributed versioning. Correct identifier types in `EHR`, `ACTION` classes.	T Beale H Frankel	
Release 0.96			
Release 0.95			
4.5	SPEC-108 (https://specifications.openehr.org/tickets/SPEC-108). Minor changes to change_control package.	T Beale	10 Dec 2004
	SPEC-24 (https://specifications.openehr.org/tickets/SPEC-24). Revert meaning to `STRING` and rename as *`archetype_node_id`*.	S Heard, T Beale	
	SPEC-98 (https://specifications.openehr.org/tickets/SPEC-98). `EVENT_CONTEXT.`*`time`* should allow optional end time.	S Heard, DSTC	

Issue	Details	Raiser	Completed
	SPEC-109 (https://specifications.openehr.org/tickets/SPEC-109). Add `act_status` to `ENTRY`, as in CEN prEN13606.	A Goodchild	
	SPEC-116 (https://specifications.openehr.org/tickets/SPEC-116). Add `PARTICIPATION.function` vocabulary and invariant.	T Beale	
	SPEC-118 (https://specifications.openehr.org/tickets/SPEC-118). Make package names lower case.	T Beale	
	SPEC-64 (https://specifications.openehr.org/tickets/SPEC-64). Re-evaluate `COMPOSITION.is_persistent` attribute. Converted is_persistent to a function; added category attribute.	D Kalra	
	SPEC-102 (https://specifications.openehr.org/tickets/SPEC-102). Make `DV_TEXT` `language` and `charset` optional.	DSTC	
Release 0.9			
4.4.1	SPEC-96 (https://specifications.openehr.org/tickets/SPEC-96). Allow 0..* `SECTION`s as `COMPOSITION` content.	DSTC	11 Mar 2004
4.4	SPEC-19 (https://specifications.openehr.org/tickets/SPEC-19). Add `HISTORY` & `STRUCTURE` supertype.	T Beale	06 Mar 2004
	SPEC-28 (https://specifications.openehr.org/tickets/SPEC-28). Change name of `STRUCTURE` class to avoid clashes.	H Frankel	
	SPEC-87 (https://specifications.openehr.org/tickets/SPEC-87). `EVENT_CONTEXT.location` should be optional.	DSTC	

Issue	Details	Raiser	Completed
	SPEC-88 (https://specifications.openehr.org/tickets/SPEC-88). Move `INSTRUCTION.guideline_id` to `ENTRY`.	T Beale, D Kalra	
	SPEC-92 (https://specifications.openehr.org/tickets/SPEC-92). Improve `EVENT_CONTEXT` modelling. Rename `author` to `composer`. Formally validated using ISE Eiffel 5.4.	S Heard	
4.3.10	SPEC-44 (https://specifications.openehr.org/tickets/SPEC-44). Add reverse ref from `VERSION_REPOSITORY<T>` to owner object. Add invariants to `DIRECTORY` and `VERSIONED_COMPOSITION` classes.	D Lloyd	25 Feb 2004
	SPEC-46 (https://specifications.openehr.org/tickets/SPEC-46). Rename `COORDINATED_TERM` and `DV_CODED_TEXT.definition`.	T Beale	
4.3.9	SPEC-21 (https://specifications.openehr.org/tickets/SPEC-21). Rename `CLINICAL_CONTEXT.practice_setting` to `setting`.	A Goodchild	10 Feb 2004
4.3.8	SPEC-57 (https://specifications.openehr.org/tickets/SPEC-57). Environmental information needs to be included in the EHR.	T Beale	02 Nov 2003

Issue	Details	Raiser	Completed
4.3.7	SPEC-48 (https://specifications.openehr.org/tickets/SPEC-48). Pre-release review of documents. SPEC-49 (https://specifications.openehr.org/tickets/SPEC-49). Correct reference types in `EHR`, `DIRECTORY` classes. `EHR`.*`contributions`*, *`all_compositions`*, `FOLDER`.*`compositions`* attributes and invariants corrected. SPEC-50 (https://specifications.openehr.org/tickets/SPEC-50). Update Path syntax reference model to ADL specification.	T Beale, D Lloyd	25 Oct 2003
4.3.6	SPEC-41 (https://specifications.openehr.org/tickets/SPEC-41). Visually differentiate primitive types in openEHR documents.	D Lloyd	04 Oct 2003
4.3.5	SPEC-13 (https://specifications.openehr.org/tickets/SPEC-13). Rename key classes, according to CEN ENV 13606.	S Heard, D Kalra, T Beale	15 Sep 2003
4.3.4	SPEC-11 (https://specifications.openehr.org/tickets/SPEC-11). Add author attribute to `EVENT_CONTEXT`. SPEC-27 (https://specifications.openehr.org/tickets/SPEC-27). Move feeder_audit to `LOCATABLE` to be compatible with CEN 13606 revision.	S Heard, D Kalra	20 Jun 2003

Issue	Details	Raiser	Completed
4.3.3	SPEC-20 (https://specifications.openehr.org/tickets/SPEC-20). Move `VERSION.territory` to `TRANSACTION`. SPEC-18 (https://specifications.openehr.org/tickets/SPEC-18). Add `DIRECTORY` class to `rm.ehr` Package. SPEC-5 (https://specifications.openehr.org/tickets/SPEC-5). Rename `CLINICAL_CONTEXT` to `EVENT_CONTEXT`.	A Goodchild	10 Jun 2003
4.3.2	SPEC-6 (https://specifications.openehr.org/tickets/SPEC-6). Make `ENTRY.provider` a `PARTICIPATION`. SPEC-7 (https://specifications.openehr.org/tickets/SPEC-7). Replace `ENTRY.subject` and `subject_relationship` with `RELATED_PARTY`. SPEC-8 (https://specifications.openehr.org/tickets/SPEC-8). Remove `confidence` and `is_exceptional` attributes from `ENTRY`. SPEC-9 (https://specifications.openehr.org/tickets/SPEC-9). Merge `ENTRY` `protocol` and `reasoning` attributes.	S Heard, T Beale, D Kalra, D Lloyd	11 Apr 2003
4.3.1	DSTC review - typos corrected.	A Goodchild	08 Apr 2003
4.3	SPEC-3 (https://specifications.openehr.org/tickets/SPEC-3), SPEC-4 (https://specifications.openehr.org/tickets/SPEC-4). Removed `ORGANISER_TREE`. `CLINICAL_CONTEXT` and `FEEDER_AUDIT` inherit from `LOCATABLE`. Changes to path syntax. Improved definitions of `ENTRY` subtypes. Improved instance diagrams. DSTC detailed review. (Formally validated).	T Beale, Z Tun, A Goodchild	18 Mar 2003
4.2	Formally validated using ISE Eiffel 5.2. Moved `VERSIONED_TRANSACTION` class to `ehr` Package, to correspond better with serialised formalisms like XML.	T Beale, A Goodchild	25 Feb 2003

Issue	Details	Raiser	Completed
4.1	Changes post CEN WG meeting Rome Feb 2003. Moved `TRANSACTION.version_id` postcondition to an invariant. Moved feeder_audit back to `TRANSACTION`. Added `ENTRY.act_id`. `VERSION_AUDIT.attestations` moved to new `ATTESTATIONS` class attached to `VERSIONED<T>`.	T Beale, S Heard, D Kalra, D Lloyd	8 Feb 2003
4.0.2	Various corrections and DSTC change requests. Reverted `OBSERVATION.items`: `LIST<HISTORY<T>>` to `data`: `HISTORY<T>` and `EVALUATION.items`: `LIST<STRUCTURE<T>>` to `data`: `STRUCTURE<T>`. Changed `CLINICAL_CONTEXT.other_context` to a `STRUCTURE`. Added `ENTRY.other_participations`; Added `CLINICAL_CONTEXT.participations`; removed `hcp_legally_responsible` (to be archetyped). Replaced `EVENT_TRANSACTION` and `PERSISTENT_TRANSACTION` with `TRANSACTION` and a boolean attribute `is_persistent`.	T Beale	3 Feb 2003
4.0.1	Detailed corrections to diagrams and class text from DSTC.	Z Tun	8 Jan 2003
4.0	Moved `HISTORY` classes to Data Structures RM. No semantic changes.	T Beale	18 Dec 2002
3.8.2	Corrections on 3.8.1. No semantic changes.	D Lloyd	11 Nov 2002
3.8.1	Removed `SUB_FOLDER` class. Now folder structure can be nested separately archetyped folder structures, same as for `ORGANISERs`. Removed `AUTHORED_TA` and `ACQUISITION_TA` classes; simplified versioning.	T Beale, D Kalra, D Lloyd A Goodchild	28 Oct 2002

Issue	Details	Raiser	Completed
3.8	Added practice_setting attribute to `CLINICAL_CONTEXT`, inspired from HL7v3/ANSI CDA standard Release 2.0. Changed `DV_PLAIN_TEXT` to `DV_TEXT`. Removed *`hca_coauthorising`*; renamed *`hca_recording`*; adjusted all instances of *`*ID`*; *converted* `CLINICAL_CONTEXT.`*`_start_time`*, *`end_time`* to an interval.	T Beale, S Heard, D Kalra, M Darlison	22 Oct 2002
3.7	Removed Spatial package to Common RM document. Renamed `ACTION` back to `ACTION_SPECIFICATION`. Removed the class `NAVIGABLE_STRUCTURE`. Renamed `SPATIAL` to `STRUCTURE`. Removed classes `STATE_HISTORY`, `STATE`, `SINGLE_STATE`. Removed Communication (`EHR_EXTRACT`) section to own document.	T Beale	22 Sep 2002
3.6	Removed Common and Demographic packages to their own documents.	T Beale	28 Aug 2002
3.5.1	Altered syntax of `EXTERNAL_ID` identifiers.	T Beale, Z Tun	20 Aug 2002
3.5	Rewrote Demographic and Ehr_extract packages.	T Beale	18 Aug 2002
3.3.1	Simplified `EHR_EXTRACT` model, numerous small changes from DSTC review.	T Beale, Z Tun	15 Aug 2002
3.3	Rewrite of contributions, version control semantics.	T Beale, D Lloyd, D Kalra, S Heard	01 Aug 2002
3.2	DSTC comments. Various minor errors/omissions. Changed inheritance of `SINGLE_EVENT` and `SINGLE_STATE`. Included `STRUCTURE` subtype methods from GEHR. ehr_id added to VT. Altered `EHR` / `FOLDER` attrs. Added `EXTERNAL_ID.`*`version`*.	T Beale, Z Tun	25 Jun 2002

Issue	Details	Raiser	Completed
3.1.1	Minor corrections.	T Beale	20 May 2002
3.1	Reworking of Structure section, `ACTION` class, `INSTRUCTION` class.	T Beale, S Heard	16 May 2002
3.0	Plans, actions updated.	T Beale, S Heard	10 May 2002
2.9	Additions from HL7v3 coded term model, alterations to quantity model, added explanation sections.	T Beale	5 May 2002
2.8.2a	Interim version with various review modifications	T Beale	28 Apr 2002
2.8.2	Error corrections to `EHR_EXTRACT` package. P Schloeffel comments on 2.7.	T Beale, P Schloeffel	25 Apr 2002
2.8.1	Further minor changes from UCL on v2.7.	T Beale	24 Apr 2002
2.8	Dipak Kalra (UCL) comments on v2.6 incorporated. Added External Package. Minor changes elsewhere.	T Beale, D Kalra	23 Apr 2002
2.7	Final development of initial draft, including `EHR_EXTRACT`, related models	T Beale	20 Apr 2002
2.6	Further development of path syntax, incorporation of Dipak Kalra's comments	T Beale, D Kalra	15 Apr 2002
2.5	Further development of clinical and record management clusters.	T Beale	10 Apr 2002
2.4	Included David Lloyd's rev 2.3 comments.	T Beale, D Lloyd	4 Apr 2002
2.3	Improved context analysis.	T Beale	4 Mar 2002
2.2	Added path syntax.	T Beale	19 Nov 2001
2.1	Minor organisational changes, some content additions.	T Beale	18 Nov 2001

Issue	Details	Raiser	Completed
2.0	Rewrite of large sections post-Eurorec 2001 conference, Aix-en-Provence. Added folder, contribution concepts.	T Beale	15 Nov 2001
1.2	Major additions to introduction, design philosophy	T Beale	1 Nov 2001
1.1	Major changes to diagrams; STILL UNREVIEWED	T Beale	13 Oct 2001
1.0	Based on GEHR Object Model	T Beale	22 Sep 2001

Acknowledgements

Editor

- Thomas Beale, Ars Semantica (UK); openEHR Foundation Management Board.

Contributors

This specification benefited from wide formal and informal input from the openEHR and wider health informatics community. The openEHR Foundation would like to recognise the following people for their contributions.

- Rong Chen MD, PhD, Cambio Healthcare Systems, Sweden
- Linda Bird, PhD, DSTC, Australia
- Matthew Darlinson, PhD, CHIME, UCL, UK
- Heath Frankel, CTO Ocean Informatics, Australia
- Matias Forss, Aalborg University, Denmark
- Andrew Goodchild, PhD, senior research scientist DSTC, Australia
- Grahame Grieve, Kestral Systems, Australia
- Sam Heard MD, Ocean Informatics, Australia
- David Ingram PhD, Emeritus Professor of Health Informatics, UCL, UK
- Dipak Kalra MD, PhD, Professor Health Informatics, CHIME, UCL, UK
- Bostjan Lah, Architect, Marand, Slovenia
- David Lloyd (ret), CHIME, UCL, UK
- Chunlan Ma PhD, MD, Ocean Informatics, Australia
- Jan Mewes, KS-plus.org, India/Germany
- Pablo Pazos Gutierrez, CaboLabs, Uruguay
- Matija Polajnar PhD, Senior developer, Marand, Slovenia
- Peter Schloeffel MD, Ocean Informatics, Australia
- Zar zar Tun, DSTC, Australia
- Bert Verhees, ROSA Software, The Netherlands.

Support

The work reported in this paper has been funded in part by the following organisations:

- University College London - Centre for Health Informatics and Multi-professional Education (CHIME);
- Ocean Informatics;
- Distributed Systems Technology Centre (DSTC), under the Cooperative Research Centres Program through the Department of the Prime Minister and Cabinet of the Commonwealth Government of Australia.

Special thanks to David Ingram, Emeritus Professor of Health Informatics at UCL, who provided a vision and collegial working environment ever since the days of GEHR (1992).

Trademarks

- 'openEHR' is a trademark of the openEHR Foundation
- 'Java' is a registered trademark of Oracle Corporation
- 'Microsoft' and '.Net' are trademarks of the Microsoft Corporation
- 'CORBA' is a trademark of the Object Management Group

1. Preface

1.1. Purpose

This document describes the openEHR EHR Information Model, which is a model of an interoperable EHR in the ISO RM/ODP information viewpoint. This model defines a logical EHR information architecture rather than just an architecture for communication of EHR extracts or documents between EHR systems. The openEHR definition of the EHR Extract is given in the openEHR EHR_EXTRACT Information Model.

The intended audience includes:

- Standards bodies producing health informatics standards;
- Academic groups using openEHR;
- The open source healthcare community;
- Solution vendors;
- Medical informaticians and clinicians interested in health information.
- Health data managers.

1.2. Related Documents

Prerequisite documents for reading this document include:

- The openEHR Architecture Overview (https://specifications.openehr.org/releases/BASE/latest/architecture_overview.html).

Related documents include:

- The openEHR Support Information Model (https://specifications.openehr.org/releases/RM/Release-1.1.0/support.html) ;
- The openEHR Data Types Information Model (https://specifications.openehr.org/releases/RM/Release-1.1.0/data_types.html);
- The openEHR Data Structures Information Model (https://specifications.openehr.org/releases/RM/Release-1.1.0/data_structures.html);
- The openEHR Common Information Model (https://specifications.openehr.org/releases/RM/Release-1.1.0/common.html).

1.3. Status

This specification is in the STABLE state. The development version of this document can be found at https://specifications.openehr.org/releases/RM/latest/ehr.html (https://specifications.openehr.org/releases/RM/latest/ehr.html).

Known omissions or questions are indicated in the text with a 'to be determined' paragraph, as follows:

TBD: (example To Be Determined paragraph)

1.4. Feedback

Feedback may be provided on the openEHR RM specifications forum (https://discourse.openehr.org/c/specifications/rm).

Issues may be raised on the specifications Problem Report tracker (https://specifications.openehr.org/components/RM/open_issues).

To see changes made due to previously reported issues, see the RM component Change Request tracker (https://specifications.openehr.org/components/RM/history).

1.5. Conformance

Conformance of a data or software artifact to an openEHR specification is determined by a formal test of that artifact against the relevant openEHR Implementation Technology Specification(s) (ITSs), such as an IDL interface or an XML-schema. Since ITSs are formal derivations from underlying models, ITS conformance indicates model conformance.

2. Background

This section describes the inputs to the modelling process that created the openEHR Information Model.

2.1. Requirements

There are broadly three sets of requirements which inform this model, as described in the following subsections.

2.1.1. Original GEHR Requirements

From the European GEHR project 1992-5 (Ingram, 1995), the following broad requirements areas were identified:

- the life-long EHR;
- priority: Clinician / Patient interaction;
- medico-legal faithfulness, traceability, audit-trailing;
- technology & data format independent;
- facilitate sharing of EHRs;
- suitable for both primary & acute care;
- secondary uses: education, research, population medicine;
- open standard & software deliverables;

The original deliverables can be reviewed in detail at the GEHR page at CHIME, UCL.

2.1.2. GEHR Australia Requirements

The GEHR Australia project (1997-2001; Heard (2001), Beale & Heard (2000)) introduced further requirements, including:

- support for clinical data structures: lists, tables, time-series etc;
- safer information model than the original (European) GEHR: context attributes only in valid places (but still similar style);
- separate groups for "persistent", "demographic" and "event" information in EHR, which corresponds closely to real clinical querying patterns;
- introduction of formally specified archetypes, and archetype-enabled information model;
- interoperability at the knowledge level, i.e. level of domain definitions of information such as "discharge summary" and "biochemistry result";
- XML-enabled;
- consider compatibility with ISO 13606 (https://www.iso.org/standard/40784.html), OMG Corbamed (https://www.omg.org/spec/COAS/), HL7v3 (https://www.hl7.org/implement/standards/product_brief.cfm?product_id=186).

GEHR Australia produced a proof of concept implementation in which clinical archetypes were developed and used. See Beale (2000) for a technical description of archetypes.

2.1.3. European Synapses and SynEx Project Requirements

Following the original Good European Health Record project the EU-funded Synapses (1996-8; Grimson & Groth (1996)) and SynEx (1998-2000; Sottile Ferrara Grimson Kalra & Scherrer (1999)) projects extended the original requirements basis of GEHR to include further requirements, as follows:

- the requirements of a federation approach to unifying disparate clinical databases and EPR systems: the federated health record (FHR) (Kalra, 1996);
- the need to separate a generic and domain independent high-level model for the federated health record from the (closely related) model of the meta-data which defines the domain specific health record characteristics of any given clinical specialty and any given federation of database schemata;
- a formalism to define and communicate (share) knowledge about the semantic hierarchical organisation of an FHR, the permitted data values associated with each leaf node in a record hierarchy and any constraints on values that leaf nodes may take (the Synapses Object Dictionary, Kalra (1998));
- the core technical requirements of and interfaces for a federation middleware service (Grimson & Groth, 1996).

2.1.4. European EHCR Support Action Requirements

This EU Support Action project ('SupA'; Dixon Grubb Lloyd & Kalra (1998), Dixon Grubb & Lloyd (2000), Dixon Grubb & Lloyd (2000)) consolidated the requirements published by a wide range of European projects and national health informatics organisations as a Consolidated List of Requirements (Dixon, Grubb, Lloyd, & Kalra, 2001).

2.1.5. ISO EHR Requirements

The above requirements publications and the recent experience of openEHR feed into the definition of a set of EHR requirements authored by ISO Technical Committee 215 (Health Informatics) - ISO 18308 (https://www.iso.org/standard/52823.html). This has been reviewed and a mapping to openEHR (https://specifications.openehr.org/releases/1.0.2/architecture/requirements/iso18308_conformance.pdf) developed.

2.1.6. openEHR Requirements

New requirements generated during the development of openEHR included the following:

- major improvements in information models from the point of view of reducing progammer errors and ambiguity;
- better modelling of time and context (temporal/spatial approach);
- better understanding of legacy system / federation problem;
- workflow modelling;
- harmonise with ISO 13606;
- integration with HL7v2 and other messaging systems;
- numerous specific clinical requirements.

2.2. Relationship to other Health Information Models

Where relevant, the correspondences to other information models have been documented in this specification. Correspondences to the GEHR Australia and EU Synapses/SynEx models are shown, since these are the models

on which the openEHR EHR information model is primarily based. The following sections summarise the other models and standards with which correspondences are shown.

2.2.1. ISO 13606

These models have been influenced by and have also influenced the models in ISO 13606 (2005 revision); accordingly, relationships to 13606 have been documented fairly precisely. Particular areas of openEHR which have been changed due to this process include:

- change of major class names (`TRANSACTION` → `COMPOSITION` etc; see CR-000013);
- improved model of `ATTESTATION` (see CR-000025);
- improved model of feeder audits (see CR-000027).

Implementation experience with Release 0.9 and 0.95 of openEHR has further improved these areas significantly. Nevertheless, openEHR is not a copy of CEN, for two reasons. Firstly, its scope includes systems, while ISO 13606 defines an EHR Extract; secondly, ISO 13606 suffers somewhat from 'design by committee", and has no formal validation mechanism for its models.

2.2.2. HL7 Version 3

Correspondences to some parts of HL7 version 3 (ballot 5, July 2003) are also documented where possible, however, it should be understood that there are a number of difficulties with this. Firstly, while the HL7v3 Reference Information Model (RIM) - the closest HL7 artifact to an information model - provides similar data types and some related semantics, it is not intended to be a model of the EHR. In fact, it differs from the information model presented here (and for that matter most published information models) in two basic respects: a) it is an amalgam of semantics from many systems which would exist in a distributed health information environment, rather than a model of just one (the EHR); b) it is also not a model of data, but a combination of "analysis patterns" in the sense of Fowler (1997) from which further specific models - subschemas - are developed by a custom process of "refinement by restriction", in order to arrive at message definitions. As a consequence, data in messages are not instances of HL7v3 RIM classes, as would be the case in other systems based on information models of the kind presented here.

Despite the differences, there are some areas that appear to be candidates for mapping, specifically the data types and terminology use, and the correspondence between openEHR Compositions and parts of the HL7 Clinical Document Architecture (CDA) (https://www.hl7.org/implement/standards/product_brief.cfm?product_id=7).

2.2.3. OMG HDTF

In general, the openEHR information models represent a more recent analysis of the required semantics of EHR and related information than can be found in the information viewpoint of the OMG HDTF specifications (particularly OMG PIDS (https://www.omg.org/spec/PIDS/) and OMG COAS (https://www.omg.org/spec/COAS/). However, the computational viewpoint (i.e. functional service interface definitions) is one of the inputs to the openEHR service model development activity.

3. The EHR Information Model

3.1. Overview

The figure below illustrates the package structure of the openEHR EHR information model.

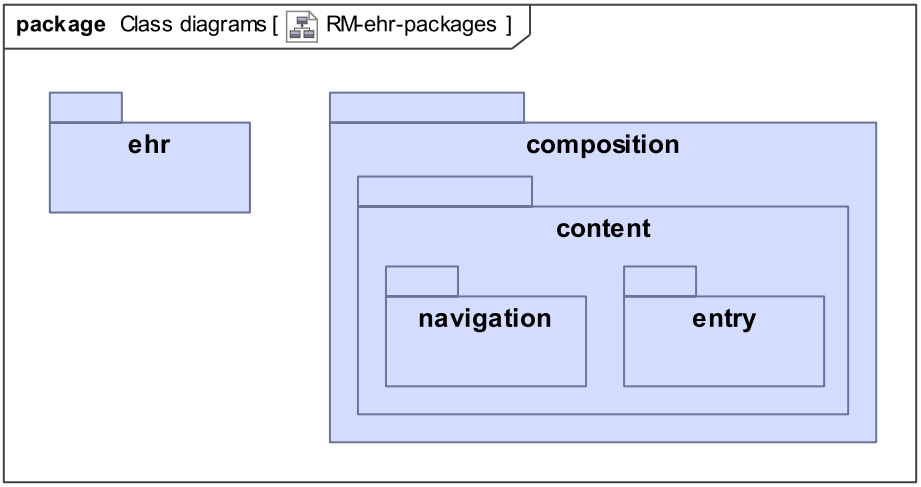


Figure 1. EHR Packages

The packages are as follows:

- `ehr` : this package contains the top level structure, the EHR, which consists of an `EHR_ACCESS` object, an `EHR_STATUS` object, versioned data containers in the form of `VERSIONED_COMPOSITIONs` , optionally indexed by one or more hierarchical `FOLDER` structures. A collection of `CONTRIBUTIONs` which document the changes to the EHR over time is also included.
- `composition` : the Composition is the EHR's top level "data container", and is described by the `COMPOSITION` class.
- `content` : this package contains the Navigation and Entry packages, whose classes describe the structure and semantics of the contents of Compositions in the health record.
  - `navigation` : the `SECTION` class provides a navigational structure to the record, similar to "headings" in the paper record. `ENTRYs` and other `SECTIONs` can appear under `SECTIONs` .
  - `entry` : This package contains the generic structures for recording clinical statements. Entry types include `ADMIN_ENTRY` , `OBSERVATION` (for all observed phenomena, including mechanically or manually measured, and responses in interview), `EVALUATION` (for assessments, diagnoses, plans), `INSTRUCTION` (actionable statements such as medication orders, recalls, monitoring, reviews), and `ACTION` (information recorded as a result of performing Instructions).

The figure below illustrates an overview of the class structure of the EHR Information Model, along with the main concepts on which they rely, namely Data Types, Data Structures, Archetyped, and Identification.

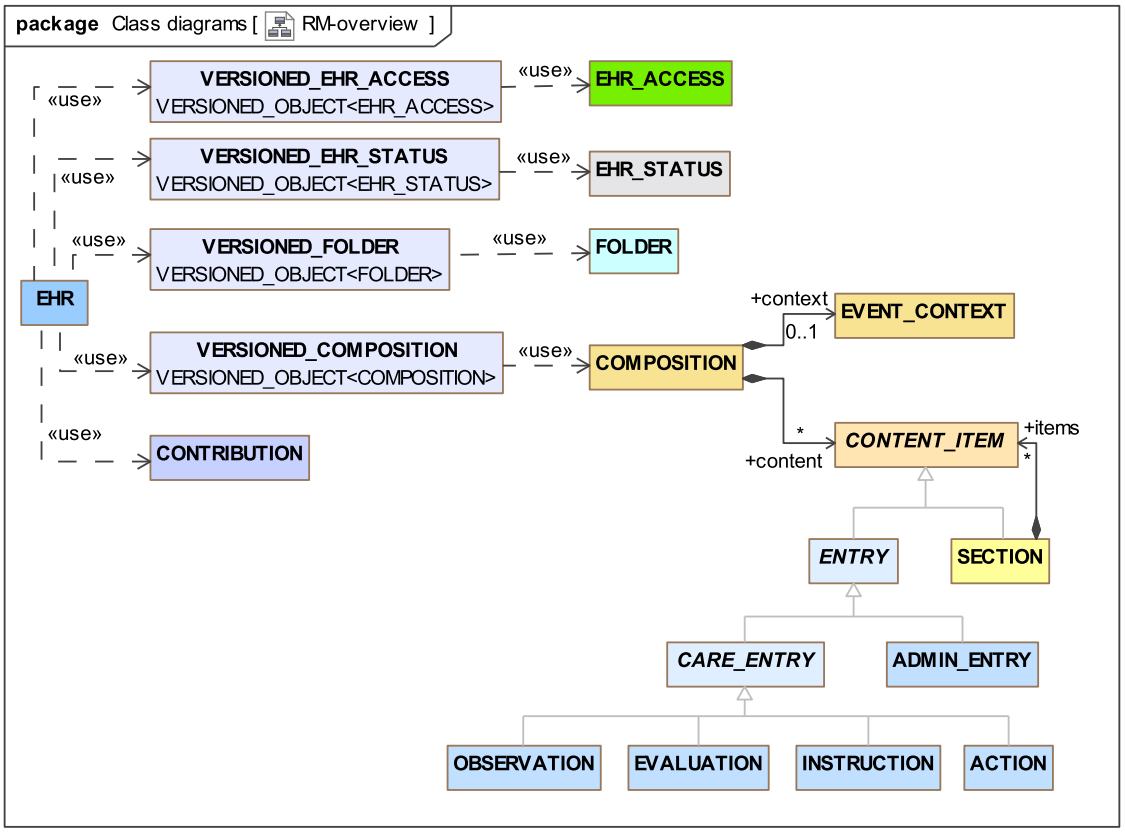


Figure 2. EHR Information Model Overview

4. EHR Package

4.1. Overview

The openEHR EHR is structured according to a relatively simple model. A central EHR object identified by an EHR id specifies references to a number of types of structured, versioned information, plus a list of Contribution objects that act as audits of change-sets made to the EHR. The high-level structure of the openEHR EHR is shown in below.

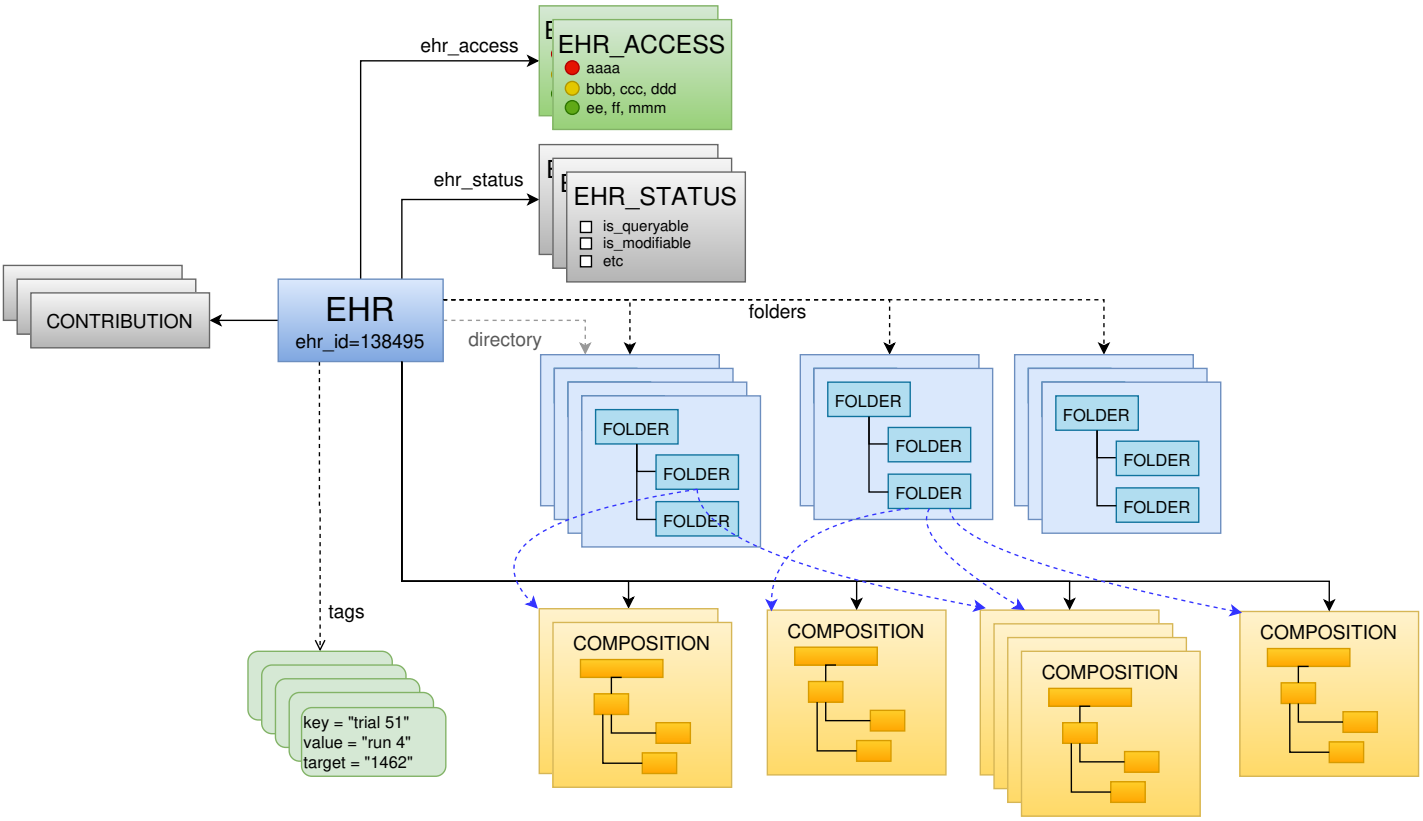


Figure 3. High-level EHR structure

In this figure, the parts of the EHR are as follows:

- *EHR*: the root object, identified by a globally unique EHR identifier;
- *EHR_access (versioned)*: an object containing access control settings for the record;
- *EHR_status (versioned)*: an object containing various status and control information, optionally including the identifier of the subject (i.e. patient) currently associated with the record;
- *Folders (versioned)*: optional hierarchical folder structures that can be used to logically index Compositions;
- *Compositions (versioned)*: the containers of all clinical and administrative content of the record;
- *Contributions*: the change-set records for every change made to the health record; each Contribution references a set of one or more Versions of any of the versioned items in the record that were committed or attested together by a user to an EHR system.

The `ehr` package is illustrated in Figure 4 below. The classes have a more or less one-to-one correspondence with the objects shown in Figure 3. Each versioned object of type `X` is defined by a class `VERSIONED_X`, which is a binding of the type `X` to the generic type paremeter `T` in the generic type `VERSIONED_COMPOSITION`

(while such bindings do not strictly require classes of their own, they facilitate implementation in languages lacking genericity).

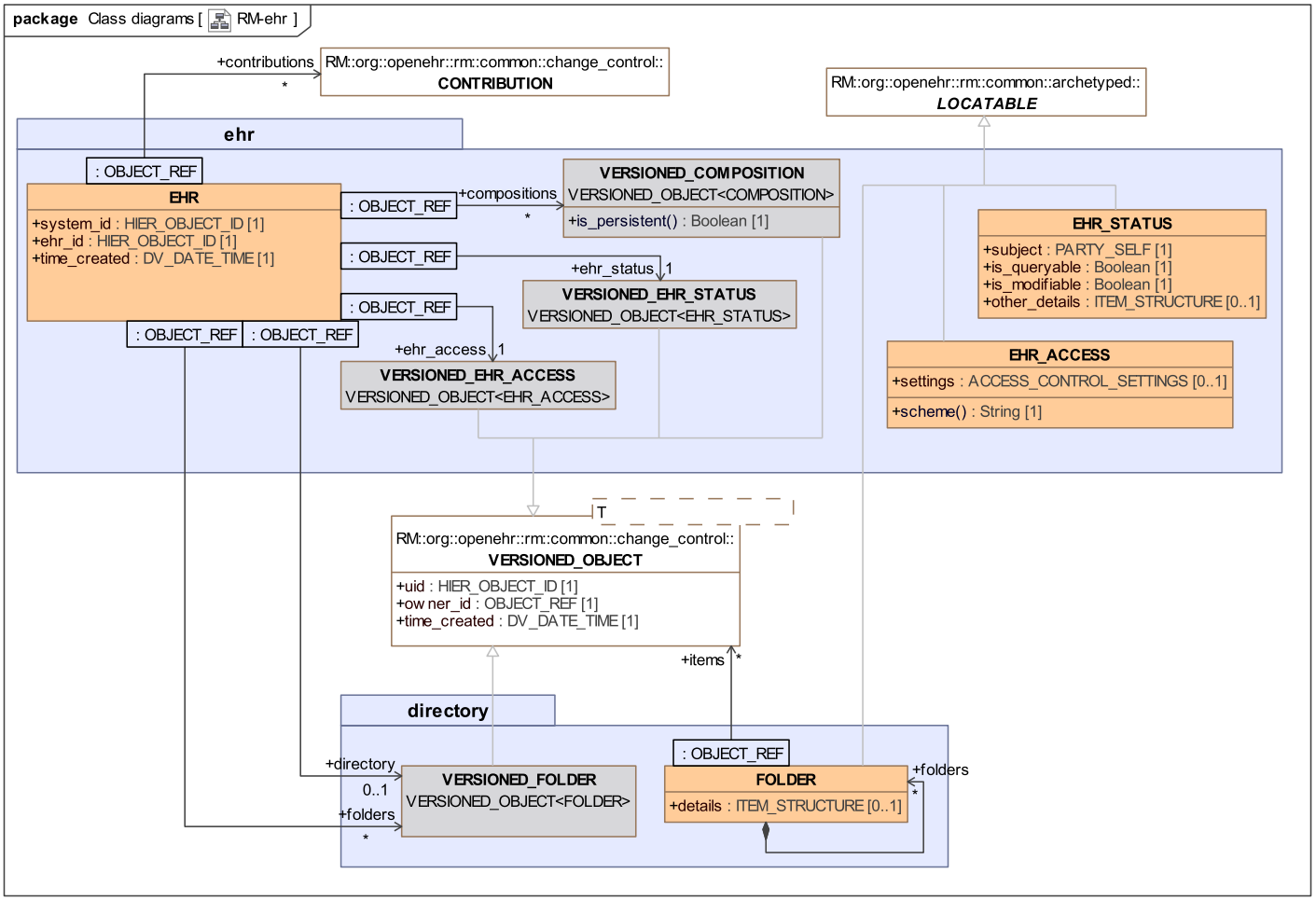


Figure 4. rm.ehr package

4.2. The Parts of the EHR

4.2.1. Root EHR Object

The root EHR object records three pieces of information that are immutable after creation: the identifier of the system in which the EHR was created, the identifier of the EHR (distinct from any identifier for the subject of care), and the time of creation of the EHR. Otherwise, it simply acts as an access point for the component parts of the EHR.

The `system_id` attribute is used to record the identifier of the logical EHR repository to which the data containing the audit are committed. What constitutes a 'system' in this context is described in more detail in the Architecture Overview (https://specifications.openehr.org/releases/BASE/latest/architecture_overview.html#_the_ehr_system).

References rather than containment by value are used for the relationship between the EHR and `VERSIONED_X` objects, reflecting the vast majority of retrieval scenarios in which only selected (usually recent) items are needed. Containment by value would lead to systems which retrieved all `VERSIONED_X` objects every time the EHR object was accessed.

4.2.2. EHR Access

Access control settings for the whole EHR are specified in the `EHR_ACCESS` object. This includes default privacy policy, lists of identified accessors (individuals and groups) and exceptions to the default policies, each one identifying a particular Composition in the EHR. All changes to the EHR Access object are versioned via the normal mechanism, ensuring that the visible view of the EHR at any previous point in time is reconstructable.

Because security models for health information are still in their infancy, the openEHR model adopts a completely flexible approach. Only two hard-wired attributes are defined in the `EHR_ACCESS` class. The first is the name of the security scheme currently in use, while the second (settings attribute) is an object containing the access settings for the EHR according to that particular scheme. Each scheme is defined by an instance of a subclass of the abstract class `ACCESS_CONTROL_SETTINGS`, defined in the Security Information Model.

4.2.3. EHR Status

The `EHR_STATUS` object contains a small number of hard-wired attributes, and an archetyped *`other_details`* part. The former are used to indicate who is (currently understood to be) the subject of the record, and whether the EHR is actively in use, inactive, and whether it should be considered queryable. As for elsewhere in the openEHR EHR, the subject is represented by a `PARTY_SELF` object, enabling it to be made completely anonymous, or alternatively to include a patient identifier. The subject is included in the EHR Status object because it can change, due to errors being discovered in the allocation of records to patients. If the anonymous form is used, the change will be made elsewhere in a cross-reference table; otherwise the EHR Status object will be updated. Because it is versioned, all such changes are audit-trailed, and the change history can be reconstructed.

Other EHR-wide meta-data may be recorded in the archetyped part of this object, including runtime environment settings, software application names and version ids, identification and versions of data resources such as terminologies and possibly even actual software tools, configuration files, keys and so on. Such information is commonly versioned in software configuration management systems, in order to enable the reconstruction of earlier versions of software with the correct tools. One reason to store such information at all is that it adds to medico-legal support when clinicians have to justify a seemingly bad decision: if it can be shown that the version of software in use at the time was faulty, they are protected, but to do this requires that such information be recorded in the first place.

4.2.4. Compositions

The main data of the EHR is found in its Compositions, i.e. instances of the `COMPOSITION` class in the RM. Composition is based on the notion of a unit of information resulting from the interaction of a healthcare agent (subject or healthcare professional) with the EHR. It is designed to satisfy the following needs, which include the well-known ACID characteristics of transactions Gray & Reuter (1993).

- *durability*: the need for a persistent unit of information committal in the record;
- *atomicity*: the need for a minimal unit of integrity for clinical information, corresponding to a minimal unit

for committal, transmission and security;
- *consistency*: the need for contributions to the record to leave the record in a consistent state;
- *isolation*: the need for contributions to the record by simultaneous users not to interfere with each other;
- *indelibility*: the requirement that information committed to the record be indelible in order to support later investigations, for both medico-legal and process improvement purposes, and the consequent requirement to be able to access previous states of the record;
- *modification*: the need for users to be able to modify EHR contents, in order to correct errors or update previously recorded information (e.g. current medications, family history); and
- *traceability*: the need to record adequate auditing information at committal, in order to provide clinical and legal traceability.

The Composition concept originated from the 'Transaction' concept of the GEHR project Lloyd (1992), Lloyd (1993), Heard Kalra & Ingram (1994), Beale Lloyd Heard Kalra & Ingram (1995). In openEHR, it was renamed to 'Composition' (the name of the equivalent concept in ISO 13606-1 (https://www.iso.org/standard/40784.html)), and it has been expanded and more formally defined in openEHR in two ways. Firstly, the idea of a unit of committal has been formalised by the openEHR model of change control (see the openEHR Common Information Model (https://specifications.openehr.org/releases/RM/Release-1.1.0/common.html)); how this applies to the EHR and Compositions is described below.

Secondly, the informational purpose of a Composition is not just to capture data from a passing clinical event such as a patient contact, but also to track longitudinal significant patient state, such as problems, allergies and medications. Accordingly, openEHR Compositions are temporally classified using a small number of broad categories recorded in the `COMPOSITION.category` attribute. Categories include *event*, *episodic* and *persistent*. From a scientific realist ontological point of view, the kind of information recorded in a persistent Composition approximates a description of some *continuant* aspect of the patient (see Arp Smith & Spear (2015), Sowa (2000)). This is in contrast with event Compositions, which record *occurrents*, i.e. events or states that occurred or were true at some moment in time. The episodic category corresponds to information that is relevant for the period of time of a care episode.

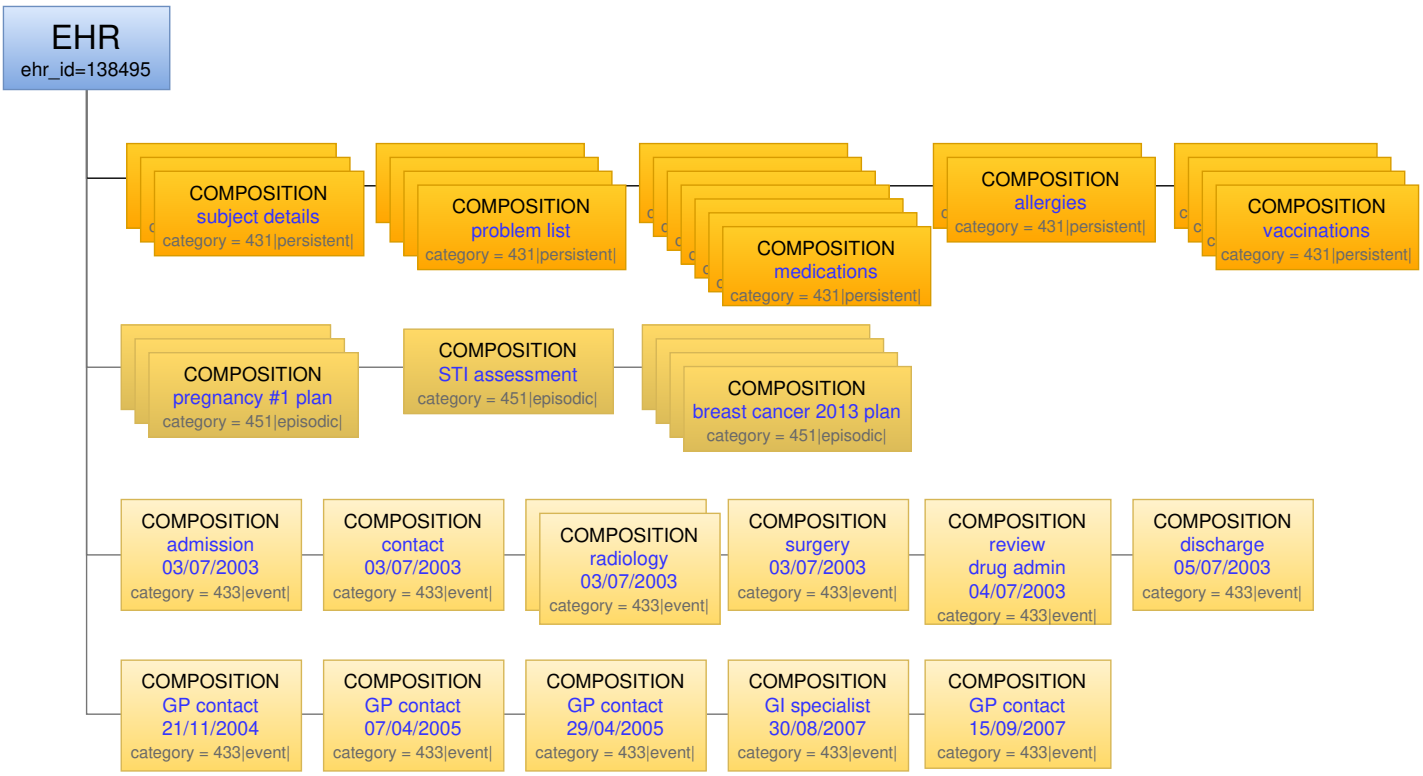


Figure 5. An EHR containing Persistent, Episodic and Event Compositions

4.2.4.1. Persistent Compositions

In an EHR, there is a need to record items that summarise the key aspects of patient state, which are of ongoing relevance in the record. These are often separated by clinicians into well-known categories, such as:

- Problem list - list of current problems, issues or concerns;
- Current medications - list of patient medications that are *active* or *suspended*;
- Therapeutic precautions - list of allergies and interactions relevant to care;
- Vaccination history - list of vaccinations; acts as a proxy for patient immunity state;
- Patient preferences - list of proscribed and/or preferred types of treatment;
- Lifestyle - e.g. substance use, profession-related information;
- Family history - major problems in biological relatives; acts as a proxy for patient risk;
- Social situation - picture of patient living situation relevant to care provision;
- Care plan - Plan(s) produced by clinicians describing planned management of chronic problems.

Persistent Compositions can be thought of as proxies for the state or situation of the patient, which are maintained as a single source-of-truth over an entire patient lifetime. The intent is that any changes or updates are applied to the same logical composition instance as new versions. Together they provide a picture of the patient over time. The total number of Persistent Compositions is normally small for any given patient even with the possibility of more than one instance of some, e.g. multiple condition-specific problem lists. However the number of versions due to updating over time may be significant.

4.2.4.2. Event Compositions

Event Compositions record what happens during healthcare system activities performed with and/or for the patient, such as:

- observations, assessments, orders and actions performed during a patient contact;
- actions performed during an activity in which the patient is not a participant e.g. surgery;
- actions performed during an activity in which the patient is not present e.g. pathology testing.

An important job of the event Composition is to record not only the data from the healthcare event, such as observations on the patient, but also to record the event context information, i.e. the who, when, where and why of the event.

Over time, the number of event Compositions is likely to far outstrip the number of persistent Compositions in a typical EHR, while the average number of versions per Composition (normally only due to corrections) will be far lower.

4.2.4.3. Episodic Compositions

Compositions may be classified as *episodic* if they contain information relevant to an ongoing care situation spanning sigificant time, such as pregnancy and birth, major surgery and recovery etc. During a specific episode of care, Event Compositions will still be used to record normal clinical events, particularly routine observations. However, assessments, summaries and care plans specific to the condition being treated may be committed in Compositions with `category` set to `episodic` to indicate that they should be treated as having a persistent-like quality i.e updated as as single instance source-of-truth, but only maintained as such for a defined period or set of conditions and not for the entire life of the patient. An example might be a Problem list which is maintained as a continuously updated single instance during a hospital admission but only for the duration of that admission. If the patient is re-admitted, a new instance of an Episodic Composition is then created and continually updated for that admission.

4.2.5. Folders

As Compositions accumulate over time in the EHR, they form a long-term patient history. The `folders` structures can be used in the EHR to index Compositions using one or more versioned hierarchies of `FOLDER` objects. In the openEHR model, **Folder structures do not contain Compositions, only references to them**. *More than one Folder can therefore refer to the same Composition* or other target object. If `folders` is not Void, the `directory` attribute always contains a reference to the first member, for backward compatibility with pre-Release 1.1.0 systems.

Structurally, each reference in `folders` points to a logical tree of `FOLDER` objects each potentially containing references to `COMPOSITIONs`, or possibly just more `FOLDERS`. **Each such tree is versioned**, i.e. each version of one folder tree contains a modified version of the previous state of the tree. Folder trees consist of `FOLDER` objects, which inherit from the `LOCATABALE`, enabling archetypes to be used to define specific folder tree structures. Within a Folder archetype, the `details` attribute may be configured to contain meta-data specific to the purpose of the Folder (tree).

The `directory` reference may be considered *by convention* as referring to a folder hierarchy that is *shared by all users of the EHR* to which it is attached, e.g. all medical specialities, departments or settings. It might thus be

used to represent episodes. In this case, the other Folder hierarchies referred to by `folders` might be created, maintained and used by different departments, specialities etc. However, completely different usage of the folder structures is possible. Use of Folder archetypes should be considered for communicating the intended usage in a particular system.

In any Folder hierarchy, regardless of specific use, Folders may be *added or removed contemporaneously or subsequently* to the committal of the referenced Compositions, including after substantial time has passed. Contemporaneous additions would normally be included in the same Contribution as the referenced Compositions. Similarly, at any time, an entirely new Folder hierarchy may be added, which will be referenced by a new member of the `EHR.folders` attribute.

Semantically, Folders may be used to manage a simple classification of Compositions, e.g into Event and Persistent, or they might be used to create categories based on episodes or other groupings of Compositions.

A Folder structure containing references to Compositions is shown below, in which the following Folders are used:

- **Persistent compositions**: compositions containing information which is valid in the long term;
- **Diabetes**: compositions relating to diabetic care, arranged in sub-folders;
  - **GP Encounters**: GP encounters relating to diabetes;
  - **Episode 03-07-2003 - 05-07-2003**: diabetes care episode;
  - **Episode xxxx**: other diabetes care episodes;
- **(other problem)**: Folders indexing Compositions relating to other problems

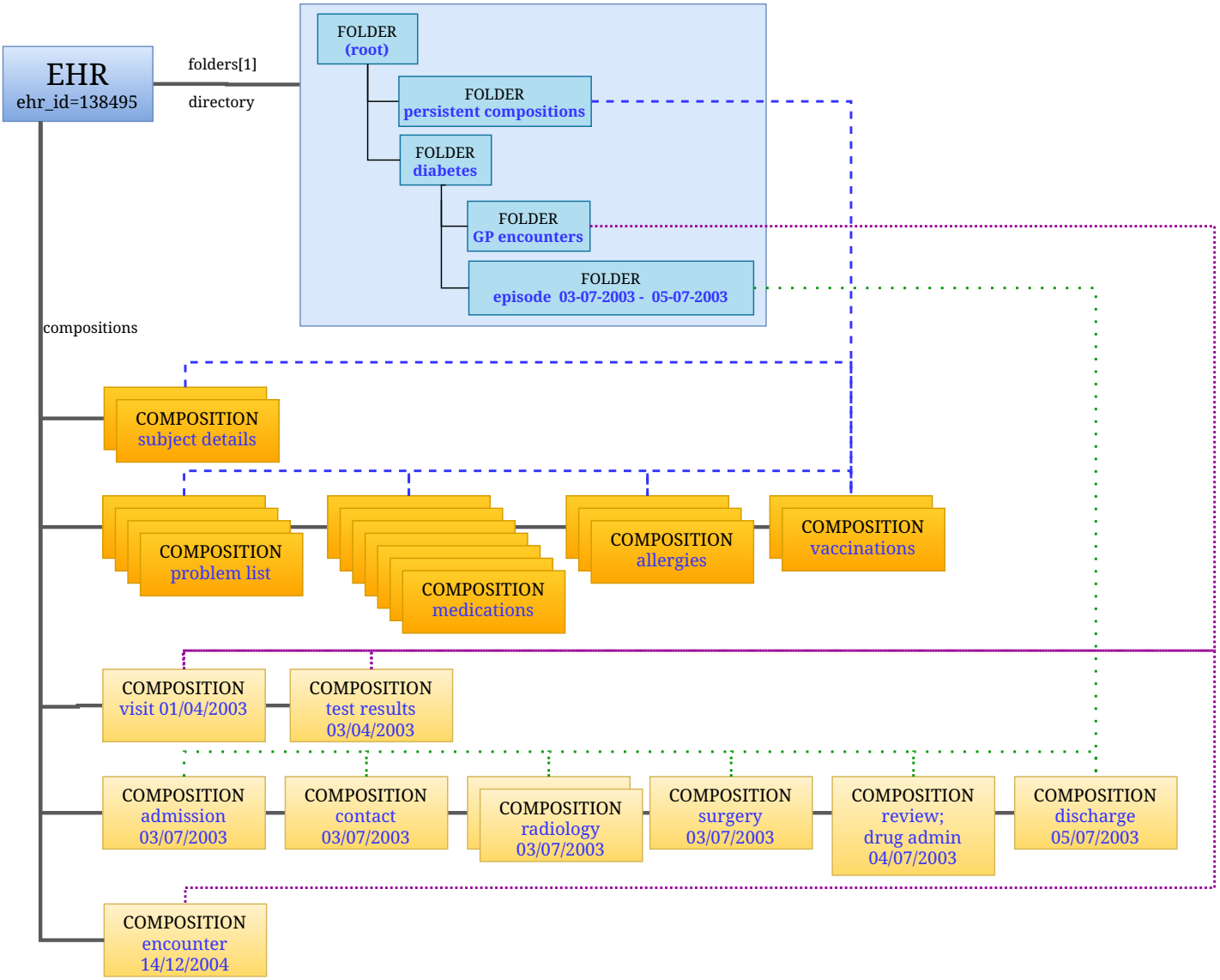


Figure 6. Using Folders to index Compositions in the EHR

This structure separates out Compositions on the basis of likely access by applications. Persistent Compositions remain relevant, usually for the patient lifetime, consequently need to be quickly accessible by numerous applications views. Event Compositions contain information whose relevance may fade quickly (e.g. vital signs measurements) or which are effectively indexed by various persistent compositions (e.g. diagnoses). They are indexed in this example by a Folder structure that makes it easy to find Compositions on the basis of problem type (diabetes etc), and then by type of visit.

Neither the Folder names nor the Composition names illustrated above are part of the openEHR EHR reference model: all such details are provided by archetypes; hence, EHR structures based on completely different conceptions of division of information, or even different types of medicine are equally possible.

The Folder hierarchy in each member of the `folders` attribute is maintained in its own versioned object, ensuring that changes to the indexing structure are versioned over time in the same way as changes to the EHR content itself.

4.3. Change Control in the EHR

Given an EHR in which there is an EHR Access object, EHR Status object, Event and Persistent Compositions, and possibly hierarchical Folders structures, the general model of update of the EHR is that any of these might be created and/or modified during an update. The simplest, most common case is the creation of a single contact Composition. Another common case will be the creation of an Event Composition, and modification of one or more Persistent Compositions, e.g. due to facts learned in the consultation about family history, or due to prescription of new medications. Other types of updates include corrections to existing Compositions, and acquisition of Compositions from another site such as a hospital. Any of these updates might also include a change to the folder structure, or the moving of existing Compositions to other Folders. Naturally these scenarios depend on a structure of the record including event and persistent compositions, and a folder structure. In the extreme, an EHR consisting only of event Compositions and no Folders will experience only the creation of a single Composition for most updates, with acquisitions being the exception. Less often, updates will be made to the EHR Access and EHR Status objects, according to the management and access control needs of the patient and health care providers.

In general, the following requirements must always be met, regardless of the particular changes made at any one time:

- the record should always be in a consistent informational state;
- all changes to the record be audit-trailed;
- all previous states of the record be available for the purposes of medico-legal investigation.

These requirements are satisfied in openEHR via the use of the change control and versioning facilities defined in the Common Information Model. A key facet of the approach is the use of change-sets, known as Contributions in openEHR. Applied to the EHR, they can be visualised as shown below:

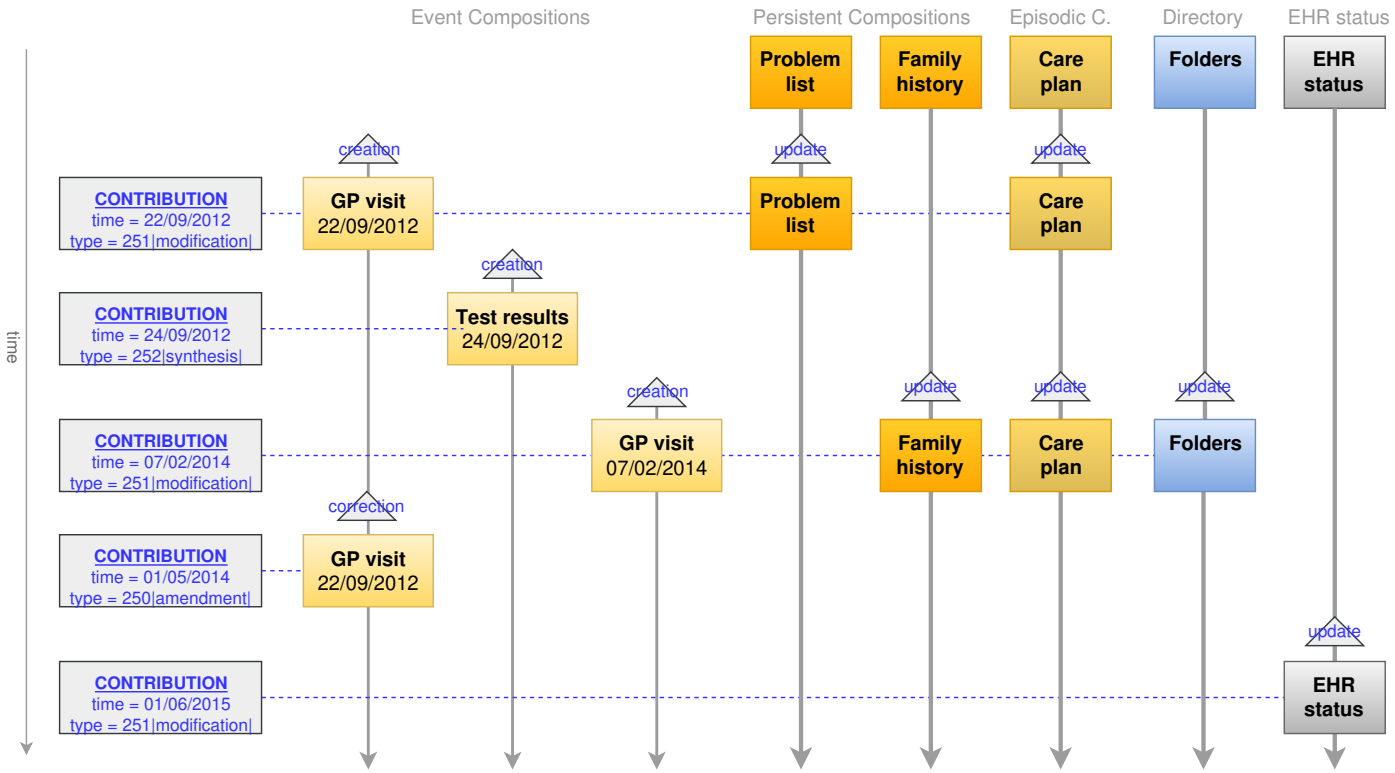


Figure 7. Contributions to the EHR

- The first is due to a patient contact, and causes the creation of a new contact composition; it also causes changes to the problem list, current medications and care plan compositions (once again, in a differently designed record, all this information might have been contained in a single event Composition; likewise, it might be been distributed into even more Compositions).
- The next Contribution is the acquisition of test results from a pathology laboratory.
- The third is another contact in which both family history and the folder structure are modified.
- This fourth is an error correction (e.g. a mispelled name, wrongly entered value), and shows that there can be a Contribution even when there is no healthcare event.
- The last is an update to the EHR status information in the EHR, due to a software upgrade.

The list of Contributions made to a record is recorded along with changes to the data, so that not only are changes to top-level objects (EHR Acces, Composition etc) captured, but the list of changes forming a change set due to a user commit is always known as well.

4.3.1. Versioning of Compositions

Versioning of Compositions is achieved with the `VERSIONED_OBJECT<T>` type from the Common IM `change_control package` (https://specifications.openehr.org/releases/RM/Release-1.1.0/common.html#_change_control_package), which in the `composition` package is explicitly bound to the `COMPOSITION` class, via the class `VERSIONED_COMPOSITION` which inherits from the type `VERSIONED<COMPOSITION>`.

The effect of version control on Compositions is visualised in the figure below. The versions (each version being an `ORIGINAL_VERSION<COMPOSITION>`) shown here in a `VERSIONED_COMPOSITION` are the same versions shown along each vertical line in Figure 7, this time shown with their associated audit items. The set of versions are understood as a set of successive modifications of the same data in time.

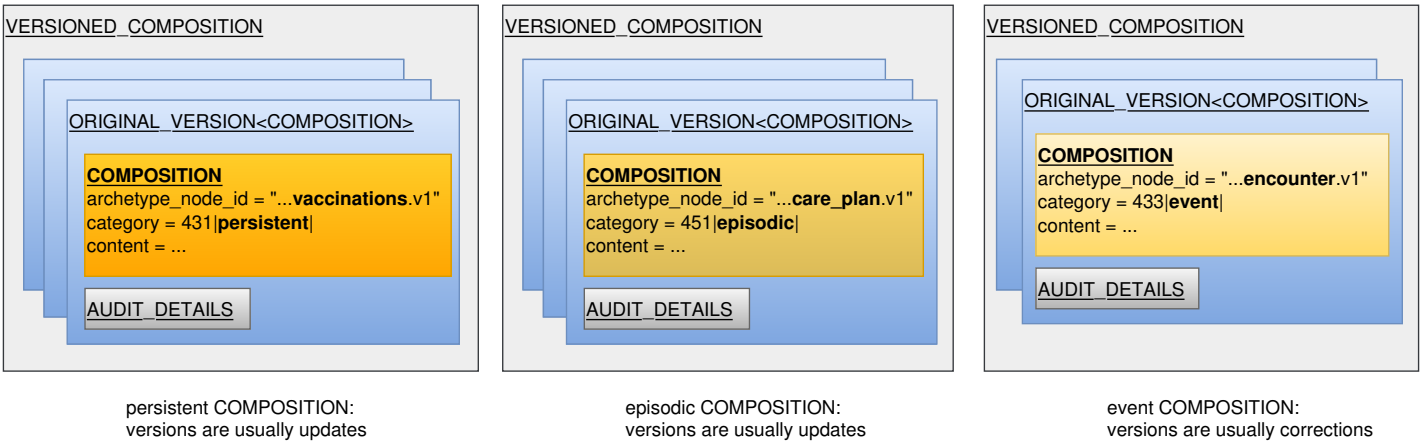


Figure 8. Versioned Compositions

The `VERSIONED_COMPOSITION` can be thought of as a smart repository: how it stores successive versions in time is an implementation concern (there are a number of intelligent algorithms available for this), but what is important is that its functional interface enables any version to be retrieved, whether it be the latest, the first, or any in between.

4.3.2. Versioning Scenarios

The following scenarios for creating new `COMPOSITION` versions have been identified as follows.

Case 0 information is authored locally, causing the creation of a new `ORIGINAL_VERSION<COMPOSITION>`. If this is the first version, a new `VERSIONED_COMPOSITION` will be created first.

Case 1 information is modified locally, such as for the correction of a wrongly entered datum in a composition. This causes the creation of a new `ORIGINAL_VERSION<COMPOSITION>` in an existing `VERSIONED_COMPOSITION`, in which the `AUDIT_DETAILS.change_type` is set to `"correction"`.

Case 2 information received from a feeder system, e.g. a test result, which will be converted and used to create a new `IMPORTED_VERSION<COMPOSITION>` and `ORIGINAL_VERSION<COMPOSITION>` pair. This kind of acquisition could be done automatically. If the receiver system needs to store a copy of the original feeder system audit details, it writes it into the `COMPOSITION.feeder_audit`.

Case 3 an `ORIGINAL_VERSION<COMPOSITION>` (such as a family history) received as part of an `EHR_EXTRACT` from another openEHR system, which will be used by a local author to manually create a new `COMPOSITION` that includes *some content chosen from the received item*. Consequently, the new version is a locally authored one (i.e. an `ORIGINAL_VERSION<COMPOSITION>`). If it is the first version, a `VERSIONED_COMPOSITION` is first created. The `AUDIT_DETAILS` documents the committal of this content, and the clinician may choose to record some details about it in the audit `description`.

In summary, the `AUDIT_DETAILS` is always used to document the addition of information locally, regardless of where it has come from. If there is a need to record original audit details (via the `COMPOSITION.feeder_audit`), they become part of the content of the versioned object.

4.4. EHR Creation Semantics

4.4.1. EHR Identifier Allocation

openEHR EHRs are created due to two types of events. The first is a new patient presenting at the provider institution. An EHR may be created due to this event, without reference to any other openEHR EHRs that may exist in the broader community or jurisdiction. In this case, the EHR will be allocated a new, globally unique EHR id. This establishes the new EHR as an intentional clone of the source EHR (or more correctly, part of the family of EHRs making up the virtual EHR for that patient).

On the other hand, an openEHR EHR may be created in an organisation as a logical clone (probably partial) of an EHR for the patient that exists in some other system. This might happen as a normal part of the front-desk

registration / admission process, i.e. the local EHR system is able to interrogate an EHR location service and discover if any other EHR exists for this patient, or it may occur due to purely electronic communications between two providers, i.e. the EHR is created because an Extract of an EHR has been sent from elsewhere as part of a referral or similar communication. In this second case, the EHR id should be a copy of the EHR id from the other institution. In all cases, the `EHR.system_id` value should be set to the value that would normally be used for locally created EHRs. In the case of creating a cloned EHR, the `system_id` is from the receiving (cloning) system.

In theory such a scheme could guarantee one EHR id per patient, but of course in reality, various factors conspire against this, and it can only approximate it. For one thing, it is known that providers routinely create new EHRs for a patient regardless of how many other EHRs already exist for that patient, simply because they have no easy way to find out about those EHRs. Ideally, this situation would be improved in the openEHR world, but due to reliance on such things as distributed services and reliable person identification, there are no guarantees. The best that can be said is that the EHR id allocation scheme can help support an ideal EHR id-per-patient situation if and when it becomes possible.

4.4.2. EHR Creation

When an EHR is created, the result should be a root EHR object, an EHR Status object, and an EHR Access object, plus any other house-keeping information the versioning implementation requires. In a normal implementation, the EHR Status and EHR Access objects would be created and committed in a Contribution, just as any Composition would be. The EHR Status object has a special status in the EHR, indicating whether the EHR should be included in querying, whether it is modifiable, and by implication, whether it is active. Flags might be set to indicate that it is test record, or for educational or training purposes. The initial creation operation has to supply sufficient parameters for creation of these two objects, including:

- `system_id` - identifier of EHR repository of the system;
- `ehr_id` - the unique identifier of this EHR;
- `subject_id` - optional; the use of `PARTY_SELF` allows completely anonymous EHRs;
- `is_queryable` flag;
- `is_modifiable` flag - indicates whether the content of the EHR is allowed to be modified (the `EHR_STATUS` object itself is always modifiable);
- any other flags required by the EHR Status object in the local implementation.

NOTE is is strongly recommended that a UUID always be used for the `ehr_id` field. This is common practice among implementers.

The EHR id will either be a new globally unique identifier, in the case of first time EHR creation for this patient in the health system, or else the same identifier as an existing EHR for the same subject in another system, in the case of an EHR move or copy. The effect of EHR copying / synchronising between systems is that EHRs with the same identifier can be found within multiple systems. However if the same patient presented at multiple provider locations with no EHR sharing capability, a new EHR with a unique identifier will be created at each

place. If a later request for copying occurs (e.g. due to a request for an EHR Extract) between two providers, the requesting institution will perform the merge of the received copy into the existing EHR for the same patient.

The main consequences in a distributed environment are as follows:

- multiple EHR ids for a given patient indicate a mobile patient, but lack of systematic EHR sharing;
- one EHR id everywhere for the patient indicates a seamlessly integrated distributed environment, most likely with a global identification service.

Note that the first situation is not a problem, and is not the same as the situation in which two EHRs are identified as being for different patients (i.e. subject id rather than EHR id is different) when in fact they are for the same person.

4.5. EHR Active Status

The `EHR_STATUS.`*`is_modifiable`* attribute is used to indicate whether *the contents of an EHR* are modifiable, and is set to `True` if the EHR is considered *active*. An EHR's 'contents' consist of everything other than the `EHR_STATUS` object, i.e. its Compositions (Versioned objects referred to by the *`compositions`* attribute), its hierarchical Folders (Versioned objects referred to by the *`folders`* attribute) and any other content that may be added in later releases of this specification. An active EHR is used within a given EHR system as the primary patient record for a subject. An EHR may be deactivated in circumstances such as the following:

- the patient has died, and no further updates (typically relating to the death) are required;
- the EHR is discovered to be a duplicate or additional record for a patient already having a primary EHR;
- the patient formally opts out of further data recording and/or sharing, with the effect that no further additions can be made by healthcare providers to this EHR;
- the EHR is to be moved to another system (usually due to patient move), but the original copy will not or cannot be deleted for legal or other administrative reasons.

In any of these situations, the EHR can be deactivated by setting *`is_modifiable`* to `False`. An inactive EHR is still visible in the system, and will remain queryable, unless the *`is_queryable`* attribute is also set to `False`.

4.6. Time in the EHR

There are numerous times recorded in the EHR, at varying levels of granularity. Certain times are a guaranteed by-product of the scientific investigation process, including time of sampling or collection; time of measurement, time of a healthcare business event, time of data committal. The following figures illustrate the relationship between points on the timeline of real-world clinical activity, and the corresponding data elements in the openEHR EHR.

The first figure shows the relationships for a physical examination at a GP visit.

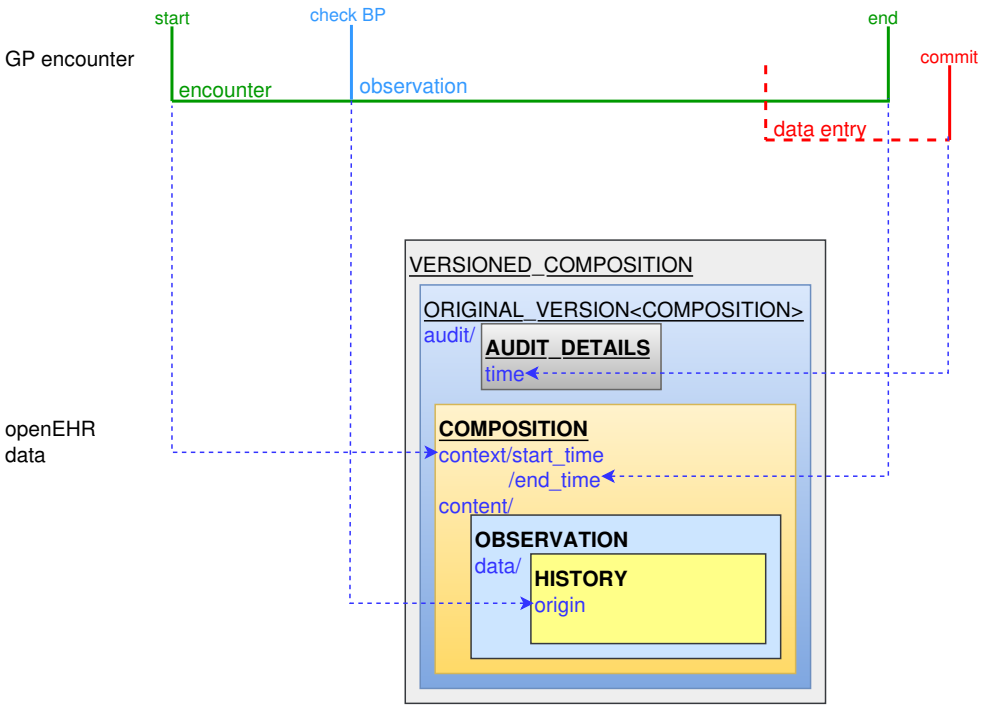


Figure 9. Time in the EHR - GP encounter

In this scenario, an observation (say, of blood pressure) is instantaneous for all practical purposes. This may be contrasted with lab testing, shown below, where the sample time (moment of specimen collection) is usually different from the time of analysis and reporting of the sample. The time lag may be as long as the specimen is biochemically stable such that the analysis will still accurately reflect the clinical situation in the patient at the time of specimen-taking.

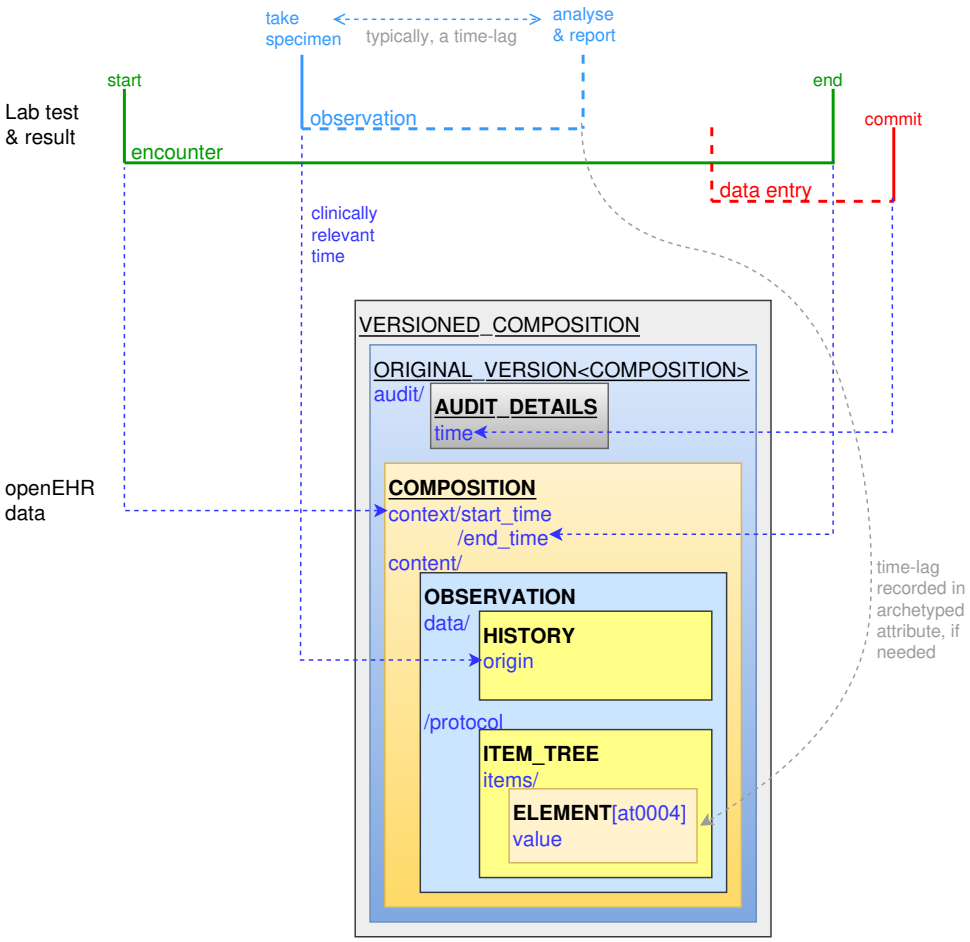


Figure 10. Time in the EHR - Lab test

The following figure shows the relationships between real world time points and openEHR data for radiology.

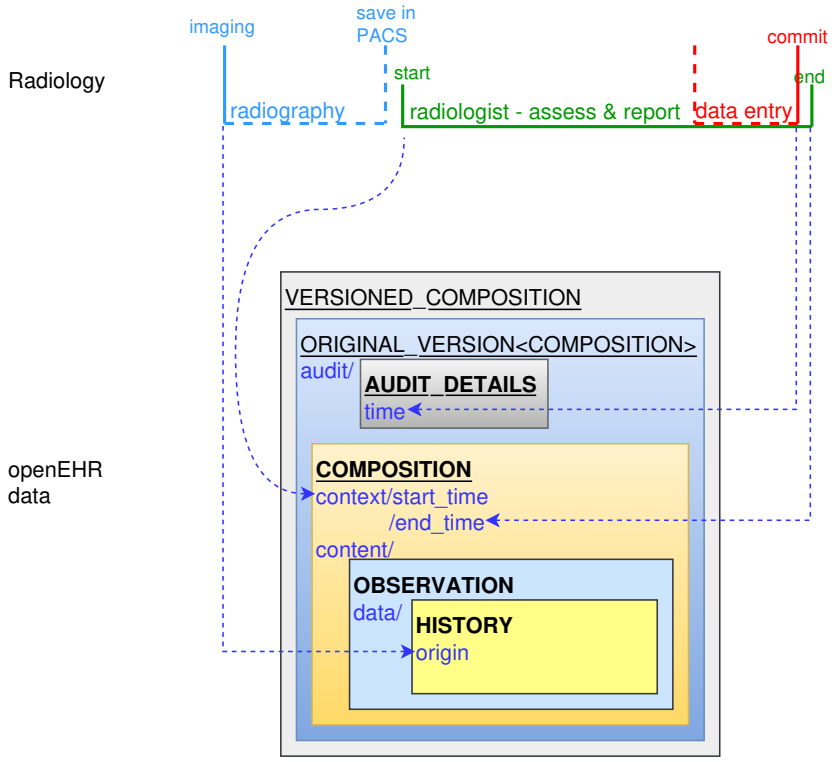


Figure 11. Time in the EHR - Radiology

The last example shown below shows the relationship of time and EHR data for in-patient nursing observations, where the notion of 'encounter' is somewhat artificial.

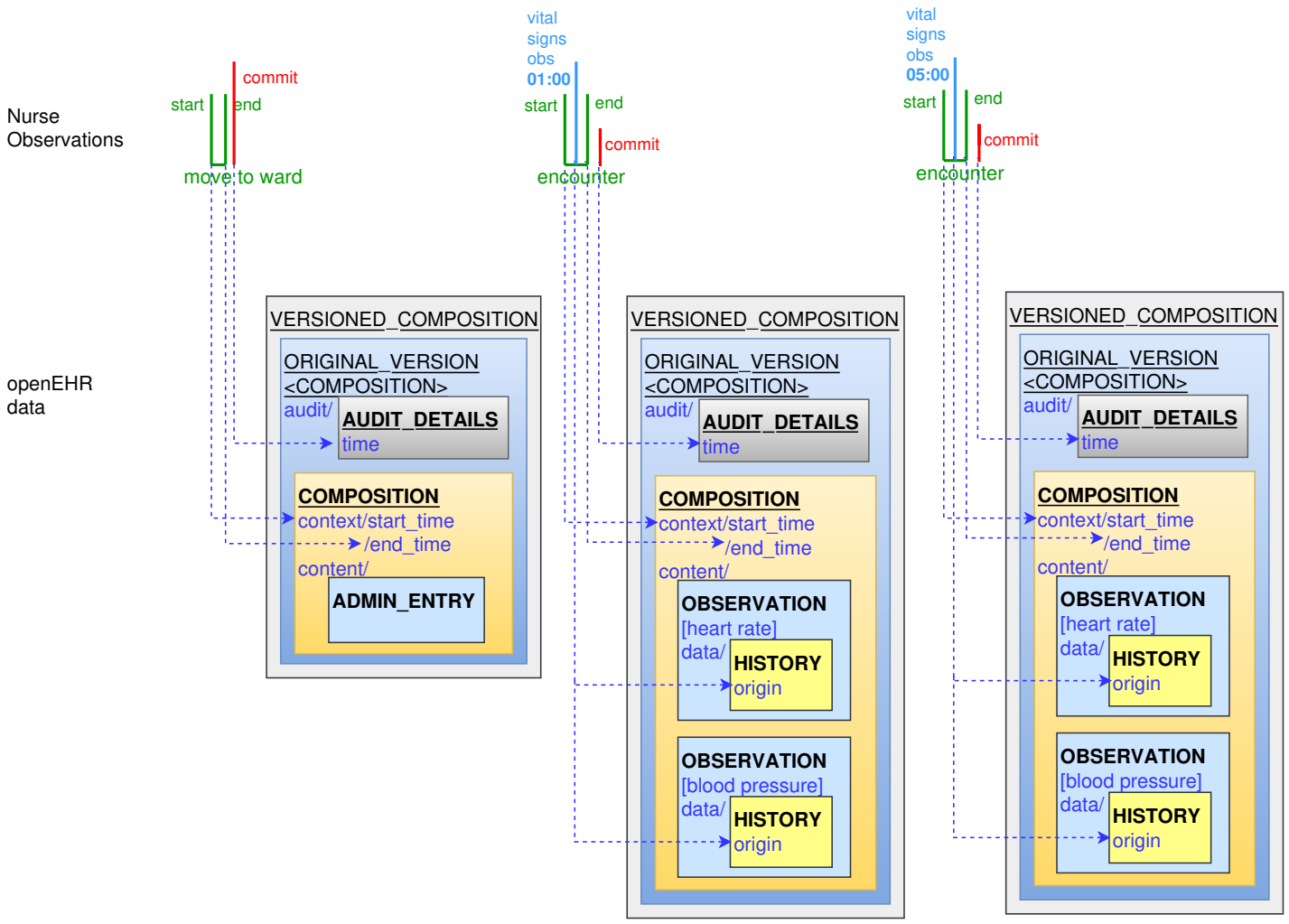


Figure 12. Time in the EHR - Nursing observations

Other times to do with diagnoses (time of onset, time of resolution, time of last episode) and medication management (time started taking medication, time stopped, time suspended, etc) are specific to the particular type of information (e.g. diagnosis versus prognosis versus recommendation) and are generally represented as archetyped date/time values within Evaluation objects. Basic timing information is modelled concretely in the Instruction and Action Entry types, while archetyping is used to express timing information that is specific to particular content.

4.7. Historical Views of the Record

It is important to understand that the `COMPOSITION` versions at a previous point in time represent a previously available *informational state* of the EHR, at a particular EHR node. Such previous state include only those Compositions from other sources as have been acquired by that point in time, *regardless of whether the acquired information pertains to clinical information recorded earlier*. A previous historical state of the EHR thus corresponds to what users of a system could see at a particular moment of time. It is important to differentiate this from previous clinical states of the patient: previous informational states of the EHR might include acquired information which is significantly older than the point in time when merging occurred. A previous clinical state of the patient would be a derivable view of the EHRs in all locations for the patient - what is sometimes called the virtual EHR - at a given point in time, minus acquired Compositions, since these constitute (usually out-ofdate) copies of Compositions primarily available elsewhere.

It is previous informational states with which we are concerned for medico-legal purposes, since they represent the information actually available to clinicians at a health-care facility, at a point in time. But previous clinical views may be useful for reconstructing an actual sequence of events as experienced by the patient.

4.8. Class Descriptions

4.8.1. EHR Class

Class	EHR	
Description	The EHR object is the root object and access point of an EHR for a subject of care.	
Attributes	Signature	Meaning
1..1	**system_id**: HIER_OBJECT_ID	The identifier of the logical EHR management system in which this EHR was created.
1..1	**ehr_id**: HIER_OBJECT_ID	The unique identifier of this EHR. **NOTE** is is strongly recommended that a UUID always be used for this field.
0..1	**contributions**: List<OBJECT_REF>	List of contributions causing changes to this EHR. Each contribution contains a list of versions, which may include references to any number of `VERSION` instances, i.e. items of type `VERSIONED_COMPOSITION` and `VERSIONED_FOLDER`.
1..1	**ehr_status**: OBJECT_REF	Reference to `EHR_STATUS` object for this EHR.
1..1	**ehr_access**: OBJECT_REF	Reference to `EHR_ACCESS` object for this EHR.
0..1	**compositions**: List<OBJECT_REF>	Master list of all Versioned Composition references in this EHR.
0..1	**directory**: OBJECT_REF	Optional directory structure for this EHR. If present, this is a reference to the first member of *`folders`*.
1..1	**time_created**: DV_DATE_TIME	Time of creation of the EHR.

0..1	**folders**: List<OBJECT_REF>	Optional additional Folder structures for this EHR. If set, the *directory* attribute refers to the first member.
Invariants	*Contributions_valid*: `for_all c in contributions | c.type.is_equal("CONTRIBUTION")`	
	Ehr_access_valid: `ehr_access.type.is_equal ("VERSIONED_EHR_ACCESS")`	
	Ehr_status_valid: `ehr_status.type.is_equal("VERSIONED_EHR_STATUS")`	
	Compositions_valid: `for_all c in compositions | c.type.is_equal ("VERSIONED_COMPOSITION")`	
	Directory_valid: `directory /= Void implies directory.type.is_equal ("VERSIONED_FOLDER")`	
	Folders_valid: `folders /= Void implies for_all f in folders | f.type.is_equal("VERSIONED_FOLDER")`	
	Directory_in_folders: `folders /= Void implies folders.item(1) = directory`	

4.8.2. VERSIONED_EHR_ACCESS Class

Class	VERSIONED_EHR_ACCESS
Description	Version container for EHR_ACCESS instance.
Inherit	VERSIONED_OBJECT

4.8.3. EHR_ACCESS Class

Class	EHR_ACCESS

Description	EHR-wide access control object. All access decisions to data in the EHR must be made in accordance with the policies and rules in this object. **NOTE**: It is strongly recommended that the inherited attribute *uid* be populated in EHR_ACCESS objects, using the UID copied from the *object_id()* of the *uid* field of the enclosing VERSION object. For example, the ORIGINAL_VERSION.uid 87284370-2D4B-4e3d-A3F3-F303D2F4F34B::uk.nhs.ehr1::2 would be copied to the *uid* field of the EHR_ACCESS object.	
Inherit	LOCATABLE	
Attributes	**Signature**	**Meaning**
0..1	**settings**: ACCESS_CONTROL_SETTINGS	Access control settings for the EHR. Instance is a subtype of the type ACCESS_CONTROL_SETTINGS, allowing for the use of different access control schemes.
Functions	**Signature**	**Meaning**
1..1	**scheme** (): String	The name of the access control scheme in use; corresponds to the concrete instance of the settings attribute.
Invariants	*Scheme_valid*: not scheme.is_empty	
	Is_archetype_root: is_archetype_root	

4.8.4. VERSIONED_EHR_STATUS Class

Class	**VERSIONED_EHR_STATUS**
Description	Version container for EHR_STATUS instance.
Inherit	VERSIONED_OBJECT

4.8.5. EHR_STATUS Class

Class	**EHR_STATUS**

Description	Single object per EHR containing various EHR-wide status flags and settings, including whether this EHR can be queried, modified etc. This object is always modifiable, in order to change the status of the EHR as a whole. **NOTE**: It is strongly recommended that the inherited attribute *uid* be populated in EHR_STATUS objects, using the UID copied from the *object_id()* of the *uid* field of the enclosing VERSION object. For example, the ORIGINAL_VERSION.uid 87284370-2D4B-4e3d-A3F3-F303D2F4F34B::uk.nhs.ehr1::2 would be copied to the *uid* field of the EHR_STATUS object.	
Inherit	LOCATABLE	
Attributes	**Signature**	**Meaning**
1..1	**subject**: PARTY_SELF	The subject of this EHR. The *external_ref* attribute can be used to contain a direct reference to the subject in a demographic or identity service. Alternatively, the association between patients and their records may be done elsewhere for security reasons.
1..1	**is_queryable**: Boolean	True if this EHR should be included in population queries, i.e. if this EHR is considered active in the population.
1..1	**is_modifiable**: Boolean	True if the EHR, other than the EHR_STATUS object, is allowed to be written to. The EHR_STATUS object itself can always be written to.
0..1	**other_details**: ITEM_STRUCTURE	Any other details of the EHR summary object, in the form of an archetyped ITEM_STRUCTURE .
Invariants	*Is_archetype_root*: is_archetype_root	

4.8.6. VERSIONED_COMPOSITION Class

Class	**VERSIONED_COMPOSITION**
Description	Version-controlled composition abstraction, defined by inheriting VERSIONED_OBJECT<COMPOSITION> .

Inherit	VERSIONED_OBJECT	
Functions	**Signature**	**Meaning**
1..1	**is_persistent** (): Boolean	Indicates whether this composition set is persistent; derived from first version.
Invariants	*Archetype_node_id_valid*: `for_all v in all_versions | v.archetype_node_id.is_equal (all_versions.first.archetype_node_id)`	
	Persistent_validity: `for_all v in all_versions | v.is_persistent = all_versions.first.data.is_persistent`	

5. Composition Package

5.1. Overview

The Composition is the primary 'data container' in the openEHR EHR and is the root point of clinical content. Instances of the `COMPOSITION` class can be considered as self-standing data aggregations, or documents in a document-oriented system. The key information in a `COMPOSITION` is found in its *`content`*, *`context`*, and *`composer`* attributes. The UML diagram below illustrates the `composition` package.

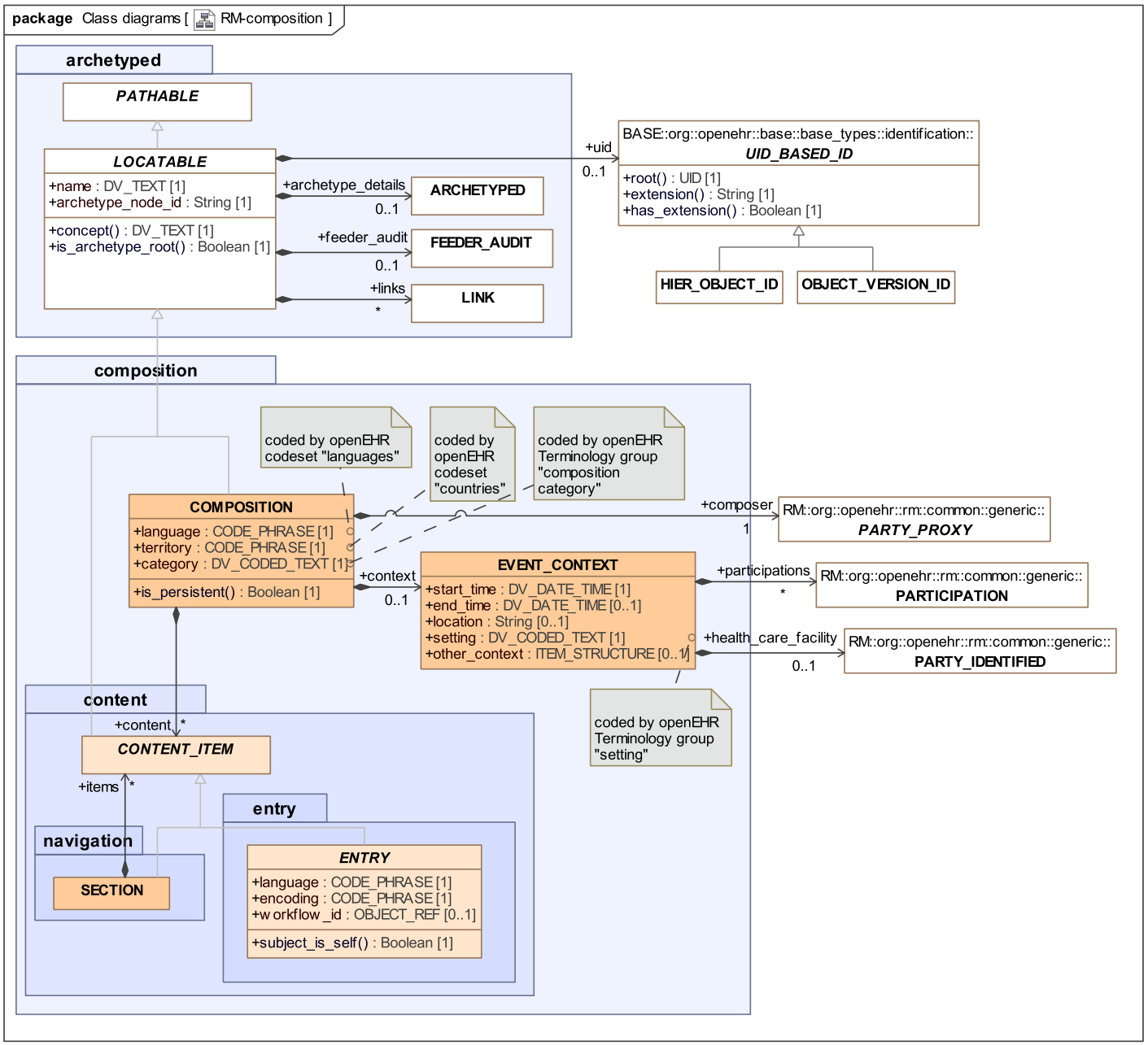


Figure 13. rm.composition package

5.2. Context Model of Recording

5.2.1. Overview

The openEHR EHR model takes account of a systematic analysis of "context". Contexts in the real world are

mapped to particular levels of the information model in a clear way, according to the scheme shown in FIGURE 12. On the left hand side is depicted the context of a data-entry session in which the information generated by a "healthcare event", containing "clinical statements", is added to the EHR. A healthcare event is defined as any business activity of the health care system for the patient, including encounters, pathology tests, and interventions. A clinical statement is the minimal indivisible unit of information the clinician wants to record. Clinical statements are shown in the diagram has having temporal and spatial structure as well as data values. Each of these three contexts has its own audit information, consisting of who, when, where, why information.

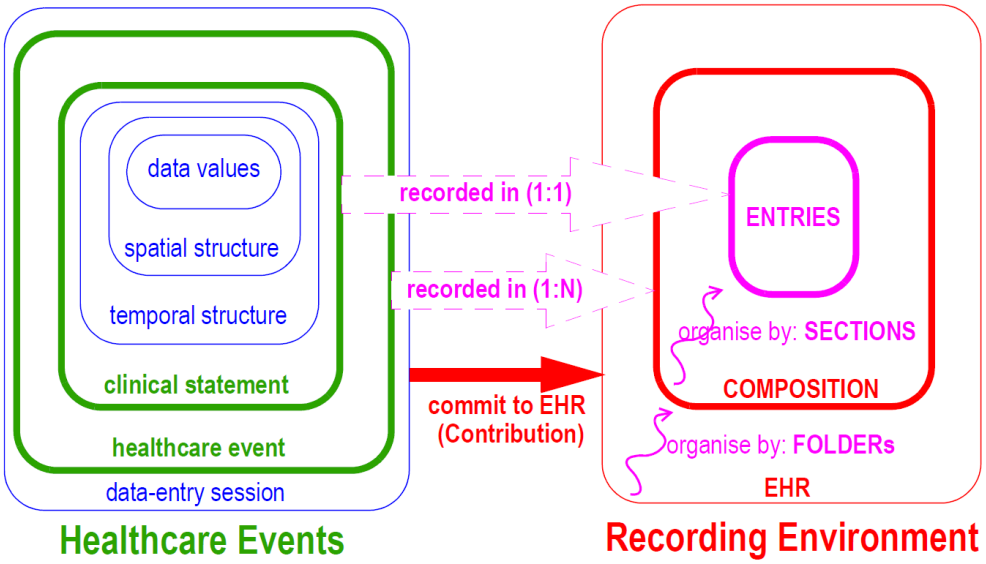


Figure 14. General Model of Recording

On the right-hand side of Figure 14, the EHR recording environment is represented. The EHR consists of distinct, coarse-grained items known as Compositions added over time and organised by Folders. Each Composition consists of Entries, organised by Sections within the Composition. The audit information for each context is recorded at the corresponding level of the EHR.

5.2.2. Composer

The composer is the person who was primarily responsible for the content of the Composition. This is the identifier that should appear on the screen. It could be a junior doctor who did all the work, even if not legally responsible, or it could be a nurse, even if later attested by a more senior clinician; it will be the patient in the case of patient-entered data. It may or may not be the person who entered and committed the data. it may also be a software agent. This attribute is mandatory, since all content must be been created by some person or agent.

Since in many cases Compositions will be composed and committed by the same person, it might seem that two identifiers `COMPOSITION.composer` and `VERSION.audit.committer` (which are both of type `PARTY_PROXY`) will be identical. In fact, this will probably not be the case, because the kind of identifier to represent the composer will be a demographic identifier, e.g. "RN Jane Williams", "RN 12345678", whereas the identifier in the audit details will usually be a computer system user identifier, e.g. "jane.williams@westmead.health.au". This difference highlights the different purposes of these attributes: the first exists to identify the clinical agent who created the information, while the second exists to identify the

logged-in user who committed it to the system.

In the situation of patient-entered data, the special "self" `PARTY_PROXY` instance (see Common IM generic package) is used for both `COMPOSITION.composer` and `VERSION.audit.committer`.

5.2.3. Event Context

5.2.3.1. Overview

The optional event_context in the `COMPOSITION` class is used to document the healthcare event causing new or changed content in the record. Here, 'healthcare event' means 'a (generally billable) business activity of the healthcare system with, for or on behalf of the patient'. Generally this will an encounter involving the subject of care and physician, but is variable in a hospital environment. In this sense, a visit to a GP is a single care event, but so is an episode in a hospital, which may encompass multiple encounters. The information recorded in Event context includes start and (optional) end time of the event, health care facility, setting (e.g. primary care, aged care, hospital), participating healthcare professionals, and optional further details defined by an archetype.

Healthcare events that require an Event_context instance in their recorded information include the following.

- Scheduled or booked patient encounters leading to changes to the EHR, including with a GP, hospital consultant, or other clinical professional such as mobile nurse. In this case, the Event context documents the time and place of the encounter, and the identity of the clinical professional.
- Case conferences about a patient, leading to modifications to the health record; here the Event context documents the case conference time, place and participants.
- Pathology, imaging or other test process. In this case, the Event context documents the place and period during which testing and analysis was carried out, and by whom.
- Data resulting from care in the home provided by health professional(s) (often allied health care workers). Situations in which Event context is optional include the following.
- Nurse interactions with patients in hospital, including checking vital signs, adjusting medication or other aspects of bed situation for the patient. Each instance of a nurse's observations are generally not considered to be a separate 'care event', rather they are seen as the continuation of the general activities of monitoring. In such situations, the overall context is given by `ADMIN_ENTRY` instances in the record indicating date/time and place of admission and discharge.

Situations in which Event context is not used include:

- Any modification to the EHR which corrects or updates existing content, including by administrative staff, and by clinical professionals adding or changing evaluations, summaries etc.
- Patient-entered data where no interaction with health professionals took place; typically readings from devices in the home such as weighing scales, blood glucose measuring devices, wearable monitors etc.

Ultimately, the use of Event context will be controlled by Composition-level archetypes.

5.2.3.2. Occurrence in Data

For situations requiring an `EVENT_CONTEXT` object to be recorded, it is worth clarifying which Compositions carry such objects. Consider the example shown in Figure 15. In this example, a Contribution is made to the EHR, consisting of one or more Compositions that were each created or modified due to some clinical activity. Within such a set, there will usually be one Composition relating directly to the event, such as the patient contact - this is the Composition containing the doctor's observations, nurses' activities etc, during the visit, and is therefore the one which contains the `EVENT_CONTEXT` instance. Other Compositions changed during the same event (e.g. updates to medication list, family history and so on) do not require an Event context, since they are part of the same Contribution, and the event context of the primary Composition can always be retrieved if desired. Contributions A, B and C in the figure illustrate this case.

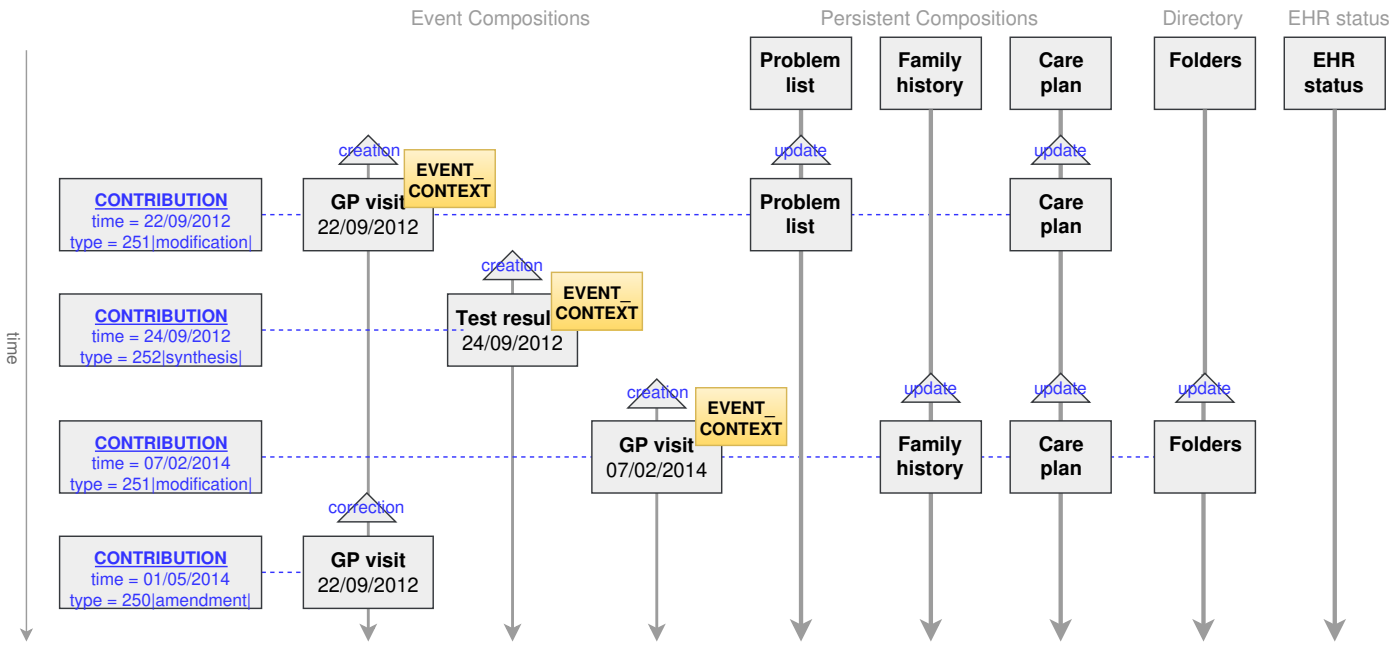


Figure 15. Use of Event context

In cases where Contributions are made to the record with no event context, the Event context of any Compositions from the original commit will remain intact and visible (unless the correction is to the event context itself of course), and will correctly reflect the fact that no new clinical interaction occurred. This is the case with Contribution D in the figure.

NOTE

Persistent Compositions may optionally have an Event context. In openEHR releases up to 1.0.3, Persistent Compositions had no Event context. This was relaxed in subsequent releases, to allow the inclusion of context information (e.g. encounter data) for changes to Persistent Compositions where no Event Composition was created.

5.2.3.3. Time

The times recorded in the Event context represent the time of an encounter or other activity undertaken by a health provider to/for/on behalf of the patient. The time is represented as a mandatory start time and optional end time. It is assumed that where there is a clinical session (i.e. an `EVENT_CONTEXT` object does exist), the start time is known or can be reasonably approximated. It is quite common that the end time of a consultation or

encounter is not recorded, but rather inferred from e.g. average consult times, or the start time of the next consult for the same physician.

Event context is used as described above even if the additions are made to the EHR long after the event took place, such as happens when a doctor writes his/her notes into the record system at night, after all patients have been seen. In such cases, the versioned Composition audit trail records the context of when the data were entered, as distinct from the context of when the clinical interaction took place.

5.2.3.4. Participations

Is part of the Event context, the participations attribute can be used to describe who participated, and how. Each participation object describes the "mode" of participation as well, such as direct presence, video-conference and so on. In many cases such information is of no interest, since the subject of any Entry is known (`ENTRY` . *`subject`*) and the clinician will be known (`COMPOSITION` . *`composer`*), and the mode of communication is assumed to be a personal encounter. The participations attribute is therefore used when it is desired to record further details of how the patient and or physician interacted (e.g. over the internet), and/or other participants, such as family, nurses, specialists etc.

There are no general rules about who is included as a participant. For example, while there will be a patient participation during a GP visit, there will be no such participation recorded when the clinical event is a tissue test in a laboratory. Conversely, a patient might record some observations and drug self-administration in the record, in which case the composer will be the patient, and there will be no clinician participation. Consequently, the use of participations will mostly be archetype-driven.

5.2.3.5. Healthcare Facility, Location and Setting

The *`health_care_facility`* (HCF) attribute is used to record the health care facility in whose care the event took place. This is the most specific identifiable (i.e. having a provider id within the health system) workgroup or care delivery unit within a care delivery enterprise which was involved in the care event. The identification of the HCF can be used to ensure medico-legal accountability. Often, the HCF is also where the encounter physically took place, but not in the case of patient home visits, internet contacts or emergency care; the HCF should not be thought of as a physical place, but as a care delivery management unit. The physical place of care can be separately recorded in `EVENT_CONTEXT` . *`location`* . The health_care_facility attribute is optional to allow for cases where the clinical event did not involve any care delivery enterprise, e.g. self-care at home by the patient, emergency revival by a non-professional (e.g. CPR by lifeguard on a beach), care by a professional acting in an unofficial capacity (doctor on a plane asked to aid a passenger in difficulty). In all other cases, it is mandatory. Archetypes are used to control this.

Two other context attributes complete the predefined notion of event context in the model: *`location`* and *`setting`* . The location attribute records: the physical location where the care delivery took place, and should document a reasonably specific identifiable location. Examples include "bed 5, ward E", "home". This attribute is optional, since the location is not always known, particularly in legacy data.

The setting attribute is used to document the "setting" of the care event. In clinical record keeping, this has been found to be a useful coarse-grained classifier of information. The openEHR Terminology "setting" group is used to code this attribute. It is mandatory, on the basis that making it optional will reduce its utility for querying and classification.

5.3. Composition Content

The data in a Composition is stored in the *content* attribute. There are four kinds of data structuring possible in the content attribute:

- it may be empty. Although for most situations, there should be content in a Compostion, there are at least two cases where an empty Composition makes sense:
  - the first is a Composition in 'draft' editing state (VERSION.*lifecycle_state* = 'incomplete')
  - the second is for systems that are only interested in the fact of an event having taken place, but want no details, such as so-called clinical 'event summary' systems, which might record the fact of visits to the doctor, but contain no further information. This can be achieved using Compositions with event context, and no further content.
- it may contain one or more SECTIONs which are defined in the archetype of the Composition;
- it may contain one or more SECTION trees, each of which is a separately archetyped structure;
- it may contain one or more ENTRYs directly, with no intermediate SECTIONs ;
- it may be any combination of the previous three possibilities.

The actual structures used in a Composition at runtime are controlled by a template, which in turn controls the particular combination of archetypes used.

5.4. Class Descriptions

5.4.1. COMPOSITION Class

Class	COMPOSITION

Description	Content of one version in a `VERSIONED_COMPOSITION`. A Composition is considered the unit of modification of the record, the unit of transmission in record Extracts, and the unit of attestation by authorising clinicians. In this latter sense, it may be considered equivalent to a signed document. **NOTE**: It is strongly recommended that the inherited attribute *`uid`* be populated in Compositions, using the UID copied from the *`object_id()`* of the *`uid`* field of the enclosing `VERSION` object. For example, the `ORIGINAL_VERSION.uid 87284370-2D4B-4e3d-A3F3-F303D2F4F34B::uk.nhs.ehr1::2` would be copied to the *`uid`* field of the Composition.	
Inherit	`LOCATABLE`	
Attributes	**Signature**	**Meaning**
1..1	**language**: `CODE_PHRASE`	Mandatory indicator of the localised language in which this Composition is written. Coded from openEHR Code Set `languages`. The language of an Entry if different from the Composition is indicated in `ENTRY.`*`language`*.
1..1	**territory**: `CODE_PHRASE`	Name of territory in which this Composition was written. Coded from openEHR countries code set, which is an expression of the ISO 3166 standard.
1..1	**category**: `DV_CODED_TEXT`	Temporal category of this Composition, i.e. • `431|persistent|` - of potential life-time validity; • `451|episodic|` - valid over the life of a care episode; • `433|event|` - valid at the time of recording (long-term validity requires subsequent clinical assessment). or any other code defined in the openEHR terminology group 'category'.

0..1	**context**: `EVENT_CONTEXT`	The clinical session context of this Composition, i.e. the contextual attributes of the clinical session.
1..1	**composer**: `PARTY_PROXY`	The person primarily responsible for the content of the Composition (but not necessarily its committal into the EHR system). This is the identifier which should appear on the screen. It may or may not be the person who entered the data. When it is the patient, the special self instance of `PARTY_PROXY` will be used.
0..1	**content**: `List<CONTENT_ITEM>`	The content of this Composition.
Functions	**Signature**	**Meaning**
1..1	**is_persistent** (): `Boolean`	True if category is `431|persistent|` , False otherwise. Useful for finding Compositions in an EHR which are guaranteed to be of interest to most users.
Invariants	*Category_validity*: `terminology (Terminology_id_openehr).has_code_for_group_id (Group_id_composition_category, category.defining_code)`	
	Territory_valid: `code_set(Code_set_id_countries).has_code(territory)`	
	Language_valid: `code_set(Code_set_id_languages).has_code(language)`	
	Content_valid: `content /= Void implies not content.is_empty`	
	Is_archetype_root: `is_archetype_root`	

5.4.2. EVENT_CONTEXT Class

Class	**EVENT_CONTEXT**
Description	Documents the context information of a healthcare event involving the subject of care and the health system. The context information recorded here are independent of the attributes recorded in the version audit, which document the system interaction context, i.e. the context of a user interacting with the health record system. Healthcare events include patient contacts, and any other business activity, such as pathology investigations which take place on behalf of the patient.

Inherit	PATHABLE	
Attributes	**Signature**	**Meaning**
1..1	**start_time**: DV_DATE_TIME	Start time of the clinical session or other kind of event during which a provider performs a service of any kind for the patient.
0..1	**end_time**: DV_DATE_TIME	Optional end time of the clinical session.
0..1	**location**: String	The actual location where the session occurred, e.g. 'microbiology lab 2', 'home', 'ward A3' and so on.
1..1	**setting**: DV_CODED_TEXT	The setting in which the clinical session took place. Coded using the openEHR Terminology, setting group.
0..1	**other_context**: ITEM_STRUCTURE	Other optional context which will be archetyped.
0..1	**health_care_facility**: PARTY_IDENTIFIED	The health care facility under whose care the event took place. This is the most specific workgroup or delivery unit within a care delivery enterprise that has an official identifier in the health system, and can be used to ensure medico-legal accountability.
0..1	**participations**: List<PARTICIPATION>	Parties involved in the healthcare event. These would normally include the physician(s) and often the patient (but not the latter if the clinical session is a pathology test for example).
Invariants	*Setting_valid*: `Terminology (Terminology_id_openehr).has_code_for_group_id (Group_id_setting, setting.defining_code)`	
	Participations_validity: `participations /= Void implies not participations.is_empty`	
	location_valid: `location /= Void implies not location.is_empty`	

6. Content Package

6.1. Overview

The `content` package contains the `CONTENT_ITEM` class, ancestor class of all content types, and the navigation and entry packages, which contain `SECTION`, `ENTRY` and related types.

6.2. Class Descriptions

6.2.1. CONTENT_ITEM Class

Class	*CONTENT_ITEM (abstract)*
Description	Abstract ancestor of all concrete content types.
Inherit	`LOCATABLE`

7. Navigation Package

7.1. Overview

The `navigation` Package defines a hierachical heading structure, in which all individual headings are considered to belong to a "tree of headings". Each heading is an instance of the class `SECTION`, visible in the lower left side of Figure 13.

Sections provide both a logical structure for the author to arrange Entries, and a navigational structure for readers of the record, whether they be human or machine. Sections are archetyped in trees with each tree containing a root Section, one or more sub-sections, and any number of Entries at each node. Section trees that are separately archetyped, such as the SOAP headings, or the heading structure for a physical examination, can be combined at runtime by type to form one large heading structure, as shown in the figure below.

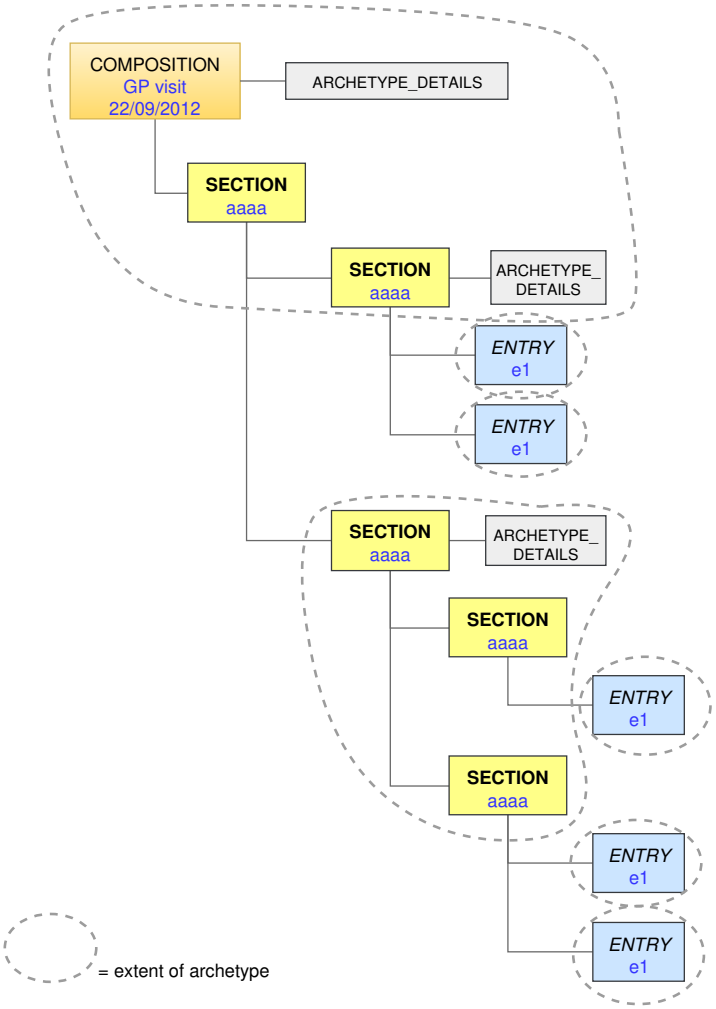


Figure 16. Section View of a General Practice Contact Composition

In terms of understanding of clinical data, Section structures are not essential in a Composition - they can always be removed or ignored (typically in machine processing such as querying) without losing the meaning of the Entries in the Composition. While Sections are often used to group Entries according to status, e.g. 'family history', 'problems', 'observations', it is the Entries themselves that indicate the definitive category of information contained therein. This principle is explained in more detail in Section 8.2.

Despite the above, Section structures do not have to be regarded as ad hoc or unreliable structures. On the contrary, as they are archetyped, their structures can be relied upon in the same way as any other structure in the record can be relied on to conform to its archetype. Accordingly, solid assumptions can be made about Sections, based on their archetypes, for the purposes of querying. In fact, the main benefit of Sections is that they may provide significant performance benefits to querying, whether by interactive application or by automated systems.

One potentially confusing aspect of any Section structure is that while the root Section in a tree is logically a Section, it does not appear in a display or printed form as a visible section. This is due to the fact that humans don't usually write down top-level headings for anything, since there is always a containing structure acting as a top-level organising context (such as the piece of paper one is writing on). For example, consider the way a clinician writes down the problem/SOAP headings on paper. She writes the name of the first problem, then under that, the S/O/A/P headings, then repeats the process for further problems. But she doesn't write down a heading above the level of the problems, even though there must be one from a data structure point of view.

7.2. Class Descriptions

7.2.1. SECTION Class

Class	**SECTION**	
Description	Represents a heading in a heading structure, or section tree. Created according to archetyped structures for typical headings such as SOAP, physical examination, but also pathology result heading structures. Should not be used instead of `ENTRY` hierarchical structures.	
Inherit	`CONTENT_ITEM`	
Attributes	**Signature**	**Meaning**
0..1	**items**: `List<CONTENT_ITEM>`	Ordered list of content items under this section, which may include: • more `SECTION`s ; • `ENTRY`s .
Invariants	*Items_valid*: `items /= Void implies not items.is_empty`	

7.3. Section Instance Structures

7.3.1. Problem/SOAP Headings

An example of an section tree representing the problem/SOAP heading structure is shown below.

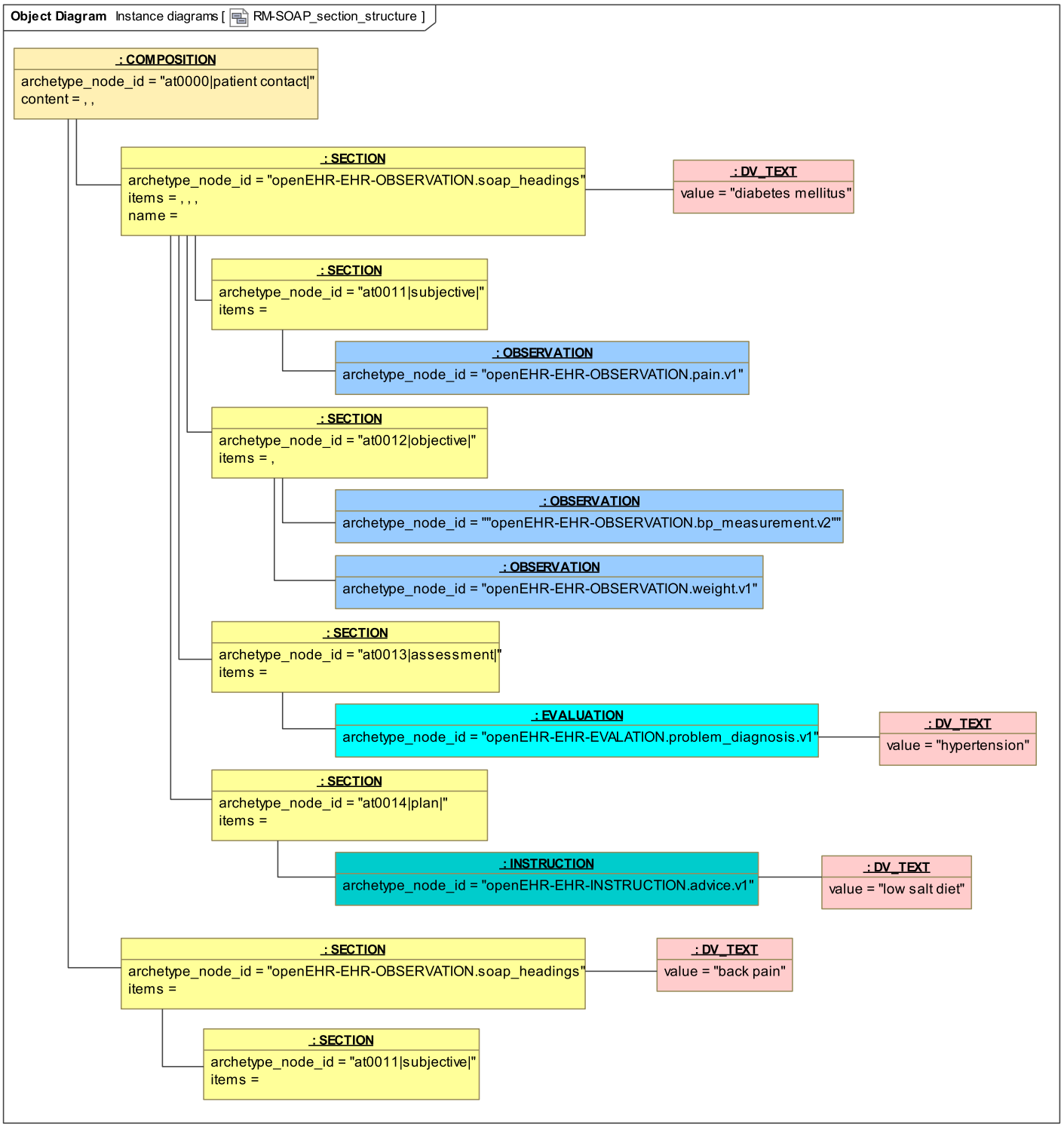


Figure 17. "problem/SOAP" Section Structure

8. Entry Package

8.1. Design Principles

8.1.1. Information Ontology

All information which is created in the openEHR health record is expressed as an instance of a class in the entry package, containing the `ENTRY` class and a number of descendants. An `ENTRY` instance is logically a single 'clinical statement', and may be a single short narrative phrase, but may also contain a significant amount of data, e.g. a microbiology result, a psychiatric examination, a complex prescription. In terms of clinical content, the Entry classes are the most important in the openEHR EHR Information Model, since they define the semantics of all the 'hard' information in the record. They are intended to be archetyped, and in fact, archetypes for Entries make up the vast majority of important clinical archetypes defined.

The design of `entry` package is based on the Clinical Investigator Recording process and ontology, described in Beale & Heard (2007). The process is shown in following figure.

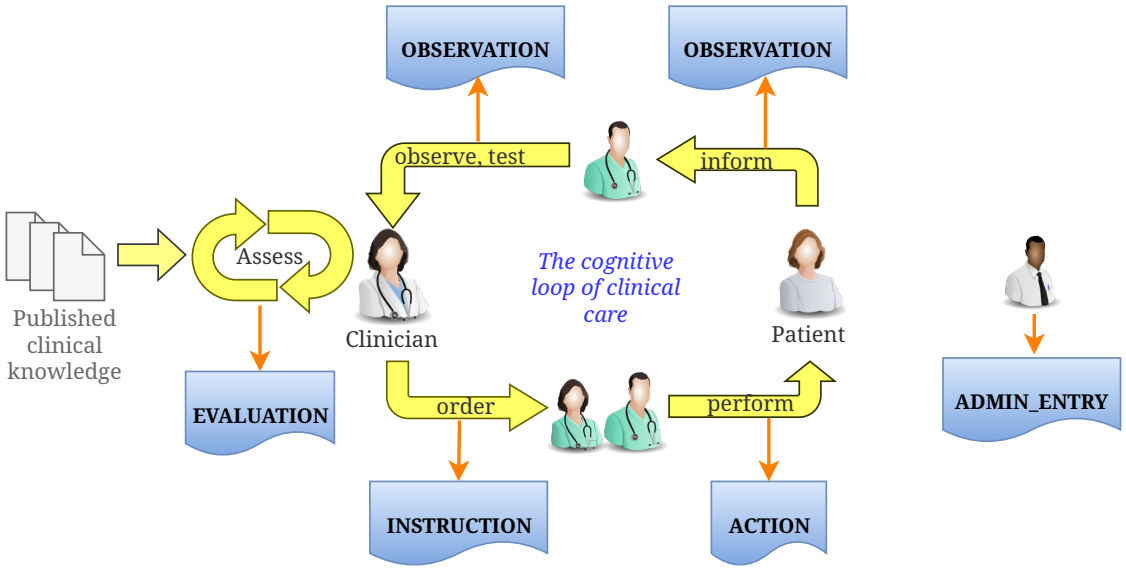


Figure 18. Clinical Investigator Recording Process

This figure shows the cycle of information created by an iterative, problem solving process undertaken by a "clinical investigator system", which consists of health carers, and may include the patient (at points in time when the patient performs observational or therapeutic activities). Starting from the patient (right hand side of figure) observations are made, which lead to opinions on the part of the investigator, including assessment of the current situation, goals for a future situation, and plans for achieving the goals. Personal and published evidence and knowledge almost always play an important part in this process. The latter lead to instructions designed to help the patient achieve the goals. A complex or chronic problem may take numerous iterations - possibly a whole lifetime's worth - with each step being quite small, and future steps depending heavily on past progress. The role of the investigator (and associated agents) is normally filled by health care professionals, but may also be filled by the patient, or a guardian or associate of the patient. Indeed, this is what happens every time a person goes home from the pharmacy with prescribed medication to take at home.

The process illustrated in Figure 18 is a synthesis of the 'problem-oriented' approach of Weed (1969) and the "hypothetico-deductive model" of clinical reasoning described by Elstein Shulman & Sprafka (1978). However, as pointed out in Elstein & Schwarz (2002), hypothesis-making and testing is not the only successful process used by clinical professionals - evidence shows that many (particularly those older and more experienced) rely on pattern recognition and direct retrieval of plans used previously with similar patients or prototype models. The investigator process is compatible with both cognitive approaches, since it does not say how opinions are formed, nor imply any specific number or size of iterations to bring the process to a conclusion. As such, the openEHR information model does not impose any process model, only the types of information used.

On the basis of this process, a Clinical Investigator Recording ontology is developed (Beale & Heard, 2007), as shown below. From this ontology, the openEHR class model for Entries is derived. The openEHR Entry class names are annotated next to their originating ontological categories.

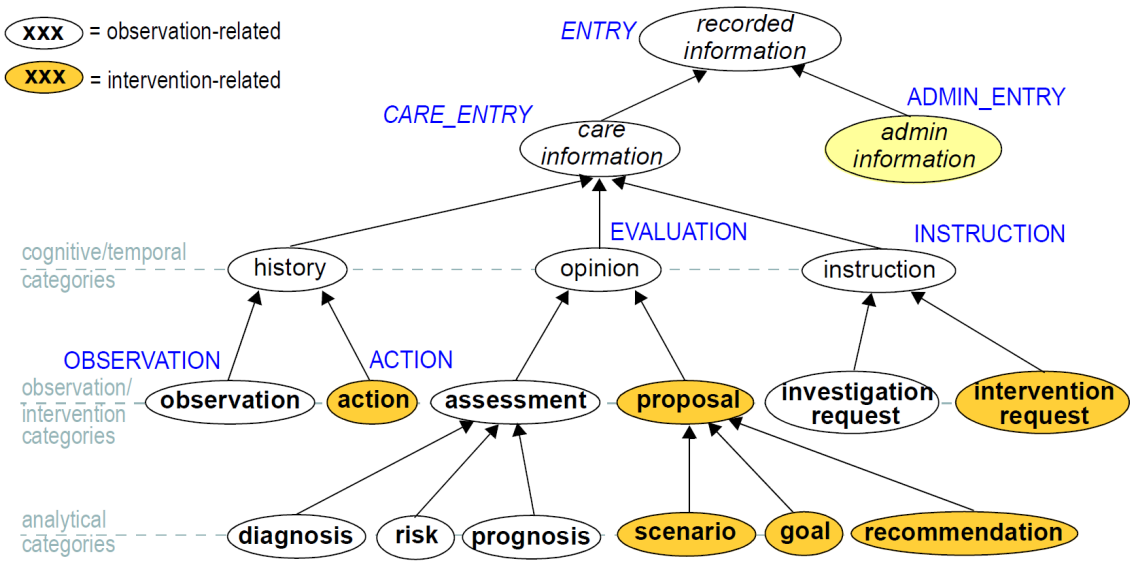


Figure 19. The Clinical Investigator Recording (CIR) ontology

The key top-level categories in the ontology are 'care information' and 'administrative information'. The former encompasses all statements that might be recorded at any point during the care process, and consists of the major subcategories 'history', 'opinion' and 'instruction', which themselves correspond to the past, present and future in time (ISO TC215 uses the terms 'retrospective', 'current' and 'prospective'). The administrative information category covers information which is not generated by the care process proper, but relates to organising it, such as appointments and admissions. This information is not about care, but about the logistics of care being delivered. Categories that relate to the patient system as observed and understood are shown as white bubbles, while categories that relate to intervention into the patient system are shown as shaded. The opinion category has features of both passive analysis and active intervention.

There are two key justifications for using the ontology in Figure 19 as a basis for class design. Firstly, although for all categories in the ontology there is a meaning for 'contextual' attributes of time, place, identity, reason and so on, each category has a different structure for these attributes. For example, time in the Observation category has a linear historical structure, whereas in Instruction it has a branching, potentially cyclic structure. The

separation of types allows these contextual attributes to be modelled according to the type. Secondly, the separation of types provides a systematic solution to the so-called problem of "status" or "meaning modification" of clinical statement values, as described below.

8.1.2. Clinical Statement Status and Negation

A well-known problem in clinical information recording is the problem of assigning "status", including variants like "actual value of P" (P stands for some phenomenon), "family history of P", "risk of P", "fear of P", as well as negation of any of these, i.e. "not/no P", "no history of P" etc. A proper analysis of these so called statuses [4] shows that they are not "statuses" at all, but different categories of information as per the ontology. The common statement types mentioned here are mapped as follows:

- actual value of P → Observation (of P);
- no/not P → Observation (of excluded P or types of P, e.g. allergies).
- family history of P → Evaluation (that patient is at risk of P);
- no family history of P → Evaluation (that P is an excluded risk);
- risk of P → Evaluation (that patient is at risk of P);
- no risk of P → Evaluation (that patient is not at risk of P);
- fear of P → Observation (of FEAR, with P mentioned in the description);

In general, negations of the kind mentioned above are handled by using "exclusion" archetypes for the appropriate Entry type. For example, "no allergies" can be modelled using an Evaluation archetype that describes which allergies are excluded for this patient.

Another set of statement types that can be confused in systems that do not properly model information categories concern interventions, e.g. "hip replacement (5 years ago)", "hip replacement (planned)", "hip replacement (ordered for next tuesday 10 am)". Ambiguity is removed here as well, with the use of the correct information categories, e.g. (I stands for an intervention):

- I (distant past/unmanaged/passively documented) → Observation (of I present in patient);
- I (recent past) → Action (of I having been done to/for patient);
- I (proposed) → Evaluation, subtype Proposal (of I suggested/likely for patient);
- I (ordered) → Instruction (of I for patient for some date in the future).

Related to the problem of clinical status is the need to differentiate between diagnoses that have been made in the present from those that were made in the past, and are reported by the patient. In openEHR, diagnosis and indeed all opinion category types are modelled as Evaluations, regardless of when they were made. Time information for diagnosis and other opinions is simply handled within the archetype for Evaluation, enabling a diagnosis (say of diabetes) that was made 15 years ago to have the same status as one made during the period when the EHR was actually operating, and the patient was under the care of the physician using it. Archetypes for the opinion categories tend to have quite a number of times, including "time first noticed", "time of onset", and so on.

Correct use of the categories from the ontology is facilitated by using archetypes designed to map particular kinds of clinical statement to particular `ENTRY` subtypes. In a system where Entries are thus modelled, there will

be no danger of incorrectly identifying the various kinds of Entries, as long as the Entry subtype, time, and certainty/negation are taken into account.

8.1.3. Standard Clinical Types of Data

Commonly used types of clinical information are directly representable using the openEHR Entry types. As described in Section 4.2.4, two kinds of Composition are identified: persistent and event. The persistent kind corresponds to information that characterises the patient in the long term, and is maintained by clinicians - it can be thought of as a proxy "model of the patient". Most of the following are examples of Entry level data belonging inside persistent Compositions:

- *Basic information* (dob, sex, height, weight, pregnant etc): recorded as Observations and/or Admin_entries;
- *Problem list*: maintained as one or more Evaluations which themselves are generated by clinicians as a result of observations made elsewhere in the record;
- *Medications list*: derived from Instructions and Actions in the record. The status of Actions allows medications to be displayed as past, current, suspended etc.
- *Therapeutic precautions (allergies and alerts)*: recorded as Evaluations of various kinds (adverse reactions are essentially a kind of diagnosis);
- *Patient preferences*: recorded as Evaluations (since they act like contra-indications for certain drugs or treatments);
- *Patient consents*: recorded as instances of Admin_entry;
- *Family history*:
  - actual events / conditions in family members are recorded as Observations (e.g. father died of MI at 62)
  - patient risks are expressed using Evaluations, often including family history reasons.
- *Social history/situation*: current and previous social situation (e.g. in nursing care, details of feeding, sleeping arrangments) are documented as Observations.
- *Lifestyle*: there are various Observation archetypes for recording aspects of lifestyle, including exercise, smoking/tobacco, alcohol, drug use and so on.
- *Vaccination record*: vaccinations are a kind of Instruction; vaccinations that have actually been given are available as Actions in the record.
- *Care plan*: combinations of goals, targets (Evaluations), monitoring, education (Instructions), etc organised in a care plan Section hierarchy.

The remaining vast majority of remaining clinical information is recorded inside event Compositions, and includes the following:

- *laboratory results* including imaging: recorded as Observations;
- *physical examinations*: recorded as Observations appointment, admission and discharge: recorded as Admin_entry
- *prescriptions*: one or more medication orders (each one is an Instruction) within a prescription Section structure, within a prescription Composition.
- *referrals*: recorded as Instructions (i.e. a request for care by another provider).

The above concepts are defined in terms of archetypes of Entry and other reference model types in openEHR; their definition is therefore completely separate from the reference model.

8.1.4. Demographic Data in the EHR

The general approach of openEHR is to enable the complete separation of demographic (particularly patient-identification information) from health records, both in the interests of privacy (in some cases required by national legislation) and separated data management. The Demographic IM therefore defines demographic information. However, there is nothing to prevent certain demographic information occurring in the EHR, and in some cases this is desirable. The two main cases for this are:

- clinically-relevant patient information, such as age, sex, height, weight, eye-colour, ethnicity or 'race', occupation;
- identifiers and/or names of health care provider individuals and organisations may be stored directly in the EHR, regardless of whether there is also more detailed information about such entities in the demographic system.

The model of all information intended for a separate openEHR demographic service (itself usually a front-end to an existing hospital master patient index or similar) is defined in the openEHR Demographic Information Model.

8.2. Entry and its Subtypes

The Entry model is defined by the `composition.content.entry package`, shown in the UML diagram below. The choice of subtypes of `ENTRY` is based on the ontology shown in Figure 19 and its associated model. The names do not coincide exactly however, for a number of reasons. Firstly, the category names in the hierarchy were chosen based on a philosophical/scientific model of investigation, and reflecting linguistic norms for the meanings of the terms, whereas the class names used here reflect common health computing and clinical usage of these terms (i.e. names which will make sense to software developers in the health arena). Names in these two cultures do not always coincide. Additionally, for categories such as Opinion and Instruction, the subcategories shown in the ontology (e.g. Assessment, Goal and Plan) are too variable to safely be subtyped in software, and are distinguished only at the archetype level. Only a single class for each of these ontological groups is used in the formal model. The use of different names and a slightly simplified mapping has not however prevented the faithful implementation of the semantics of the model. The model classes are described in the following subsections.

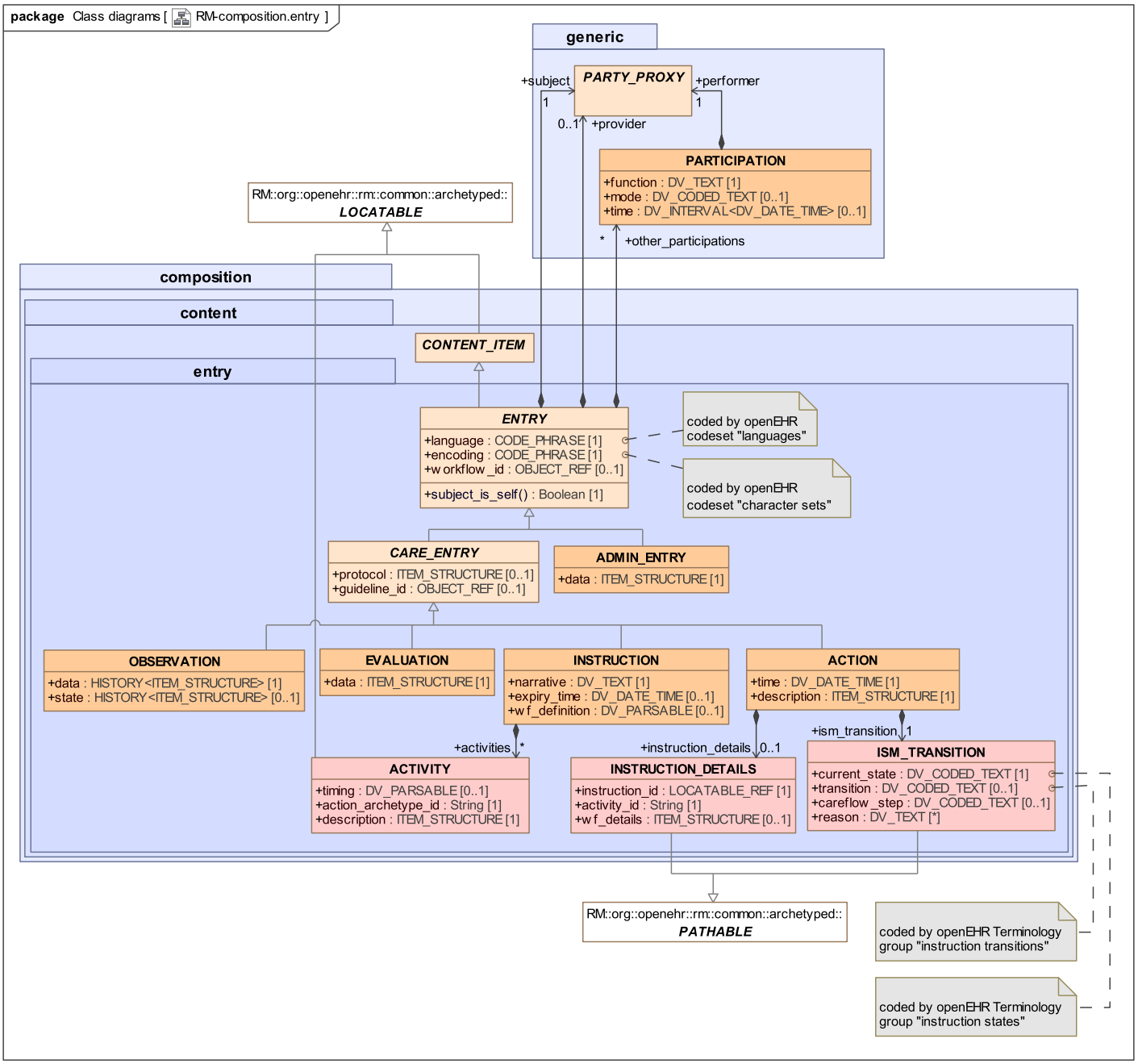


Figure 20. rm.entry package

8.2.1. The Entry class

All Entries have a number of attributes in common. The language and encoding attributes indicate how all text data within the Entry are to be interpreted linguistically and at the character set level. Normally, the language will be the same throughout the entire Entry (if not Composition), but in cases where it is not, the optional language attribute of `DV_TEXT` can be used to override the value in the enclosing `ENTRY` (or other enclosing structure, if a `DV_TEXT` is being used in some other context). Character encoding can be overridden in the same fashion by the encoding attribute within `DV_TEXT`.

The other attributes common to all Entry subtypes are as follows:

- *`subject`* : this attribute records the subject of the Entry as an instance of a subtype of `PARTY_PROXY`. When this is the record subject, (i.e. the patient), the value is an instance of `PARTY_SELF`. Otherwise it is

typically a family member, sexual partner, or other acquaintance of the record subject. It could also be an organ donor. The latter is expressed in the form of a `PARTY_IDENTIFIED` or `RELATED_PARTY` instance, which describes the kind of relationship, and optionally, identifies the demographic entity.

- *`subject_is_self`* : convenience function returning True when the Entry is about the subject of the record.
- *`provider`* : the agent who provided the information. This is usually the patient or the clinician, but may be someone else, or a software application or device. If participation details of the provider (e.g. mode of communication) need to be recorded, the details should be recorded once in the `EVENT_CONTEXT`.*`participations`*. The provider attribute is optional, since it is often implicit in the information that was recorded.
- *`other_participations`* : other participations which existed for this Entry, e.g. a nurse who administered a drug in an `INSTRUCTION`; only required in cases where participants other than the subject of the information and the provider of the information need to be recorded.

Note that the term 'provider' as used here should not be confused with the more specific healthcare term used in many english-speaking countries meaning 'health care provider', which is usually understood to be a physician or healthcare delivery enterprise such as a hospital. In the model here, it simply means 'provider of information' in the context of an Entry. The information provider is optional, and in many cases will not be recorded, since it will be obvious from the *`composer`* and *`other_participations`* of the enclosing Composition. In many cases, it is not sensible to record a provider, e.g. in the most mundane case where a GP asks "where does it hurt" and the patient says "here" - in such cases, it can only be considered mutual. It is expected to be used only when the composer of the Composition really needs to specify the origin of specific statements, such as in the following circumstances:

- the information provider is specifically accountable for the Entry data (it is their opinion, their decision, they carried out the test etc) - they might also need to attest it;
- the information provider is an authoritative source, or has provided information from a unique perspective (e.g. the view of a spouse/ carer on the patient's functional health status or mental state);
- the information provider's view might not reflect the consensus (e.g. a patient opinion not held by the composer, a difference between father and mother on a description of a child's sleeping pattern);
- the information provider is not one of the Composition-level participants (e.g. an outside information provider such as someone telephoned during the encounter to provide a lab result, or an automated measurement device, or a decision support software component).

8.2.2. Care_entry and Admin_entry

A basic division occurs between clinical and non-clinical information. The `CARE_ENTRY` class is an abstract precursor of classes that express information of any clinical activity in the care process around the patient, while `ADMIN_ENTRY` is used to capture administrative information. The division may occasionally seem ambiguous at a theoretical level, but at a practical level, it is almost always clear. Administrative information has the following characteristics:

- it is created by non-clinical staff, or clinical staff acting in an administrative capacity (e.g. a nurse or doctor who has to fill out an admission form in A&E);

- it expresses details to do with coordinating the clinical process, by recording e.g. admission information (enables clinical staff to know who is in the hospital and where they can be found), appointments (ensuring patient and physician get together at an agreed time and place), discharge/dismissal (allowing clinical staff to know that a patient has been sent home healthy, or transferred to another institution), billing and insurance information (where such information is required in the EHR; it may well be in its own system);
- removing administative information from the EHR would not compromise its clinical integrity, it would simply mean that carers and patients would no longer know when and where they were supposed to meet, or when the patient has entered or left a health care facility.

Conversely, every instance of a `CARE_ENTRY` subtype is clinically significant, even if it also carries information which might be of interest to other health management functions billing (e.g. ICD10 (http://www.who.int/classifications/icd/en/) coded diagnoses), practice management (e.g. date, time and place in an order for day surgery). The `CARE_ENTRY` type includes two attributes particular to all clinical entries, namely *`protocol`* and *`guideline_id`*.

These attributes allow the 'how' and 'why' aspects of any clinical recording to be expressed. Protocol is often recorded for Observations, e.g. staining method in microscopy, and Instructions, e.g. Bruce protocol for ECG test.

8.2.3. Observation

Instances of the `OBSERVATION` class record the observation of any phenomenon or state of interest to do with the patient, including pathology results, blood pressure readings, the family history and social circumstances as told by the patient to the doctor, patient answers to physician questions during a physical examination, and responses to a psychological assessment questionnaire. Observations are distinguished from Actions in that Actions are interventions whereas Observations record only information relating to the situation of the patient, not what is done to him/her.

The significant information of an Observation is expressed in terms of 'data', 'state' and 'protocol'. The first of these is recorded in the data attribute, defined as follows:

- *`data`* : the actual datum being recorded; expressed in the form of a History of Events, each of which can be a complex data structure such as a List, Table, Single (value), or Tree, in its own right. Examples include blood pressure, heartrate, ECG traces.

State information can be recorded in the state attribute of Observation, or within the state attribute of each Event in the Observation data attribute (see below and also Data Structures IM for more detailed explanations). The state attribute in Observation is defined as follows:

- *`state`* : any particular information about the state of the subject of the Entry necessary to correctly interpret the data, which is not already known in the health record (i.e. facts such as the patient being female, pregnant, or currently undergoing chemotherapy). For example, exersion level (resting, post-marathon…),

position (lying, standing), post- glucose challenge, and so on. The form of the *state* attribute is the same as that of the *data* attribute: a HISTORY of EVENT of ITEM_STRUCTURE .

The inherited protocol attribute is defined as follows:

- *protocol* : details of how the observation was carried out, which might include a particular clinical protocol (e.g. Bruce protocol for treadmill exercise ECG) and/or information about instruments other observational methods. This information can always be safely omitted from the user interface, i.e. has no bearing on the interpretation of the data.

The detailed semantics of Observation data are described in the following subsections.

8.2.3.1. Timing in Observations

Many health information models express observation time as one or more attributes with names like 'observation_time', 'activity_time' and so on. The openEHR model departs from this by modelling historical time inside a History/Event structure defined in the data_structures.history package. In short, this package defines the HISTORY class with an origin attribute, and a series of EVENT instances each containing a time attribute. Instantaneous and interval events are distinguished via the EVENT subtypes POINT_EVENT and INTERVAL_EVENT ; interval events have the width attribute is set to the duration of the interval.

8.2.3.2. Valid Contents of a History

One of the aims of the model of Observation described here is to represent in the same way single sample and multi-sample time-based data for which measurement protocol is invariant. It is not intended for measurements in "coarse" time taken by different people, different instruments, or with any other difference in data-gathering technique. In these cases, separate, usually single-sample histories are used, usually occurring in distinct container objects (e.g. distinct Compositions, in the EHR).

Accordingly, in the general practice setting, the use of HISTORY will correspond to measurement series which occur during the clinical session (i.e. during a patient contact). In a hospital setting, nurses' observations might occur in 4-hourly intervals, and there is no well-defined clinical session - simply a series of ENTRYs during the time of the episode. Two approaches are possible here.

- If each Observation is to be committed to the EHR as soon as it is made, the result will be distinct COMPOSITIONs in time, each with its event_context corresponding to the period of the nurse's presence. Each Composition will contain one or more Observations, each containing in their data a History of one sample of the measured vital sign.
- If Observations are not committed to the EHR immediately, but are stored elsewhere and only committed (say) at the end of each day, then the result will be a single Composition whose event_context corresponds to the total data gathering period, and which contains Observations whose data are multi-event Histories representing the multiple measurements made over the day.

Whether time-based data remain outside the record until a series of desired length is gathered, or entered as it

occurs is up to the design of applications and systems; the approach taken should be based on the desired availability of the data in the system in question. If for example, it must be visible in the EHR as soon as the appropriate Compositions are written, then it should be represented as Histories in each relevant Composition; if it need only be available at some much later point in time (e.g. because it is known that no-one but the treating clinician is interested in it), then it can be stored in another system until sufficient items have been gathered for committal to the EHR.

8.2.3.3. Clinical Semantics of Event Time

In most cases, the times recorded in a History (`HISTORY.origin` and `EVENT.time`, `width`) can be thought of as "the times when the observed phenomena were true". For example, if a pulse of 88bpm is recorded for 12/feb/2005 12:44:00, this is the time at which the heartrate (for which pulse is a surrogate) existed. In such cases, the sample time, and the measuring time are one and the same.

However in cases where the time of sampling is different from that of measurement, the semantics are more subtle. There are two cases. The first is where a sample is taken (e.g. a tissue sample in a needle biopsy), and is tested later on, but from the point of view of the test, the time delay makes no difference. This might be because the sample was immediately preserved (e.g. freezing, placed in a sterile anaerobic transport container), or because even if it decays in some way, it makes no difference to the test (e.g. bacteria may die, but this makes no difference to a PCT analysis, as long as the biological matter is not physically destroyed).

The second situation is when the sample does decay in some way, and the delay is relevant. Most such cases are in pathology tests, where presence of live biological organisms (e.g. anaerobic bacteria) is being measured. The sample time (or 'collection' time) must be recorded. Depending on when the test is done, the results may be interpreted differently.

The key question is: what is the meaning of the `HISTORY.origin` and `EVENT.time` attributes in these situations? It is tempting to say that their values are (as in other cases) just the times of the actual act of observation, e.g. microscopy, chromatography etc. However, there are two problems with this. Firstly, and most importantly, all physical samples must be understood as being *indirect surrogates for some aspect of the patient state* at the time of sampling, which cannot be observed by direct, instantaneous means in the way a pulse can be taken. This means that no matter when the laboratory work is done, the time to which the result applies is the sample time. It is up to the laboratory to take into account time delays and effects of decay of samples in order to provide a test result which correctly indicates the state of the patient at the time of sampling. The common sense of this is clear when one considers the extreme case where the patient is in a coma or dead (possibly for reasons completely unrelated to the problem being tested for) by the time laboratory testing actually occurs; however, the test result indicates the situation at the point in time when the sample was taken, i.e. when the patient was alive. The second reason is that some kinds of testing are themselves lengthy. For example fungal specimens require 4-6 weeks to confirm a negative result; checks will be made on a daily or weekly basis to find positive growth. However, the result data reported by the laboratory (and therefore the structure of the Observation) is not related to the timing of the laboratory testing; it is reported as being the result for the time of collection of the specimen

from the patient.

The meaning therefore of the `HISTORY.origin` and `EVENT.time` attributes in openEHR is always the time of sampling. Where delays between sample and measurement times exist and are significant, they are noted in the protocol section of the Observation; such times are modelled in the appropriate archetypes, and taken into account in results.

8.2.3.4. Two ways of Recording State

State information is optional, and is not needed if the data are meaningful on their own. If it is recorded, it can either be as a History of its own (i.e. using the `OBSERVATION.state` atttribute described above), or else as state values within the `EVENT` instances in the `OBSERVATION.data` History. Both methods are useful in different circumstances. A separate state history is more likely to be used in a correlation study such as a sports medicine study on heartrate with respect to specific types of exercise. In this method, the state information is a History of Events whose times and widths need not match those of the History in the data attribute. The state data under this approach generally express the condition of the subject in absolute terms, i.e. they are standalone statements about the subject's state at certain points in time, such as "walking on treadmill 10km/h, 10° incline".

The other method will be used in most general medicine, e.g. for recording fasting and post glucose challenge states of a patient undergoing a glucose tolerance test. (See the Data Structures Information Model for more details). State values stored within the `data` History represent the situation in the subject at the time of the Event within the History and usually in relation to it, for example "post 8 hour fast". Recording the latter example in an independent state History would require an Event of 8 hours' duration called 'fast'. The latter would be technically still correct, but would be very unnatural to most clinicians. The figure below illustrates the two methods of recording state.

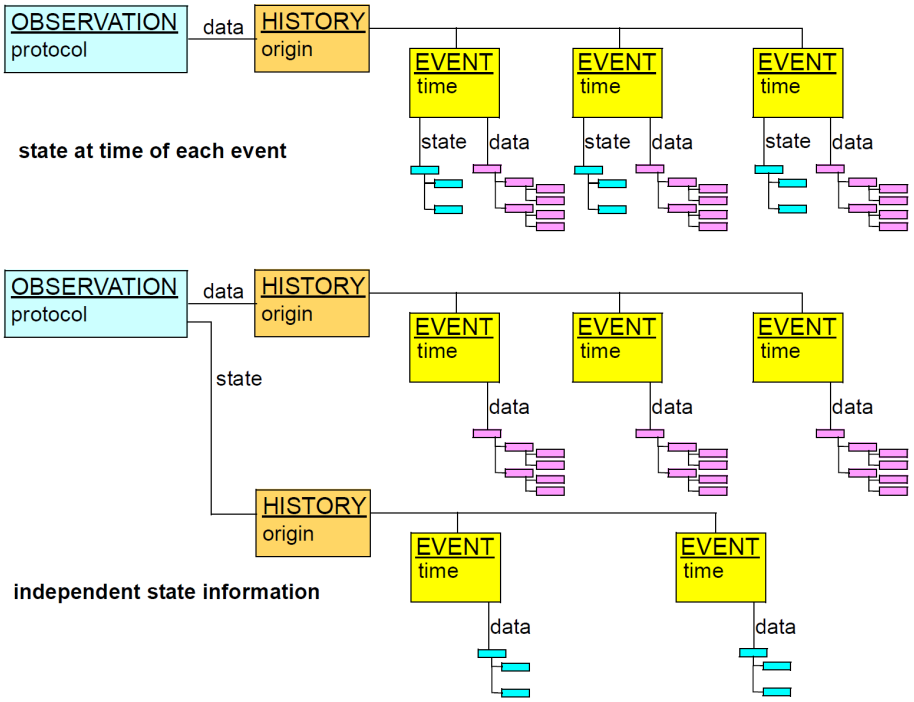


Figure 21. Alternative ways of recording state

8.2.4. Evaluation

According to the ontology described in Beale & Heard (2007), the Opinion category covers a number of concrete concepts, as follows.

- *problem/diagnosis*: the assignment of a known diagnosis or problem label to a set of observed signs and symptoms in the patient, for the purpose of determining and managing treatment. The physician will usually include a date of initial onset, date clinically recognised, date of last occurrence, date of resolution of last occurrence and possibly other timing information.
- *risk assessment*: an evaluation of the likelihood and timing of a certain event occurring or condition appearing.
- *scenario*: an opinion about the outcome if a certain intervention is proceeded with.
- *goal*: statement of a target, and a time at which it should be reached.
- *recommendation*: a description in general terms of a suggested care approach for the patient, based on diagnosis; includes various times or time-periods for activities, such as monitoring, taking of medications, and review.

The approach taken to modelling these concepts in openEHR is heavily based upon the development of archetypes for assessments such as "diagnosis" (various kinds), "goal", "adverse reactions", "alert", "exclusion", "clinical synopsis", "risk based on family history" and so on. Experience has shown that the Opinion category is too variable for safely modelling its sub-categories directly in the reference model. Instead, a single class `EVALUATION` is used for all instances of the Opinion category. (The name Evaluation has been present in openEHR for some years, and is retained for reasons of continuity).

The design of the `EVALUATION` class is very simple. In addition to the attributes inherited from `ENTRY` and `CARE_ENTRY`, it has only one attribute, *`data`* : `ITEM_STRUCTURE`. This structure is intended to be archetyped so as to model all the details of any particular clinical information in the Opinion category. No timing attributes are included, since there is no time associated with creation or capturing of Evaluation information as such, only times included in the information. The only times of generic significance are (potentially) the time of a patient consultation during which the Evaluation was created (recorded in `COMPOSITION.`*`event_context.start_time`* and *`end_time`*) and the time of committal to the EHR system (recorded in `VERSION.`*`commit_audit`* attribute).

The general meaning of the inherited attributes is as for all Entries. In Evaluations, the provider is almost always the physician, while the protocol may be used to indicate how a particular assessment was made. The *`other_participations`* attribute is not as likely to be used for Evaluations representing diagnoses, since a diagnosis is usually the result of thinking on the part of the physician; an exception to this would be a case conference or if an expert system were used. However, Plans for complex patients may well be constructed by multiple physicians.

8.2.5. Instruction and Action

Instructions in openEHR specify actions to be performed in the future. They differ from information in the

Proposal sub-category of Opinion in the ontology (i.e. instances of Evaluation in the class model) in that they are specified in sufficient detail to be directly enacted without further clinical decision-making related to the design of the Instruction, e.g they can be performed by the patient or a nurse. Any decisions that could be made during the performance of an Instruction are either constrained by the Instruction itself (e.g. dose range; suspend if adverse reaction) or else are assumed knowledge of the expected performer. For example, an Evaluation may say that "oral cortico-steroids are indicated at a peak flow of 200 l/m". A corresponding Instruction would indicate the actual drug, route, dose, frequency, and so on. The informed patient might be reasonably expected to be able to vary the dosage on his or her own within a dosage guideline explained by his/her GP.

In the ontology shown in Figure 19, Instructions are further categorised as Investigation and Intervention. However, as for Evaluation, only a single key class, `INSTRUCTION`, is used to model all types of the Instruction category, with archetypes defining the details of the Instruction. A second Entry subtype, `ACTION`, is used to model the information recorded due to the execution of an Instruction by some agent.

The following subsections describe Instruction and Action in some detail.

8.2.5.1. Requirements

The Instruction and Action classes are designed to satisfy the following requirements:

- All kinds of interventions, from simple medication orders to complex multi-drug courses should be representable using the same model;
- Instructions should always have a narrative expression, with an optional machine-processible expression in cases where automation will be used;
- The freedom must exist to model any particular intervention in as much or little detail as required by circumstances;
- Clinicians must be able to specify Instruction steps in their own terms, i.e. using terms like "prescribe", "dispense", "start administering", etc;
- Instructions representing diverse clinical workflows must be queryable in a standard way, so that it can be ascertained what Instructions are 'active', 'completed' and so on for a patient;
- It should be possible to provide a coherent view of the state of execution of an Instruction even if parts of it have been executed in different healthcare provider environments;
- It must be possible to record ad hoc actions in the record, i.e. acts for which no Instruction was defined (at least in the EHR in question);
- Instructions must integrate with notification / alert services;
- An interoperable expression of computable workflow definitions of Instructions will be supported.

8.2.5.2. Design Principles

The design approach is based on four principles. The first is that the *specification* of an Instruction as one or more Activities is distinguished from the information representing actions performed as a result. This makes the model and resulting information instances clear to software designers and data users. It also enables workflow engines to determine which parts of the specification have already been executed, and allows for Actions actually performed to differ from those specified. The separation is realised in terms of the `INSTRUCTION` and `ACTION`

classes and their helpers. Instances of the former specify an Instruction, while instances of the latter describe steps which have actually been performed.

The second principle is the use of a standard instruction state machine (ISM) defining standardised states and transitions for each Activity of an Instruction, no matter what its clinical meaning. The use of standardised states means that the execution state of any given Activity can be characterised in exactly the same way (e.g. 'planned', 'active', etc), and that it is therefore possible to query the EHR and find out all interventions of any kind in a particular state. The ISM is formally modelled in openEHR. The Instruction itself can also be considered to be in a state derived from the states of its Activities. An informal description of this aggregate state machine is given below.

The third principle is to provide a way of mapping steps in any care pathway process (i.e. healthcare business process) to states in the Instruction state machine. A care pathway process covers the entirety of steps required to effect an Instruction, for example prescribing, booking, dispensing, referring, suspension etc. Any such step when performed leaves the relevant Instruction in one of the states of the ISM.

The fourth principle is to support the expression of the formal workflow definition for an Instruction, where full automation is required. It must be recognised that automation of most therapies and drug admnistration, as well as other interventions like biopsies is minimal today, and is likely to remain so for some time. This is for the simple reason that the cost of automating most tasks is prohibitive compared to human execution, particularly when Instruction activities can often be executed by healthcare professionals already present for other reasons (e.g. ward nurses). It also has to be said that serious research into the use of workflow automation in healthcare is only quite recent, and that so far, there are no standard models for clinical workflow. In the openEHR approach to modelling workflow, such uncertainty is dealt with in two ways. Firstly, formal workflow specification of an Instruction is an optional addition to the base model of Instruction and Action classes, and is not required to obtain a basic level of computability, including use of the ISM. Secondly, the formal expression of workflow is in the form of parsable syntax rather than objects. This is a generally appropriate design choice, since the safest and most convenient persistent form of of a complex formalism is the syntax form rather than the parsed fine-grained object form; this both optimises storage and allows for changes in the syntax over time.

8.2.5.3. Model Overview

Instruction definitions are modelled in terms of the `INSTRUCTION` and `ACTIVITY` classes, with optional workflow attributes. These two classes carry the basic information relating to an Instruction, with all formal workflow definition expressed in parsable syntax in the `INSTRUCTION.`*`wf_definition`* attribute. An `INSTRUCTION` instance includes the narrative description of the Instruction, and a list of `ACTIVITY` instances. It also includes all the attributes inherited from the `CARE_ENTRY` class, including subject, participations and so on.

Many Instructions will have only one Activity, usually describing a medication to be administered and its timing. Some will have more than one drug or therapy, such as the typical 3 drug Losec-HP regime for treating ulcers, and multi-drug chemotherapy. The base Instruction model does not explicitly try to indicate the exact order,

serial or parallel administration, or other dependencies, since the knowledge of how to administer the drugs is known by the relevant clinicians, and/or contained in published guidelines. However, the timing information in each Activity does indicate times, days and the usual specifications of "with meals" etc. The timing information is also sufficient to specify a three drug chemotherapy regime, by indicating which days each drug is administered on. It is only when the Instruction is to be automated by a workflow engine that the full structure of the Activity graph will be given. Activity instances may be completely absent from an Instruction, in which case only the narrative will be present. This will typically occur with imported legacy data which itself has no structured representation of medications.

There are three possible levels of representation of an Instruction, as shown in the figure below. These are the minimal narrative-only level, a level with formal representation of Instruction activities by `ACTIVITY` instances, and a level where a formal workflow definition can also be used. It is expected that the vast majority of openEHR systems for the forseeable future will support only the minimal and basic levels.

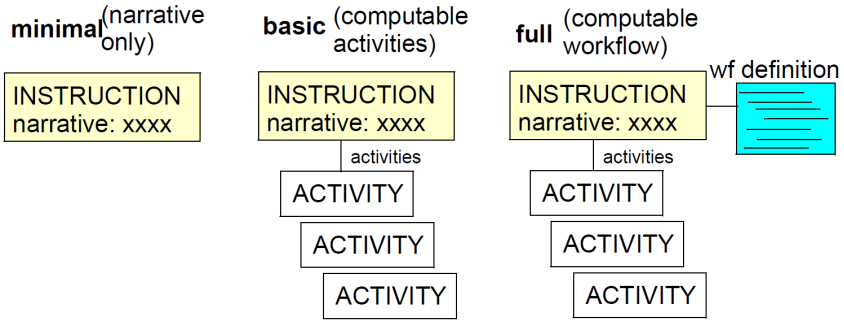


Figure 22. Levels of Instruction representation

The following figure illustrates the correspondence between Instruction and Activity structures, and Action objects that are created due to executing the instructions. Each Activity has a standard Instruction State Machine logically associated with it, indicated in blue. When any step in an Instruction Activity is executed within an ICT-supported environment, an `ACTION` instance describing what was done should be created and committed to the EHR. In some cases, ad hoc Actions are created even when there is no Instruction, such as by self-medicating patients and nurses reacting quickly to patient changes. For such Actions, there is always a notional Activity understood by the health professional.

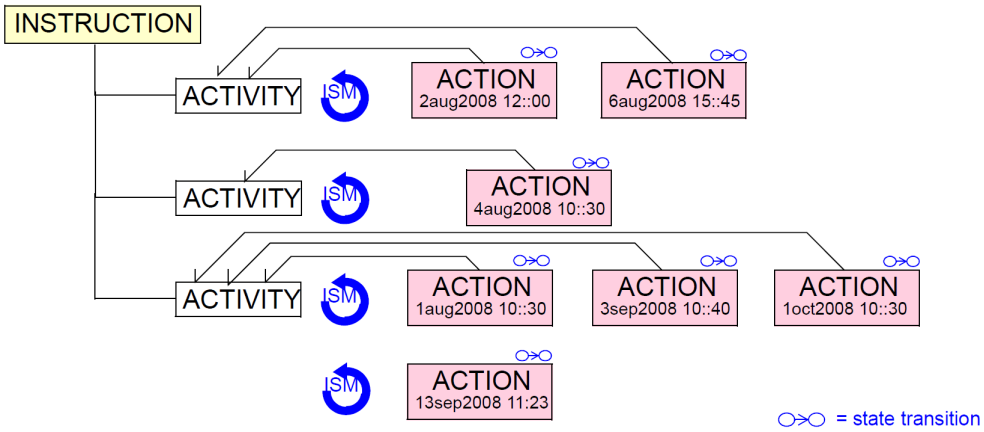


Figure 23. Correspondence of Actions to Instructions

All `ACTIONs` include the inherited `CARE_ENTRY` attributes, along with the time of being performed, a description of what was performed, and an `ISM_TRANSITION` object indicating the state of the Activity (whether explicit in an Instruction or not). The current state of an Activity is thus found not in the `ACTIVITY` instance but in the most recent `ACTION` instance for that Activity.

If the Action did correspond to an Instruction, it will also include an `INSTRUCTION_DETAILS` instance, which indicates which Activity of which Instruction was executed, including workflow execution details if relevant.

Under this scheme, the state of each intervention happening to the patient can be ascertained by querying, regardless of whether explicit Instructions exist.

Note particularly that Actions are often performed in a different provider location, or the home, rather than the provider organisation responsible for the Instruction. Action objects for a given Instruction can thus easily appear in multiple health record systems.

8.2.5.4. Instruction Activity Timing

Timing information may be minimal and simplistic, or highly specified. As a consequence, there is no one method of representation that suits all purposes. The simplest expression of timing information is as informal narrative, as found in a prescription for antibiotics, such as "take 3 times a day, for 7 days, with meals". This may be represented using a text value in `INSTRUCTION.`*`narrative`* .

Formally computable representations can be expressed in two ways. The first is to use the `ACTIVITY.`*`timing`* field, of type `DV_PARSABLE` , which enables the use of syntaxes such as the ISO 8601 (https://en.wikipedia.org/wiki/ISO_8601) 'R' syntax. For example, 5 repetitions of the period '10 days' starting on 01 march 2008 is `"R5/2008-03-01T13:00:00Z/P10D"` . However timing in clinical situations varies widely, and numerous other ways of expressing times are commonly required, ranging from simple frequencies, to complex patterns of time, association with events such as meals, menstrual period and so on. To accommodate such variant structures, which are generally beyond the capabilities of the ISO 8601 syntax, the `ACTIVITY.`*`description`* attribute may be used to contain these timing structures.

The three approaches are illustrated below.

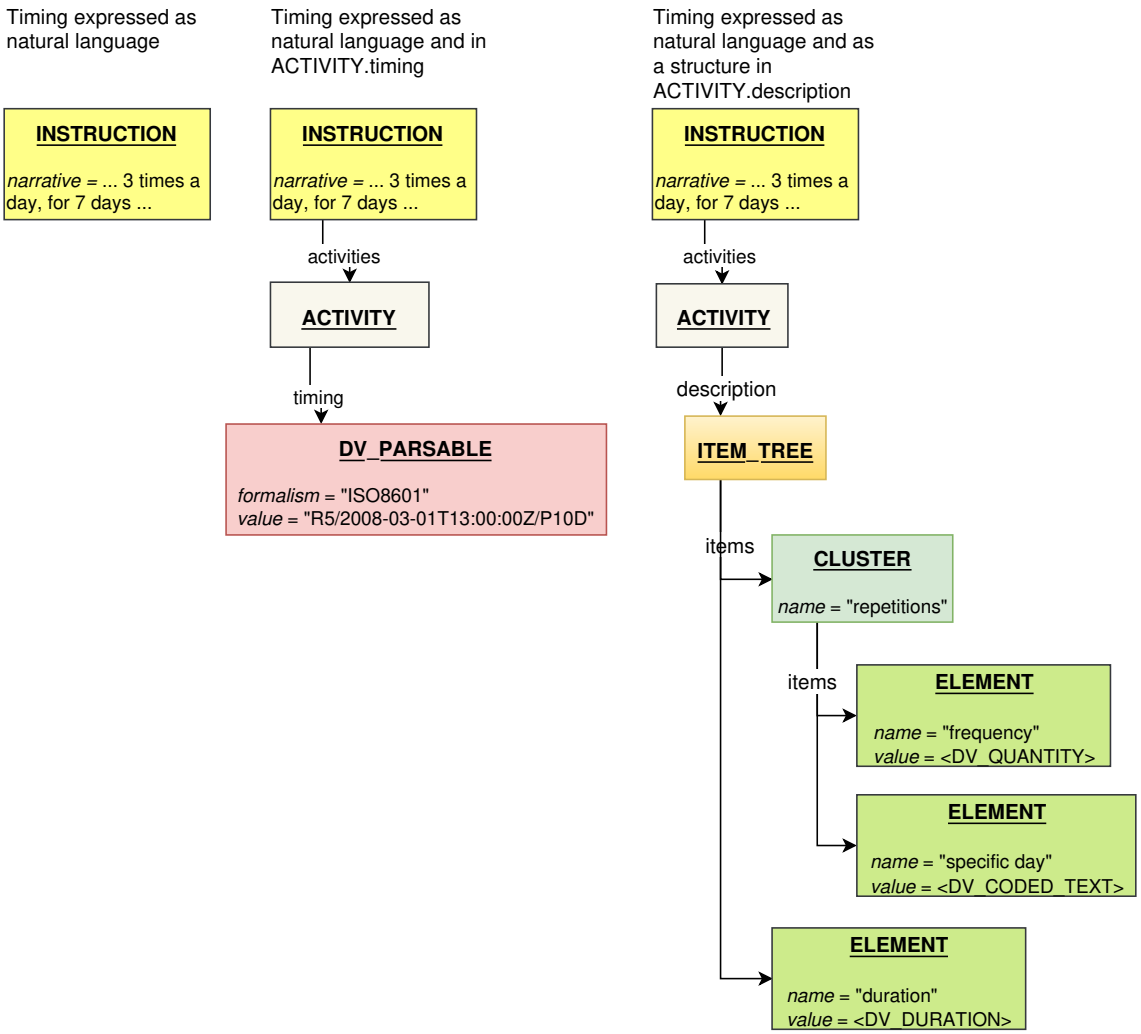


Figure 24. Activity Timing

8.2.5.5. The Standard Instruction State Machine (ISM)

When an intervention defined by an Activity is unfolding in a clinical context, EHR users want to know things such as the following:

- what is the current status of an Instruction for the patient?
- what is the current step in the Activity for the patient?
- for a given patient, what Activities are planned, active, suspended, completed?
- for the populations of patients, what is the state of a particular workflow, such as a recall, for each one?

The approach chosen here to support such functionality is to define a standard instruction state machine whose state transitions can be mapped to the steps of a specific care pathway, enabling it to be used as a descriptive device for indicating the state of any Activity. The status of an Instruction is determined algorithmically from the states of the constituent Activities, as described below. The state machine is illustrated below.

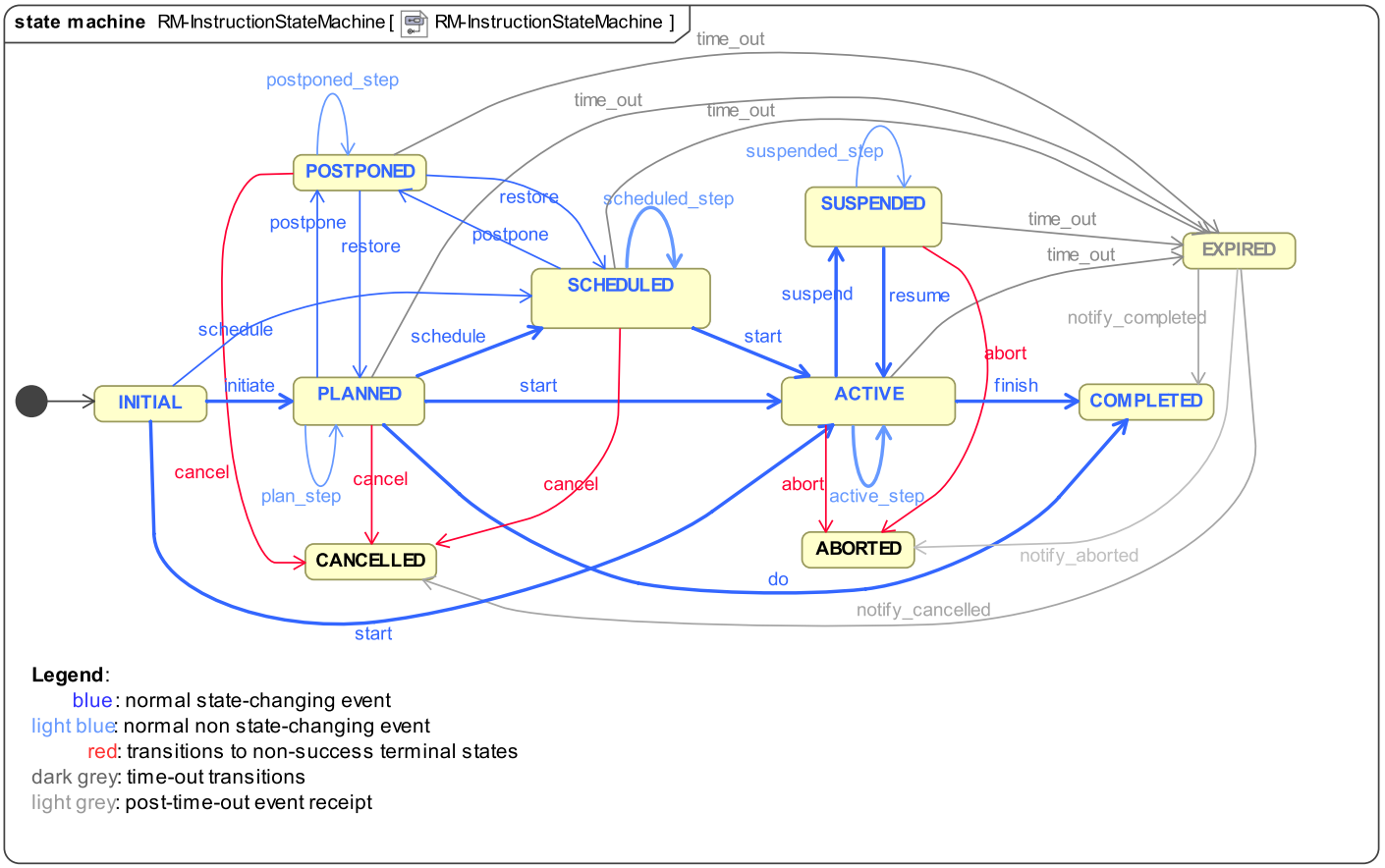


Figure 25. openEHR standard Instruction State Machine

This state machine is the result of long term experience with clinical workflows and act management systems. The states are as follows (Note: the SCHEDULED state was inspired by van de Velde & Degoulet (2003), Fig 5.5).

INITIAL	initial state, prior to planning activity (default starting state for computable representation of state machine).
PLANNED	the action has been described, but has not as yet been performed.
POSTPONED	the action has not been performed and will not without specific conditions being met. Specifically, events and conditions that would normally 'activate' the instruction will be ignored, until a restore event occurs.
SCHEDULED	the action will be performed at some designated future time, and has been booked in a scheduling system.
CANCELLED	the action was defined, but was cancelled before being performed.
ACTIVE	the action is being performed according to its definition. The entire course of medication or therapy corresponds to this state.

SUSPENDED	the action was begun, but has been stopped temporarily, and will not be restarted until explicitly resumed.
ABORTED	the action began but was permanently terminated before normal completion.
COMPLETED	the action began and was completed normally.
EXPIRED	the time during which the action could have been relevant has expired; the action may have completed, been cancelled, or never occurred.

States `CANCELLED`, `ABORTED` and `COMPLETED` are all terminal states. The `EXPIRED` state is a pseudo-terminal state, from which transitions are allowed to proceed to any of the true terminal states, due to information being received after the fact (such as a patient reporting that they did indeed finish a course of antibiotics). However it is likely in the EHR that Instructions for many simple medications will finish in the `EXPIRED` state and remain there.

The transitions are self-explanatory for the most part, however a few deserve comment. The `start` and `finish` events correspond to situations when the administration is not instantaneous, as is the situation with most medications. The `do` event is equivalent to the `finish` event occurring immediately after the `start` event, corresponding to an instantaneous administration, completion of which puts the Activity in the completed state. A single shot vaccination or patient taking a single tablet are typical examples. The states `PLANNED`, `POSTPONED`, `ACTIVE`, `SUSPENDED`, each have a `xxx_step` transition which return the state machine to the same state. Workflow steps that cause no transition are mapped to these events and thus leave the Instruction in the same state. An example is a medication review, which will leave the medication in the `ACTIVE` state, if the physician chooses to continue.

In the future, if delegation of an Instruction to another Instruction is required, nesting of a new Instruction state machine within the Active state of a previous one might need to be supported.

The state machine states and transition names are defined in the openEHR terminology groups "Instruction states" and "Instruction transitions" respectively.

8.2.5.6. Instruction Aggregate State

Each Activity within an Instruction constitutes a clinically identifiable medication or therapy of some kind, while the Instruction usually corresponds to a grouping or combination of therapies designed to treat an overall problem. Accordingly, the Instruction can be seen as having an aggregate state derived from the states of the Activities, as shown in the figure below. The rules for entering each aggregate state are indicated in terms of the states of the constituent Activities.

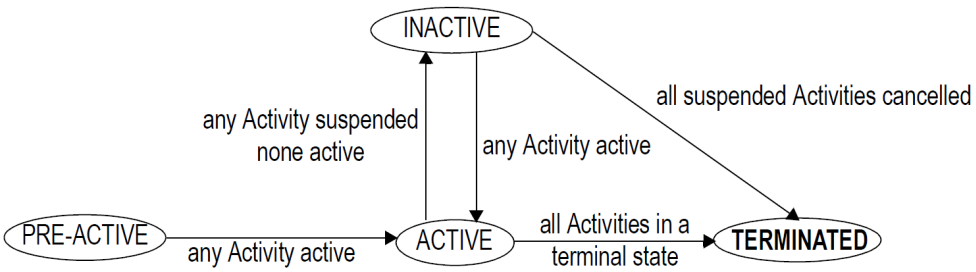


Figure 26. Aggregate Instruction state

This state machine is not formally modelled or coded in openEHR, although it may be useful to do so within an application.

8.2.5.7. Careflow Process to State Machine Mapping

From a health professional's point of view, a healthcare workflow, or 'careflow', consists of steps and events designed to meet one or more goals. The steps are highly dependent on the particular kind of workflow, and will usually be named in terms familiar to the relevant kinds of clinical professionals, such as "prescribe", "book", "suspend" and so on (note that some of these names may be the same as ISM transitions, but may or may not indicate the same thing). However, the need of users of health information is to know what state the execution of an Instruction is in, regardless of which particular careflow step might have just been executed. This is achieved by defining the mapping between the steps of a particular careflow to the states of the ISM in an Action archetype. When each Action corresponding to a particular Instruction Activity is performed, it will be known both which careflow step it corresponds to and which ISM state the Activity is now in. The following table provides an example of the mapping for a UK GP medication order workflow.

UK GP Workflow Step	State machine transition
Start	initiate (initial → planned)
Prescribe	plan_step (planned → planned)
Dispense	start (planned → active)
Administer	active_step (active → active)
Request	Renewal active_step (active → active)
Re-issue	active_step (active → active)
Review	active_step (active → active)

UK GP Workflow Step	State machine transition
Finish	finish (active → completed)
Cancel	abort (active → aborted)

Mappings like this are specified in the `ACTION` archetypes for the Instruction. When an `ACTION` instance is committed to the EHR, the `ISM_TRANSITION` object records the step performed and the ISM state and transition. The careflow step must be one of the steps from the corresponding Instruction.

An optional *`reason`* property (of type `List<DV_TEXT>`) is provided in the `ISM_TRANSITION` class in order to enable one or more specific reasons for the step having occurred to be recorded.

The following section provides details of how archetypes are used to represent such mappings.

8.2.5.8. Relationship to Archetypes

Much of the semantics of particular Instructions and Actions derive from archetypes. Currently, archetypes are used to define two groups of Instruction semantics. The first is the descriptions of activities that are defined in Instructions (`ACTIVITY` . *`description`*) and executed in Actions (`ACTION` . *`description`*). These descriptions are always of the same form for any given Instruction, and it is highly desirable to have the same archetype component for both. An example is where the description is of a medication, commonly consisting of a tree or list of ten or more elements describing the drug, its name, form, dose, route and so on. The same information structure is needed in the Instruction, where it defines what is to be administered, and in the Action, where it describes what has been administered. In any particular instance, there may be small differences in what was administered (e.g. dose or route are modified) even though the archetype model will be the same for both.

In terms of archetypes therefore, definition of say two `ACTIVITYs` in an `INSTRUCTION` (see example illustrated below) will actually create separate archetypes of the Activity structures, each of which will be one of the subtypes of `ITEM_STRUCTURE` (since this is the type of `ACTIVITY` . *`description`* and `ACTION` . *`description`*). The archetypes will then be used by both the `INSTRUCTION` archetype and the `ACTION` archetype, via the archetype slot mechanism (i.e. the standard way of composing archetypes from other archetypes; see `use_archetype` statements in the figure below).

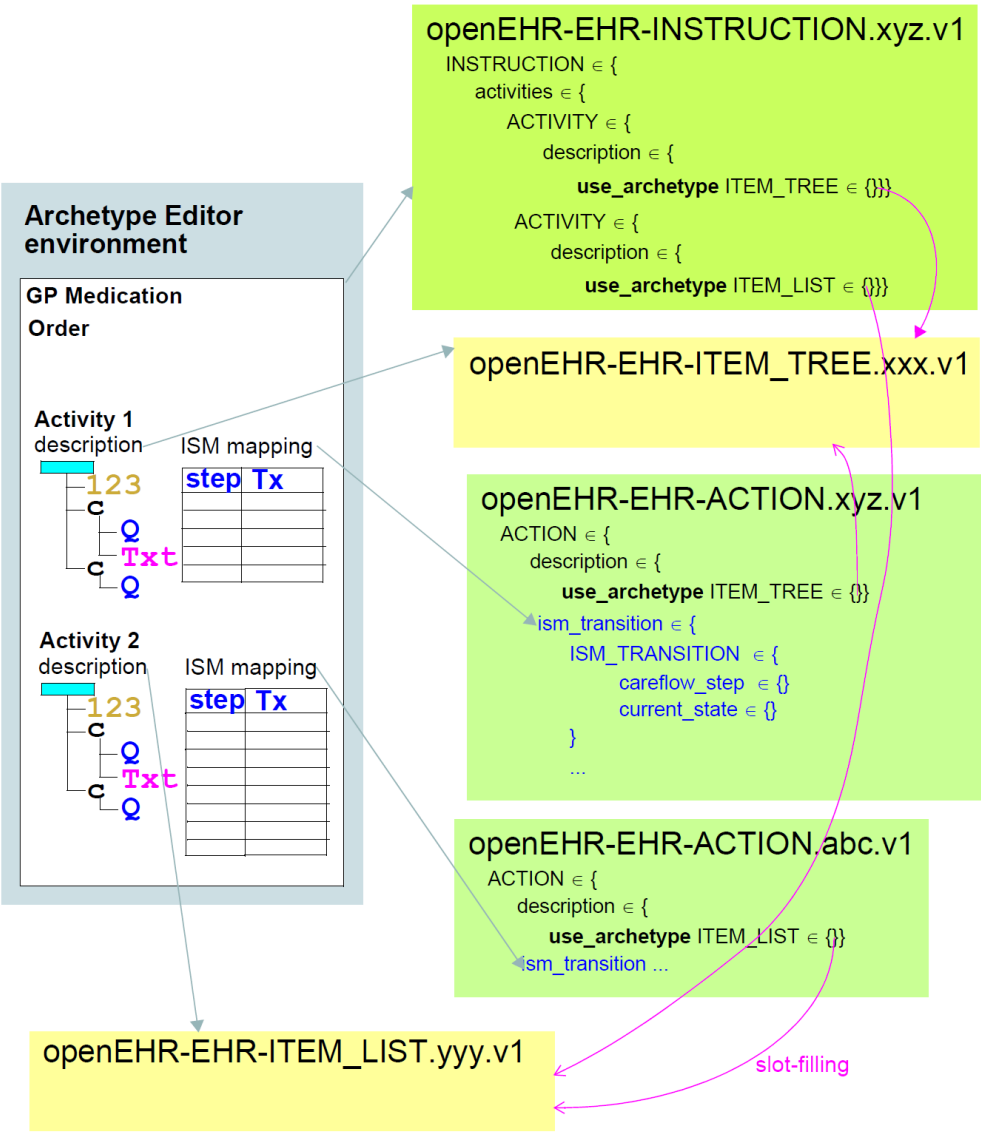


Figure 27. Instruction and Action Archetypes

The second category of archetypable semantics is the correspondence between steps in a healthcare business process and the standard instruction state machine, as described above. This mapping is an archetype of the `ism_transition` attribute of an `ACTION` attribute, and therefore defines part of the `ACTION` archetype. Figure 27 shows how logical archetype elements in the archetyped editor environment corresponds to the resulting archetypes.

Consequently, the set of archetypes defining the clinical semantics of a specific kind of intervention will normally consists of archetypes of `INSTRUCTION`, `ACTION` and potentially component archetypes of the `ITEM_STRUCTURE` classes.

8.2.6. Clinical Workflow Definition

Clinical workflows exist at multiple hierarchical levels, from the health system level (reduce diabetes costs; manage obesity) to the fine-grained (asthmatic medication prescription for a particular patient). At all levels, there are goals, actors and tasks designed to satisfy the goals. At a coarse-grained business process level, workflows may be enacted by more than one actor, and may encompass the whole cycle of Observation, Diagnosis, Instruction and Action. For example a workflow describing the steps "prescribe", "dispense",

"administer", "repeat", "review" and so on, around a medication order might include GP, pharmacist, patient as actors. At the finer level of an actual drug or therapy administration, there is usually a single agent or group that performs a specific task, usually within one provider institution, or at home. The correspondence between workflows at these different levels and particular patients, rather than just categories of patients (e.g. all insulin-dependent diabetics), typically increases at finer levels of granularity. Thus from the point of view of automation, it is likely to be fine-grained workflows that have patient-specific definitions which would reasonably appear in the EHR. Most typical medication administrations are in this category. How automated workflow definitions at higher levels of organisational hierarchy are represented and coordinated with lower-level automated workflows is known to be a difficult problem generally, and given that health computing is generally more complex than most domains, implementation of distributed, coordinated (or "orchestrated" or "choreographed", to use the terms of the workflow community) clinical workflows is likely to be some years off.

In this context, the possible scope for formal workflow definition in openEHR appears to be as follows:

- To enable the recording of links between openEHR Entries and workflow executions, e.g. a particular guideline. This allows openEHR data to be integrated with coarse-grained nonpatient specific workflows.
- To support a standardised, interoperable representation of fine-grained formal workflow definitions for activities like medication administration.
- To use formal workflow definitions only where automation is actually useful, and is likely to be used. The kind of workflows that are likely to be worth automating are those which run over several days, weeks or longer, i.e. where humans might easily forget to perform a step. In these cases, the output of the workflow system will be reminders for humans to do certain things at certain times, rather than direct machine automation of the task. Execution of such workflow definitions will generate entries in "worklists" for staff or other agents to perform. Examples include asthma drug management for a child and PAP recall management.
- To ensure that any workflow definition takes account of other existing clinical activities, i.e. does not attempt to define all activities that might possibly be relevant. A simple example is a workflow for asthma medication administration probably does not need to explicitly model the taking of peak flow measurements, since this would normally be occurring anyway.

There are various technical challenges with proposing a standard workflow formalism for clinical use. Firstly, executable workflow definitions are essentially structured programs, similar to programs in procedural languages, but with the addition of temporal logic operators, including alternative paths, parallel paths, wait operations, and also references to outside data sources and services. Recent work in clinical workflow modelling e.g. Müller (2003), Barretto (2005), Browne (2005) appears to favour a structural (i.e. parse-tree) approach to representation, due to the need to compute potential modifications to an executing workflow, including dropping, replacing and moving nodes. (Whether such "live" modification of executing workflows is realistic from the designer’s point of view might be questionable, since it means that the design of each workflow has include every possible exceptional case at a detailed level.)

Secondly, the need to connect workflows to the outside world, i.e. data sources and services like notification and

worklist management is crucial in making workflows and guidelines implementable. This problem is probably the main weakness of all guideline and workflow languages to date, including Arden, GLIF, and the various workflow languages such as those mentioned earlier.

The approach taken by the current release of openEHR in representing computable workflow is the following.

- Where used at all, formal workflow definitions are expressed in terms of syntax, not structures, since syntax is always a more appropriate representation for persistence (just as object structures, i.e. parse trees are more appropriate for computation).
- Access to patient data items are expressed within the syntax as symbolic queries.
- Actions, such as requests to the notification service are represented as symbolic commands.

The entire definition of a workflow is expressed as an optional parsable string, in the wf_definition: `DV_PARSABLE` attribute of the `INSTRUCTION` class. Any syntax may currently be used. A workflow syntax is under development by the openEHR Foundation, which is designed to incorporate the relevant features of current workflow models and research, while integrating it into the openEHR type system and archetype framework. In particular, early versions of this syntax will show how patient data access and service commands can be expressed.

8.3. Class Descriptions

8.3.1. ENTRY Class

Class	***ENTRY (abstract)***	
Description	The abstract parent of all `ENTRY` subtypes. An `ENTRY` is the root of a logical item of hard clinical information created in the clinical statement context, within a clinical session. There can be numerous such contexts in a clinical session. Observations and other Entry types only ever document information captured/created in the event documented by the enclosing Composition. An `ENTRY` is also the minimal unit of information any query should return, since a whole `ENTRY` (including subparts) records spatial structure, timing information, and contextual information, as well as the subject and generator of the information.	
Inherit	`CONTENT_ITEM`	
Attributes	**Signature**	**Meaning**
1..1	**language**: `CODE_PHRASE`	Mandatory indicator of the localised language in which this Entry is written. Coded from openEHR Code Set languages .

1..1	**encoding**: CODE_PHRASE	Name of character set in which text values in this Entry are encoded. Coded from openEHR Code Set character sets.
0..1	**other_participations**: List<PARTICIPATION>	Other participations at ENTRY level.
0..1	**workflow_id**: OBJECT_REF	Identifier of externally held workflow engine data for this workflow execution, for this subject of care.
1..1	**subject**: PARTY_PROXY	Id of human subject of this ENTRY , e.g.: • organ donor • foetus • a family member • another clinically relevant person.
0..1	**provider**: PARTY_PROXY	Optional identification of provider of the information in this ENTRY , which might be: • the patient • a patient agent, e.g. parent, guardian • the clinician • a device or software Generally only used when the recorder needs to make it explicit. Otherwise, Composition composer and other participants are assumed.
Functions	**Signature**	**Meaning**
1..1	**subject_is_self** (): Boolean *Post_condition*: Result implies subject.generating_type = "PARTY_SELF"	Returns True if this Entry is about the subject of the EHR, in which case the subject attribute is of type PARTY_SELF.
Invariants	*Language_valid*: code_set (Code_set_id_languages).has_code (language)	
	Encoding_valid: code_set (Code_set_id_character_sets).has_code (encoding)	

	Subject_validity: `subject_is_self implies subject.generating_type = "PARTY_SELF"`
	Other_participations_valid: `other_participations /= Void implies not other_participations.is_empty`
	Is_archetype_root: `is_archetype_root`

8.3.2. ADMIN_ENTRY Class

Class	ADMIN_ENTRY	
Description	Entry subtype for administrative information, i.e. information about setting up the clinical process, but not itself clinically relevant. Archetypes will define contained information. Used for administrative details of admission, episode, ward location, discharge, appointment (if not stored in a practice management or appointments system). Not to be used for any clinically significant information.	
Inherit	`ENTRY`	
Attributes	**Signature**	**Meaning**
1..1	**data**: `ITEM_STRUCTURE`	Content of the Admin Entry. The data of the Entry; modelled in archetypes.

8.3.3. CARE_ENTRY Class

Class	*CARE_ENTRY (abstract)*	
Description	The abstract parent of all clinical `ENTRY` subtypes. A `CARE_ENTRY` defines protocol and guideline attributes for all clinical Entry subtypes.	
Inherit	`ENTRY`	
Attributes	**Signature**	**Meaning**

0..1	**protocol**: ITEM_STRUCTURE	Description of the method (i.e. how) the information in this entry was arrived at. For OBSERVATIONs , this is a description of the method or instrument used. For EVALUATIONs , how the evaluation was arrived at. For INSTRUCTIONs , how to execute the Instruction. This may take the form of references to guidelines, including manually followed and executable; knowledge references such as a paper in Medline; clinical reasons within a larger care process.
0..1	**guideline_id**: OBJECT_REF	Optional external identifier of guideline creating this Entry if relevant.

8.3.4. OBSERVATION Class

Class	**OBSERVATION**	
Description	Entry subtype for all clinical data in the past or present, i.e. which (by the time it is recorded) has already occurred. OBSERVATION data is expressed using the class HISTORY<T> , which guarantees that it is situated in time. OBSERVATION is used for all notionally objective (i.e. measured in some way) observations of phenomena, and patient-reported phenomena, e.g. pain. Not to be used for recording opinion or future statements of any kind, including instructions, intentions, plans etc.	
Inherit	CARE_ENTRY	
Attributes	**Signature**	**Meaning**
1..1	**data**: HISTORY<ITEM_STRUCTURE>	The data of this observation, in the form of a history of values which may be of any complexity.
0..1	**state**: HISTORY<ITEM_STRUCTURE>	Optional recording of the state of subject of this observation during the observation process, in the form of a separate history of values which may be of any complexity. State may also be recorded within the History of the data attribute.

8.3.5. EVALUATION Class

Class	EVALUATION	
Description	Entry type for evaluation statements. Used for all kinds of statements which evaluate other information, such as interpretations of observations, diagnoses, differential diagnoses, hypotheses, risk assessments, goals and plans. Should not be used for actionable statements such as medication orders - these are represented using the `INSTRUCTION` type.	
Inherit	`CARE_ENTRY`	
Attributes	Signature	Meaning
1..1	**data**: `ITEM_STRUCTURE`	The data of this evaluation, in the form of a spatial data structure.

8.3.6. INSTRUCTION Class

Class	INSTRUCTION	
Description	Used to specify actions in the future. Enables simple and complex specifications to be expressed, including in a fully-computable workflow form. Used for any actionable statement such as medication and therapeutic orders, monitoring, recall and review. Enough details must be provided for the specification to be directly executed by an actor, either human or machine. Not to be used for plan items which are only specified in general terms.	
Inherit	`CARE_ENTRY`	
Attributes	Signature	Meaning
1..1	**narrative**: `DV_TEXT`	Mandatory human-readable version of what the Instruction is about.
0..1	**expiry_time**: `DV_DATE_TIME`	Optional expiry date/time to assist determination of when an Instruction can be assumed to have expired. This helps prevent false listing of Instructions as Active when they clearly must have been terminated in some way or other.

0..1	**wf_definition**: DV_PARSABLE	Optional workflow engine executable expression of the Instruction.
0..1	**activities**: List<ACTIVITY>	List of all activities in Instruction.
Invariants	*Activities_valid*: `activities /= Void implies not activities.is_empty`	

8.3.7. ACTIVITY Class

Class	**ACTIVITY**	
Description	Defines a single activity within an Instruction, such as a medication administration.	
Inherit	LOCATABLE	
Attributes	**Signature**	**Meaning**
0..1	**timing**: DV_PARSABLE	Timing of the activity, in the form of a parsable string. If used, the preferred syntax is ISO8601 'R' format, but other formats may be used including HL7 GTS. May be omitted if: • timing is represented structurally in the *description* attribute (e.g. via archetyped elements), or • unavailable, e.g. imported legacy data; in such cases, the INSTRUCTION.*narrative* should carry text that indicates the timing of its *activities*.
1..1	**action_archetype_id**: String	Perl-compliant regular expression pattern, enclosed in '//' delimiters, indicating the valid identifiers of archetypes for Actions corresponding to this Activity specification. Defaults to `/.*/`, meaning any archetype.
1..1	**description**: ITEM_STRUCTURE	Description of the activity, in the form of an archetyped structure.
Invariants	*Action_archetype_id_valid*: `not action_archetype_id.is_empty`	

8.3.8. ACTION Class

Class	ACTION	
Description	Used to record a clinical action that has been performed, which may have been ad hoc, or due to the execution of an Activity in an Instruction workflow. Every Action corresponds to a careflow step of some kind or another.	
Inherit	CARE_ENTRY	
Attributes	Signature	Meaning
1..1	**time**: DV_DATE_TIME	Point in time at which this action completed.
1..1	**ism_transition**: ISM_TRANSITION	Details of transition in the Instruction state machine caused by this Action.
0..1	**instruction_details**: INSTRUCTION_DETAILS	Details of the Instruction that caused this Action to be performed, if there was one.
1..1	**description**: ITEM_STRUCTURE	Description of the action that has been performed, in the form of an archetyped structure.

8.3.9. INSTRUCTION_DETAILS Class

Class	INSTRUCTION_DETAILS	
Description	Used to record details of the Instruction causing an Action.	
Inherit	PATHABLE	
Attributes	Signature	Meaning
1..1	**instruction_id**: LOCATABLE_REF	Reference to causing Instruction.
1..1	**activity_id**: String	Identifier of Activity within Instruction, in the form of its archetype path.

0..1	**wf_details**: ITEM_STRUCTURE	Various workflow engine state details, potentially including such things as: • condition that fired to cause this Action to be done (with actual variables substituted); • list of notifications which actually occurred (with all variables substituted); • other workflow engine state. This specification does not currently define the actual structure or semantics of this field.
Invariants	*Activity_path_valid*: `not activity_id.is_empty`	

8.3.10. ISM_TRANSITION Class

Class	**ISM_TRANSITION**	
Description	Model of a transition in the Instruction State Machine, caused by a careflow step. The attributes document the careflow step as well as the ISM transition.	
Inherit	PATHABLE	
Attributes	**Signature**	**Meaning**
1..1	**current_state**: DV_CODED_TEXT	The ISM current state. Coded by openEHR terminology group Instruction states.
0..1	**transition**: DV_CODED_TEXT	The ISM transition which occurred to arrive in the current_state. Coded by openEHR terminology group Instruction transitions.
0..1	**careflow_step**: DV_CODED_TEXT	The step in the careflow process which occurred as part of generating this action, e.g. dispense , start_administration. This attribute represents the clinical label for the activity, as opposed to current_state which represents the state machine (ISM) computable form. Defined in archetype.
0..1	**reason**: List<DV_TEXT>	Optional possibility of adding one or more reasons for this careflow step having been taken. Multiple reasons may occur in medication management for example.

Invariants	*Current_state_valid*: `terminology (Terminology_id_openehr).has_code_for_group_id (Group_id_instruction_states, current_state.defining_code)`
	Transition_valid: `transition /= Void implies terminology (Terminology_id_openehr). has_code_for_group_id (Group_id_instruction_transitions, transition.defining_code)`

8.4. Instance Structures

The following subsections illustrate typical Entry instance structures. For guidance on how to best model particular clinical statements, see the archetype part of the openEHR knowledge repository (https://www.openehr.org/ckm).

8.4.1. OBSERVATION

8.4.1.1. Heartrate Measurement Series

The following figure illustrates three heartrate measurements over 10 minutes.

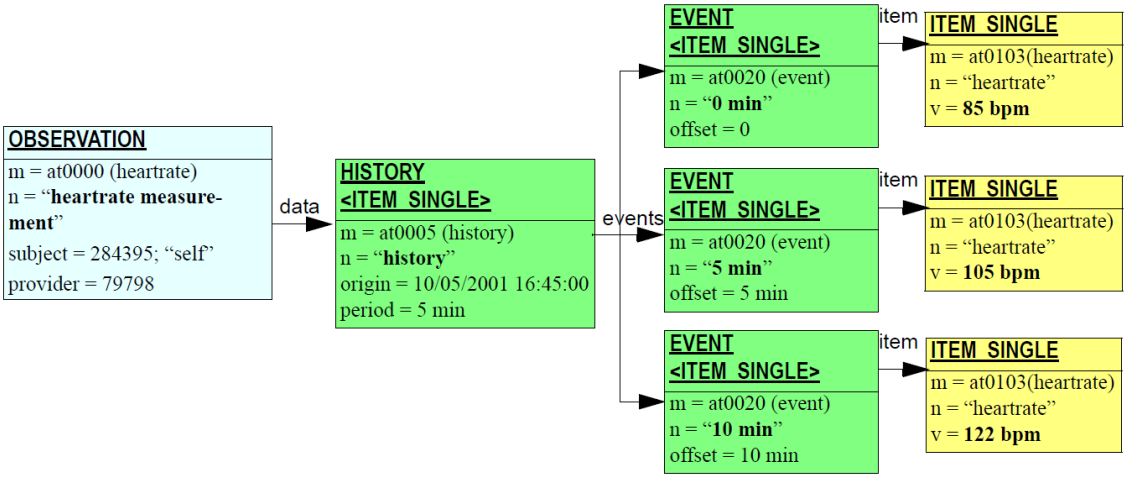


Figure 28. Periodic series instance structure

8.4.1.2. Blood Pressure with Protocol

The following figure illustrates a blood pressure observation with protocol.

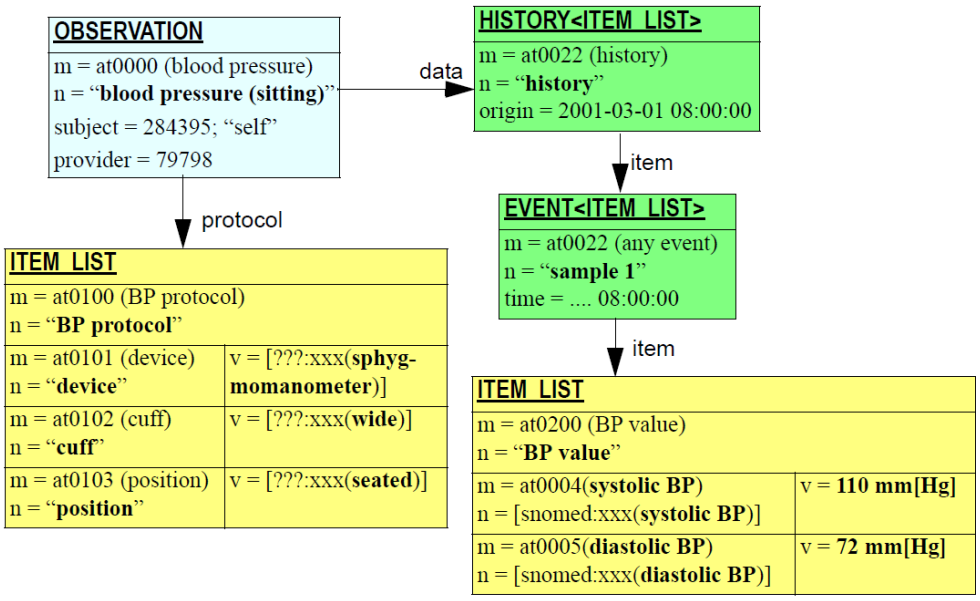


Figure 29. Blood Pressure Measurement Observation

8.4.1.3. Glucose Tolerance Test

An oral glucose tolerance test takes the following form, although the number and timing of the blood sugar levels may be slightly different in practice:

- challenge: no calories fasting from 12pm to 8am
- datum: BSL - 8am
- challenge: 75 g glucose orally - 8:01 am
- datum: BSL - 9 am
- datum: BSL - 10 am

OGTT is treated as a single clinical concept, and thus requires only one archetype. A typical instance structure is shown in the figure below. In this example, the three blood sugars are represented by `EVENTs` , with the fasting and glucose challenges being expressed as states on the relevant events.

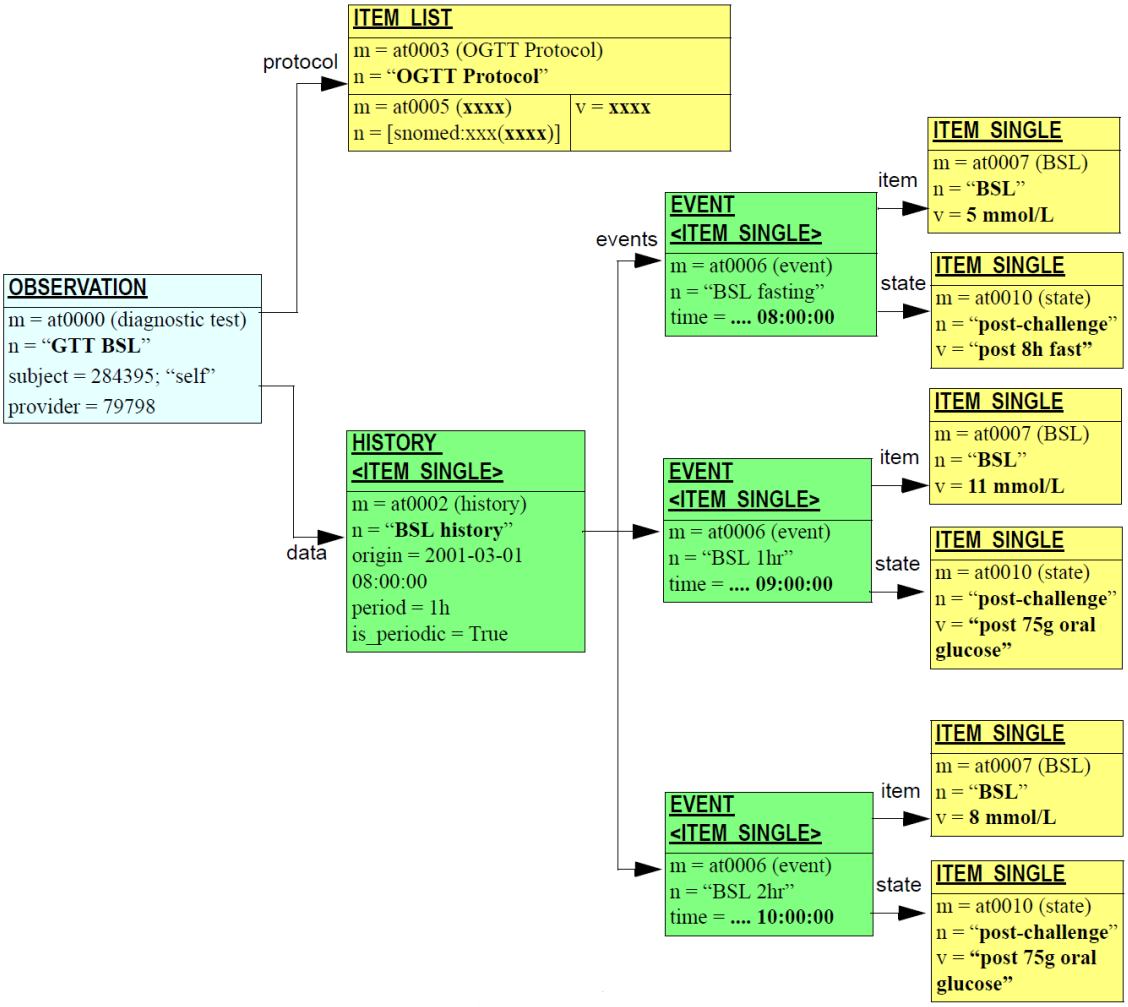


Figure 30. OGTT instance structure

8.4.2. EVALUATION

8.4.2.1. Partial Asthma Management Plan

The following figure illustrates a partial asthma management plan in which monitoring (peak flow) with dependent actions (review and admission to ER) and therapy (bronchodilator) are shown. In a complete plan, symptom monitoring and other medications might be shown. The parts of the plan are linked to the root `EVALUATION` node via the *`links`* : `Set<LINK>` attribute inherited from the `LOCATABLE` class.

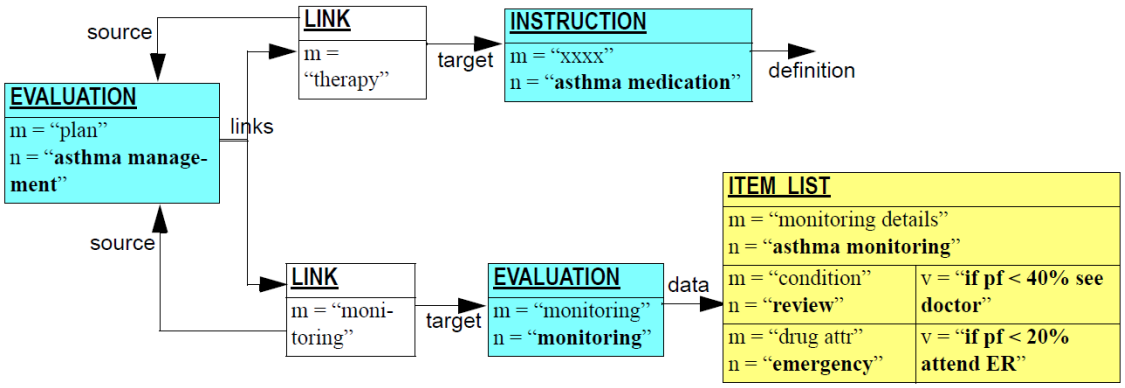


Figure 31. Partial Asthma Management Plan

8.4.3. INSTRUCTION

8.4.3.1. Chained Medication Order

Often, a medication order for one drug consists of segments in which one or more of the administration details of route, form, frequency, dose etc is changed. In hospitals, intravenous antibiotics and pain relief drugs may be followed by a tablet form of the same drug to be taken orally. Other examples are common in general practice, such as the following order:

- trade name = Panafcortelone; generic name = Prednisolone; form = tablets; dose = 25mg; route = oral; freq = bd x 3 days; od x 2 days.

The figure below illustrates the instance structure for this Instruction. Note that the timing attribute of the `ACTIVITY` instance is shown in human-readable form; in reality it will be a GTS string or similar (see Timing Specification section of openEHR Data Types IM).

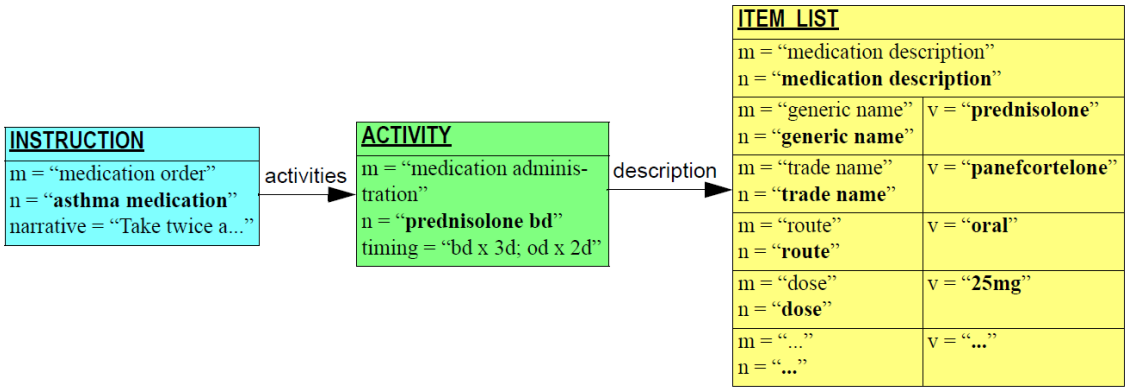


Figure 32. Chained medication order

8.4.3.2. Multi-drug Therapy

A common regime for treating duodenal ulcer and related complaints is using Losec with other drugs, such as in the following combination:

- Losec 40 mg od x 4w or until no symptoms
- amoxicillin 500 mg 3td x 7d
- metronidizole 400 mg 3td x 7d

The Instruction for this therapy is illustrated below.

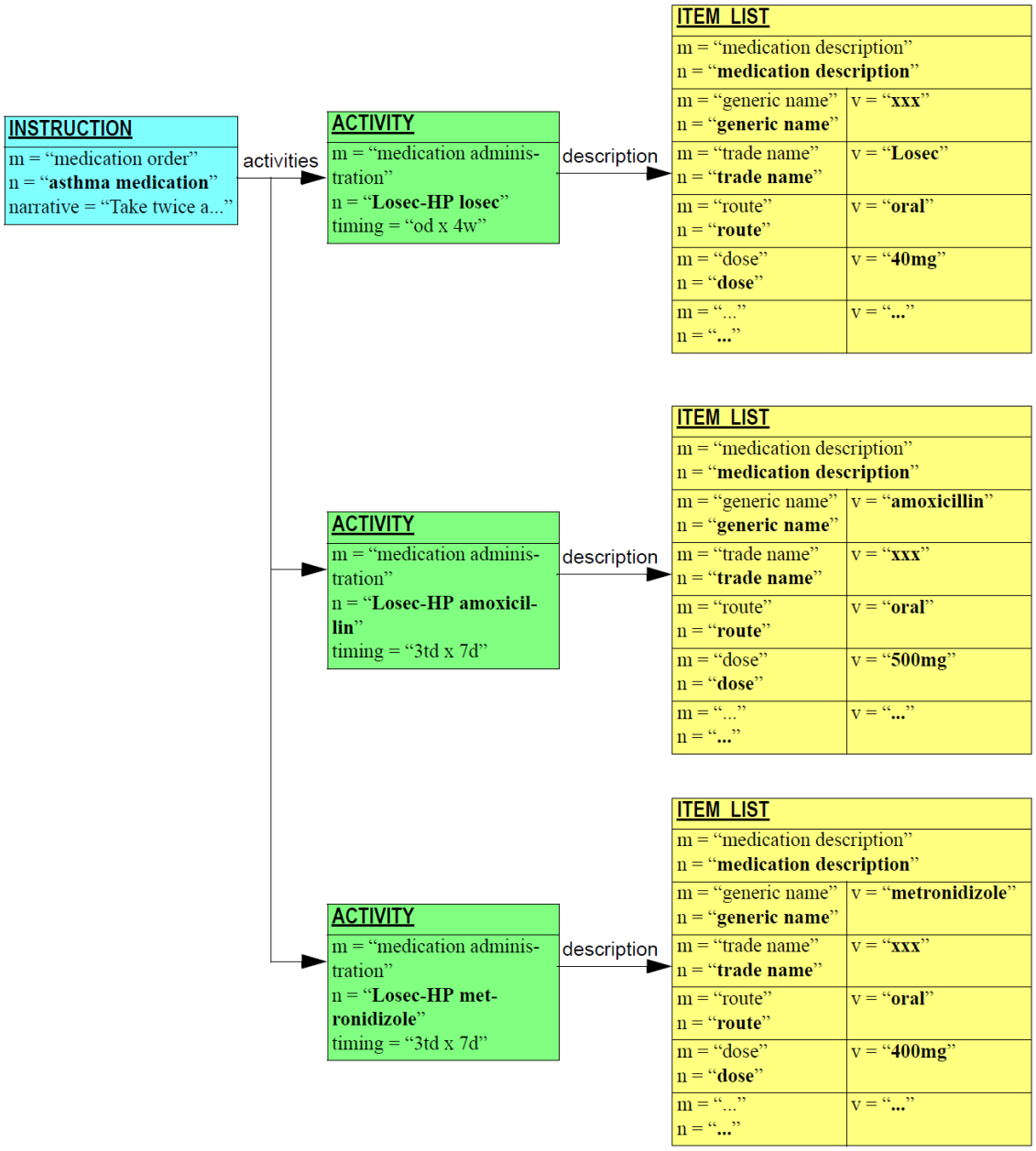


Figure 33. Multi-drug therapy Instruction

Appendix A: Glossary

openEHR Terms	
HCA	Health care agent - any doctor, nurse or other recognised staff member, or software or device
HCF	Health care facility - any place where EHRs are kept
HCP	Health care professional - any doctor, nurse or other recognised staff member of an HCF
Clinical Terms	
Care Pathway	A global care management strategy for a patient, showing management of health problems or issues in a time-based framework, similar to a project management view of an engineering work.
Episode	A series of clinical events linked in time, such as a hospital admission or a surgical episode.
Issue	A problem as identified by the patient, e.g. "inability to do exercise due to breathing difficulty"; may be the object of wider health care, e.g. social workers, physiotherapists etc.
Problem	A health problem of the patient, as identified by its underlying medical cause, e.g. asthma; the object of medical care.
IT Terms	
API	Application programmer's interface - the software interface to a library or module.
COM	Microsoft's Component Object Model; designed to enable integration of binary components obeying stated exported interfaces.
CORBA	Common Object Request Broker Architecture - an object-oriented middleware architecture enabling the construction of 3-tier systems, in which backend data providers (DBMSs etc) are known only by the services they export to the network. CORBA is an open standard managed by the Object Management Group (OMG).
DCOM	Distributed version of Microsoft COM. Similar in its aim to CORBA.

ODMG-93	A standard for object databases, which includes an object definition language (ODL) for writing schemas, an object query language (OQL) for querying, and several language bindings.

References

Arp, R., Smith, B., & Spear, A. D. (2015). *Building Ontologies with Basic Formal Ontology*. The MIT Press. Retrieved from https://mitpress.mit.edu/books/building-ontologies-basic-formal-ontology

Barretto, S. (2005). *Designing Guideline-based Workflow-Integrated Electronic Health Records* (PhD thesis). University of South Australia, Adelaide. Retrieved from https://wwww.openehr.org/publications/workflow/Barretto_PhD_Thesis_2005.zip

Beale, T. (2000). Archetypes: Constraint-based Domain Models for Future-proof Information Systems. Retrieved from https://www.openehr.org/publications/archetypes/archetypes_beale_web_2000.pdf

Beale, T., & Heard, S. (2000). *GEHR Technical Requirements*. Australia: GEHR Foundation. Retrieved from https://www.openehr.org/resources/related_projects/gehr_requirements.pdf

Beale, T., & Heard, S. (2007). An Ontology-based Model of Clinical Information. In K. K. et al. (Ed.), *Proceedings MedInfo 2007* (pp. 760–764). IOS Publishing. Retrieved from http://www.openehr.org/publications/health_ict/MedInfo2007-BealeHeard.pdf

Beale, T., Lloyd, D., Heard, S., Kalra, D., & Ingram, D. (1995). *Deliverable 19,20,24: GEHR Architecture*. Brussels: European Commission AIM. Retrieved from https://www.openehr.org/resources/related_projects/gehr_deliverable-19_20_24.pdf

Browne, E. (2005, January). *Workflow Modelling of Coordinated Inter-Health-Provider Care Plans* (PhD thesis). School of Computer and Information Science, University of South Australia, Adelaide. Retrieved from https://wwww.openehr.org/publications/workflow/Eric_Browne_WF_thesis_2005.pdf

Dixon, R., Grubb, P. A., & Lloyd, D. (2000). *EHCR Support Action Deliverable 3.5: 'Final Recommendations to CEN for future work.'* Brussels: European Commission DGXIII.

Dixon, R., Grubb, P. A., & Lloyd, D. (2000). *EHCR Support Action Deliverable 2.4 'Guidelines on Interpretation and implementation of CEN EHCRA.'* Brussels: European Commission DGXIII.

Dixon, R., Grubb, P. A., Lloyd, D., & Kalra, D. (1998). *EHCR Support Action Deliverable 3.1, 3.2 'Interim Report to CEN.'* Brussels: European Commission DGXIII.

Dixon, R., Grubb, P. A., Lloyd, D., & Kalra, D. (2001). *Consolidated List of Requirements. EHCR Support Action Deliverable 1.4.* Brussels: European Commission DGXIII. Retrieved from https://discovery.ucl.ac.uk/id/eprint/1603/1/R5.pdf

Elstein, A. S., & Schwarz, A. (2002). Clinical problem solving and diagnostic decision making: selective review of the cognitive literature. *BMJ*, *324*, 729–732.

Elstein, A. S., Shulman, L. S., & Sprafka, S. A. (1978). *Medical problem solving: an analysis of clinical reasoning*. Cambridge, MA: Harvard University Press.

Fowler, M. (1997). *Analysis Patterns: Reusable Object Models*. Addison Wesley.

Gray, J., & Reuter, A. (1993). *Transaction Processing Concepts and Techniques*. Morgan Kaufmann.

Grimson, W., & Groth, T. (1996). *The Synapses User Requirements and Functional Specification (Part B)*. Brussels: EU Telematics Application Programme.

Heard, S. (2001). *GEHR Project Australia, GPCG Trial*. Australia: GEHR Foundation. Retrieved from https://www.openehr.org/resources/related_projects

Heard, S., Kalra, D., & Ingram, D. (1994). *Deliverable 8: Ethical and legal Requirements of GEHR Architecture and Systems*. Brussels: European Commission AIM. Retrieved from https://www.openehr.org/resources/related_projects/gehr_deliverable-8.pdf

Ingram, D. (1995). The Good European Health Record Project. In M. F. Laires, M. J. Laderia, & J. P. Christensen (Eds.), . Amsterdam: IOS Press.

Kalra, D. (1996). *The Synapses User Requirements and Functional Specification (Part A)*. Brussels: EU Telematics Application Programme.

Kalra, D. (1998). *Synapses ODP Information Viewpoint*. Brussels: EU Telematics Application Programme. Retrieved from http://discovery.ucl.ac.uk/66235/

Lloyd, D. (1992). *Deliverable 4: GEHR Requirements for Clinical Comprehensiveness*. Brussels: European Commission AIM. Retrieved from https://www.openehr.org/resources/related_projects/gehr_deliverable-4.pdf

Lloyd, D. (1993). *Deliverable 7: Clinical Functional Specifications*. Brussels: European Commission AIM.

Müller, R. (2003, January). *Event-Oriented Dynamic Adaptation of Workflows: Model, Architecture, Implementation* (PhD thesis). Fakultät für Mathematik und Informatik at the Universität Leipzig, Leipzig. Retrieved from https://dbs.uni-leipzig.de/de/publication/title/event_oriented_dynamic_adaptation_of_workflows_model_architecture_and_implementation

Sottile, P. A., Ferrara, F. M., Grimson, W., Kalra, D., & Scherrer, J. R. (1999). The holistic healthcare information system. In *Toward an Electronic Health Record Europe* (pp. 259–266). TEHRE.

Sowa, J. F. (2000). *Knowledge Representation: Logical, philosophical and Computational Foundations*. California: Brooks/Cole.

Velde, R. van de, & Degoulet, P. (2003). *Clinical Information Systems: A Component-Based Approach*. New York: Springer-Verlag.

Weed, L. L. (1969). *Medical records, medical education and patient care* (Sixth.). Chicago: Year Book Medical Publishers Inc.

Last updated 2021-02-15 13:27:48 UTC